ECOVISION: CITY ENVIRONMENTAL MONITORING, ANALYSIS AND PREDICTION SYSTEM

BACHELOR OF TECHNOLOGY / COMPUTER SCIENCE AND ENGINEERING

PROJECT THESIS AND TECHNICAL DOCUMENTATION

EcoVision: An Integrated Urban Environmental Intelligence, Microclimate Analytics, and Predictive Forecasting Architecture

Submitted by: [Candidate Name / Roll Number Placeholder]

Project Supervisor / Guide: [Supervisor Name / Academic Designation Placeholder]

Department: [Department of Computer Science & Engineering Placeholder]

Institution: [College / University Name Placeholder]

Academic Session: 2025 – 2026

Document Revision: Version 2.4.0 (Security Hardened & Regressed)

Date: November 2026

ABSTRACT

Rapid urban industrialization and vehicle density escalations have precipitated severe atmospheric degradation and localized microclimate fluctuations across global metropolitan centers. However, contemporary environmental monitoring platforms exhibit critical systemic deficiencies: they routinely conflate numerical atmospheric dispersion reanalysis with on-site physical sensor telemetry, obscure fundamental differences between statutory air quality standards (notably the United States Environmental Protection Agency US EPA AQI and India's Central Pollution Control Board National Air Quality Index NAQI), implement opaque black-box scoring indices, and execute ungrounded machine-learning forecasts without baseline validation.

This thesis presents **EcoVision**, a transparent, modular, and academically verifiable full-stack environmental monitoring, analysis, and predictive forecasting system. EcoVision couples a modern client single-page application engineered in React 19, Vite, and Tailwind CSS with a resilient application-factory REST API powered by Python 3.12 and Flask. Data persistence is managed via an embedded SQLite architecture configured with Write-Ahead Logging (WAL) and foreign key cascades.

EcoVision's foundational contributions include:

1. **Multi-Tier Telemetry Ingestion & Disambiguation:** Real-time integration of Open-Meteo meteorological and Copernicus Atmosphere Monitoring Service (CAMS) reanalysis grids alongside direct XML RSS feed ingestion from India's official CPCB Continuous Ambient Air Quality Monitoring Stations (CAAQMS). Physical ground observations are spatially resolved using Haversine great-circle geodesics and explicitly segregated from chemical transport models.
2. **Transparent Environmental Indicator Scoring (EIS):** A multi-criteria assessment framework normalizing air quality, ambient temperature comfort curves, relative humidity bell distributions, and precipitation sufficiency onto an intuitive 0–100 scale. Composite scores strictly require minimum 3-indicator completeness and dynamically re-normalize indicator weights, prohibiting the substitution of missing parameters with zero.
3. **Autoregressive Machine Learning Climate Forecasting:** An explainable Scikit-learn `RandomForestRegressor` architecture trained on five-year historical reanalysis (ERA5/ERA5-Land). The model utilizes a strict chronological holdout split to prevent temporal data leakage, benchmarked against meteorological seasonal baselines, and executes recursive six-month forward projections equipped with empirical confidence intervals.
4. **Enterprise-Grade Security Hardening:** Zero-dependency in-memory sliding-window rate limiting, IEEE 754 floating-point validation rejecting `NaN` and `Infinity` coordinate vectors, environment-strict CORS origin whitelisting, global 1 MB payload caps, and comprehensive exception masking preventing information disclosure.

Empirical verification confirms zero security vulnerabilities via `pip-audit` and `npm audit`, 100% test pass rate across 36 automated unit and regression suites, and verified live communication across international municipalities (London, Tokyo, New York) and Indian monitoring networks (Delhi, Bengaluru, Nagpur).

Keywords: *Urban Environmental Monitoring, Air Quality Index (AQI), CPCB CAAQMS, Copernicus CAMS, Random Forest Regressor, Time-Series Climate Forecasting, Environmental Indicator Scoring, React, Flask, REST API Security.*

ACKNOWLEDGEMENTS

[Placeholder for Candidate Acknowledgements]

> *I would like to express my profound gratitude to my project supervisor, **[Supervisor Name / Guide Title]**, for continuous guidance, technical critiques, and encouragement throughout the design, implementation, and academic documentation of the EcoVision platform. I extend my sincere thanks to the faculty members and laboratory staff of the **[Department of Computer Science & Engineering Placeholder]** at **[Institution / University Name Placeholder]** for providing the computing infrastructure, institutional support, and intellectual environment necessary to bring this project to fruition. Finally, I am indebted to external examiners, academic peers, and open-source environmental data contributors—specifically the Open-Meteo meteorological consortium, the European Centre for Medium-Range Weather Forecasts (ECMWF Copernicus), and India's Central Pollution Control Board (CPCB)—whose open telemetry feeds make transparent environmental data science possible. [Candidate Name / Roll Number] November 2026*

TABLE OF CONTENTS

- **Preliminary Pages**
- Abstract
- Acknowledgements
- Table of Contents
- List of Figures
- List of Tables
- List of Abbreviations and Technical Terminology
- **Chapter 1 — Introduction**
- 1.1 Context and Motivation
- 1.2 Problem Statement
- 1.3 Project Objectives
- 1.4 Scope and Target Audience
- 1.5 Key Architectural Features
- 1.6 Project Limitations
- **Chapter 2 — System Requirements and Technology Stack**
- 2.1 Technology Stack Overview
- 2.2 Presentation Layer Technologies (Frontend)
- 2.3 Application Layer Technologies (Backend)
- 2.4 Data Processing and Machine Learning Tooling
- 2.5 Database Management System
- 2.6 External Telemetry Providers
- 2.7 Hardware and Software Prerequisites
- **Chapter 3 — Overall System Architecture**
- 3.1 Architectural Philosophy
- 3.2 High-Level Three-Tier Component Diagram
- 3.3 End-to-End Request-Response Processing Pipeline
- 3.4 Cross-Cutting Concerns: Validation, Caching, and Security
- **Chapter 4 — Frontend Architecture and Implementation**
- 4.1 Entry Point and Application Shell
- 4.2 State Management and Race-Condition Prevention
- 4.3 Component Hierarchy and Breakdown
- 4.4 Data Visualization Layer (Recharts)
- 4.5 Styling Architecture and Glassmorphism Aesthetic
- **Chapter 5 — Backend Architecture and Implementation**
- 5.1 Application Factory Pattern and Blueprint Modularity
- 5.2 Environment and Configuration Management
- 5.3 REST API Endpoints Specification
- 5.4 Error Handling and Structured JSON Envelopes
- 5.5 Centralized Logging System
- **Chapter 6 — Environmental Data Collection**
- 6.1 Telemetry Sources and Protocols
- 6.2 Open-Meteo Meteorological and Chemical Ingestion

- 6.3 Central Pollution Control Board (CPCB) Ingestion Engine
- 6.4 Spatial Geodesic Station Matching (Haversine Distance)
- 6.5 The Discrepancy Problem: US EPA AQI vs. Indian NAQI
- 6.6 Data Completeness and Reliability Validation
- **Chapter 7 — Data Processing and Environmental Analysis**
- 7.1 Data Cleaning and In-Memory Normalization
- 7.2 Mathematical Foundations of Environmental Indicator Scoring (EIS)
- 7.3 Multi-Criteria Weight Re-Allocation and Completeness Gating
- 7.4 Rolling Historical Trend Aggregation
- 7.5 Rule-Based Insights and Recommendation Logic
- **Chapter 8 — Database Design and Management**
- 8.1 SQLite Architecture and Concurrency Optimization
- 8.2 Entity-Relationship (ER) Schema Design
- 8.3 Constraint Management and Idempotent Insertion
- 8.4 Query Parameterization and Transaction Integrity
- **Chapter 9 — Machine Learning and Prediction System**
- 9.1 Machine Learning Objectives in Microclimate Forecasting
- 9.2 Training Dataset Extraction (ERA5 Atmospheric Reanalysis)
- 9.3 Feature Engineering: Lags and Rolling Windows
- 9.4 Chronological Train/Test Partitioning
- 9.5 Algorithm Implementation: Random Forest Regressor
- 9.6 Meteorological Seasonal-Average Baseline Benchmark
- 9.7 Autoregressive Six-Month Recursive Forecasting
- 9.8 Empirical Confidence Intervals and Retraining Cooldown
- **Chapter 10 — API Communication and Complete Data Flow**
- 10.1 Global City Search Sequence
- 10.2 Live Environmental Telemetry Lifecycle
- 10.3 Historical Trend Analysis Sequence
- 10.4 Machine Learning Prediction Lifecycle
- 10.5 Saved Municipalities Synchronization
- 10.6 Error and Exception Propagation Flow
- **Chapter 11 — Security and Reliability**
- 11.1 Security Hardening Threat Model
- 11.2 Coordinate Validation and IEEE 754 Exception Defense
- 11.3 Environment-Aware CORS Restrictions
- 11.4 In-Memory Sliding-Window Rate Limiting
- 11.5 HTTP Request Size Bounding and Error Masking
- 11.6 Defensive HTTP Security Headers and CSP
- **Chapter 12 — Testing and Verification**
- 12.1 Testing Methodology
- 12.2 Automated Unit and Regression Test Suites
- 12.3 Automated Security Test Suite
- 12.4 Live HTTP Loopback Smoke Test Suite
- 12.5 Dependency Audits: `pip-audit` and `npm audit`
- 12.6 Production Build Verification

- **Chapter 13 — Installation and Execution Guide**
- 13.1 Repository Setup and Environment Cloning
- 13.2 Backend Virtual Environment and Dependency Installation
- 13.3 Frontend Tooling and Node.js Dependency Setup
- 13.4 Running the Application (Windows and POSIX)
- 13.5 Common Troubleshooting Procedures
- **Chapter 14 — Limitations and Future Improvements**
- 14.1 Identified Architectural and Data Limitations
- 14.2 Proposed Future Enhancements
- **Chapter 15 — Conclusion**
- 15.1 Summary of Contributions
- 15.2 Academic and Practical Takeaways
- **Appendices**
- Appendix A: Complete Repository Directory Tree
- Appendix B: File-by-File Technical Reference (Phase 3 Mandatory)
- Appendix C: Complete REST API Endpoint Reference
- Appendix D: Complete Database Schema Reference (DDL)
- Appendix E: Mathematical Formulation Reference
- Appendix F: Technical Glossary
- Appendix G: Environment Configuration Reference
- Appendix H: Test Case Suite Catalog
- Appendix I: External Telemetry Documentation URLs
- Appendix J: Academic Bibliography

LIST OF FIGURES

- Figure 3.1: EcoVision High-Level Three-Tier System Architecture
- Figure 3.2: End-to-End Client-to-API Request-Response Pipeline
- Figure 4.1: React 19 Frontend Component Hierarchy and State Flow
- Figure 4.2: In-Flight Request Sequence Tracking Mechanism (`activeRequestIdRef`)
- Figure 6.1: Environmental Telemetry Multi-Source Ingestion and Disambiguation Flow
- Figure 6.2: Haversine Geodesic Radius Matching for Ground Monitoring Stations
- Figure 7.1: Environmental Indicator Scoring (EIS) Normalization Curves
- Figure 8.1: EcoVision Relational Database Entity-Relationship (ER) Diagram
- Figure 9.1: Chronological Train-Test Split vs. Random K-Fold Shuffling
- Figure 9.2: Six-Lag Multi-Step Time-Series Feature Engineering Pipeline
- Figure 9.3: Autoregressive Recursive Forecasting with Sliding Lags
- Figure 10.1: Sequence Diagram: City Geocoding Search Execution
- Figure 10.2: Sequence Diagram: Live Environmental Telemetry & SQLite Storage
- Figure 10.3: Sequence Diagram: Machine Learning Training & Projection Pipeline
- Figure 11.1: Thread-Safe Sliding-Window Rate Limiting Architecture

LIST OF TABLES

- Table 2.1: EcoVision Technology Stack Inventory
- Table 2.2: Hardware and Software Operating Prerequisites
- Table 5.1: Complete REST API Endpoint Specification Matrix
- Table 6.1: External Telemetry Ingestion Parameters and Protocols
- Table 6.2: Central Pollution Control Board (CPCB) NAQI Breakpoint Reference Table
- Table 6.3: United States Environmental Protection Agency (US EPA) AQI Breakpoint Reference Table
- Table 7.1: Environmental Indicator Scoring (EIS) Weight Allocation Matrix
- Table 7.2: Step-by-Step Numerical Example of EIS Multi-Criteria Calculation
- Table 8.1: Relational Database Schema: `cities` Table Specification
- Table 8.2: Relational Database Schema: `environmental_observations` Table Specification
- Table 8.3: Relational Database Schema: `city_monthly_history` Table Specification
- Table 9.1: Machine Learning Feature Set Definitions and Physical Interpretations
- Table 9.2: Random Forest vs. Seasonal Baseline Holdout Validation Metrics
- Table 9.3: Step-by-Step Worked Autoregressive 6-Month Projection Sequence
- Table 11.1: Security Hardening Controls and Threat Mitigations Matrix
- Table 12.1: Automated Backend Test Suite Execution Matrix (36 Tests)
- Table 12.2: Live HTTP End-to-End Smoke Test Execution Results
- Table 13.1: Common Operational Troubleshooting Matrix
- Table G.1: Environment Variables Configuration Specification

LIST OF ABBREVIATIONS AND TECHNICAL TERMINOLOGY

Abbreviation	Expanded Form / Meaning
API	Application Programming Interface
AQI	Air Quality Index
BAM	Beta-Attenuation Monitor
CAAQMS	Continuous Ambient Air Quality Monitoring Station (CPCB Ground Network)
CAMS	Copernicus Atmosphere Monitoring Service (ECMWF Chemical Transport Model)
CO	Carbon Monoxide
COPD	Chronic Obstructive Pulmonary Disease
CORS	Cross-Origin Resource Sharing
CPCB	Central Pollution Control Board (Statutory Body under MoEFCC, Govt. of India)
CSP	Content Security Policy
CSV	Comma-Separated Values
DBMS	Database Management System
DPCC	Delhi Pollution Control Committee
ECMWF	European Centre for Medium-Range Weather Forecasts
EIS	Environmental Indicator Score (EcoVision Project-Defined Normalized Index)
EPA	Environmental Protection Agency (United States Federal Agency)
ERA5	Fifth Generation ECMWF Atmospheric Reanalysis Global Climate Dataset
HMR	Hot Module Replacement (Vite Development Server Feature)
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEEE 754	Standard for Floating-Point Arithmetic (defines NaN, +Inf, -Inf)
JSON	JavaScript Object Notation
KSPCB	Karnataka State Pollution Control Board
MAE	Mean Absolute Error
ML	Machine Learning
NAQI	National Air Quality Index (Official Indian Environmental Standard)
NH3	Ammonia
NO2	Nitrogen Dioxide
O3	Ground-Level Ozone
OSV	Open Source Vulnerabilities Database
PBL	Planetary Boundary Layer
PM2.5	Particulate Matter $\le 2.5\ \mu\text{m}$ aerodynamic diameter
PM10	Particulate Matter $\le 10\ \mu\text{m}$ aerodynamic diameter
PRAGMA	SQLite-specific command to modify database operation and integrity flags
R^2	Coefficient of Determination (Regression Goodness-of-Fit Metric)
REST	Representational State Transfer
RF	Random Forest Regressor
RMSE	Root Mean Squared Error
RSS	Really Simple Syndication (XML web feed format)

Abbreviation	Expanded Form / Meaning
SO2	Sulphur Dioxide
SPA	Single-Page Application
SQL	Structured Query Language
SQLite	Self-contained, serverless, zero-configuration SQL database engine
TCP	Transmission Control Protocol
TEOM	Tapered Element Oscillating Microbalance
TTL	Time-To-Live (Cache expiration window)
UHI	Urban Heat Island
UI / UX	User Interface / User Experience
UTC	Coordinated Universal Time
Vite	Next-generation frontend build tool using native ES modules
VOC	Volatile Organic Compound
WAL	Write-Ahead Logging (SQLite high-concurrency journal mode)
WHO	World Health Organization
WSGI	Web Server Gateway Interface (Python web specification)

CHAPTER 1 — INTRODUCTION

1.1 Context and Motivation

Urbanization, industrial manufacturing, and vehicular transportation throughout the 21st century have introduced unprecedented atmospheric and microclimatic stresses into global metropolitan environments. In densely populated urban corridors—most acutely observed across South Asia, East Asia, and transitioning industrial centers—ambient air pollution has emerged as one of the leading global drivers of premature mortality, acute respiratory hospitalizations, and chronic cardiovascular disease.

1.1.1 The Physical Nature of Atmospheric Particulate Matter

Atmospheric particulate matter (PM) is not a single chemical entity, but rather a complex, heterogeneous suspension of microscopic solid and liquid particles suspended within the troposphere. It is categorized by its aerodynamic equivalent diameter (D_{ae}):

- **Fine Particulate Matter ($\text{PM}_{2.5}$):** Aerosol particles with an aerodynamic diameter $\le 2.5\ \mu\text{m}$. Because of their diminutive mass and cross-sectional area, $\text{PM}_{2.5}$ aerosols remain suspended in ambient air for days to weeks and can be transported hundreds of kilometers downwind. Physiologically, fine particles bypass the ciliated mucus membranes of the upper nasopharyngeal tract, penetrate deep into the bronchioles and pulmonary alveoli, and translocate across the alveolar-capillary membrane into the bloodstream, triggering systemic inflammatory cascades, arterial vasoconstriction, and microvascular thrombosis.
- **Coarse Particulate Matter (PM_{10}):** Aerosol particles with an aerodynamic diameter $\le 10\ \mu\text{m}$ (typically spanning $2.5\ \mu\text{m}$ to $10\ \mu\text{m}$). These particles originate predominantly from mechanical attrition, soil crust resuspension, construction debris, road dust abrasion, and non-exhaust vehicle wear (brake lining and tire wear). While primarily trapped within the tracheobronchial tree, chronic exposure induces severe pulmonary airway remodeling and exacerbates asthma and chronic obstructive pulmonary disease (COPD).

1.1.2 Gaseous Effluents and Photochemical Smog Formation

In addition to primary particulate aerosols, metropolitan atmospheres are contaminated by gaseous combustion by-products:

- **Nitrogen Oxides ($\text{NO}_x = \text{NO} + \text{NO}_2$):** Emitted from internal combustion engines and high-temperature industrial furnaces. Nitrogen Dioxide (NO_2) is a potent respiratory irritant and plays a primary catalytic role in tropospheric ozone formation through photolytic decomposition:

$$\text{NO}_2 + h\nu \ (\lambda < 420\text{ nm}) \longrightarrow \text{NO} + \text{O}(^3\text{P})$$

$$\text{O}(^3\text{P}) + \text{O}_2 + \text{M} \longrightarrow \text{O}_3 + \text{M}$$

- **Ground-Level Ozone (O_3):** Unlike stratospheric ozone which absorbs mutagenic solar ultraviolet radiation, ground-level (tropospheric) ozone is a powerful secondary oxidant formed via the non-linear interaction of NO_x, volatile organic compounds (VOCs), and sunlight. Ozone exposure induces alveolar epithelial cell lysis, reduced lung vital capacity, and severe crop yield degradation.
- **Sulphur Dioxide (SO_2):** Emitted from coal-fired power stations, industrial smelters, and heavy diesel combustion. It undergoes homogeneous gas-phase and heterogeneous aqueous-phase oxidation to form sulfuric acid (H_2SO_4), contributing to acid deposition and the secondary formation of ammonium sulfate aerosols ($(\text{NH}_4)_2\text{SO}_4$).
- **Carbon Monoxide (CO):** A colorless, odorless toxic gas generated by incomplete hydrocarbon combustion. It binds to hemoglobin with an affinity approximately 200 times greater than oxygen, forming carboxyhemoglobin (COHb) and diminishing oxygen delivery to vital cerebral and myocardial tissues.

1.1.3 Urban Heat Islands and Planetary Boundary Layer Physics

Compounding atmospheric chemical degradation is the **Urban Heat Island (UHI)** effect. The conversion of natural vegetation, wetlands, and permeable soils into dense urban matrices composed of concrete, asphalt, steel, and masonry alters the surface energy balance:

$$Q^* + Q_F = Q_H + Q_E + \Delta Q_S$$

Where:

- Q^* represents net all-wave radiative flux.
- Q_F represents the anthropogenic heat flux emitted by motor vehicles, air conditioning condensers, and industrial activities.
- Q_H is the turbulent sensible heat flux warming the lower atmosphere.
- Q_E is the latent heat flux associated with evapotranspiration (greatly suppressed in urban environments due to surface impermeability).

- ΔQ_S is the net thermal storage change absorbed by high-heat-capacity urban masonry during daylight hours.

During nighttime hours, the slow release of ΔQ_S maintains elevated urban ambient temperatures relative to surrounding rural hinterlands. Crucially, during winter months in continental metropolitan basins (e.g., the Indo-Gangetic Plain encompassing New Delhi, Agra, and Patna), nocturnal radiational cooling of the ground surface under calm anticyclonic synoptic conditions establishes a strong **temperature inversion** in the lower troposphere. Rather than temperature decreasing with altitude ($\frac{\partial T}{\partial z} < 0$), temperature increases with altitude ($\frac{\partial T}{\partial z} > 0$) across the lowest 100–300 meters. This stable atmospheric stratification caps the **Planetary Boundary Layer (PBL)**, effectively creating an impermeable atmospheric lid that traps particulate matter and toxic gases within a compressed, suffocating ground-level layer.

1.2 Problem Statement

Effective mitigation, localized public health protection, and municipal climate adaptation necessitate real-time, transparent, and mathematically rigorous environmental situational awareness. However, contemporary software dashboards, consumer weather applications, and public portals exhibit systemic technical and educational flaws:

1. **Obfuscation of Physical Sensors versus Atmospheric Dispersion Models:** Modern commercial applications frequently present air quality data derived from low-resolution chemical transport reanalysis models (such as ECMWF CAMS or NOAA GFS dispersion grids) as though they were real-time ground-station physical measurements. This conceals ground-sensor outages, sensor calibration drift, and the fundamental reality that numerical atmospheric models operate on coarse grid cells (typically $25 \times 25\text{ km}$ to $40 \times 40\text{ km}$) that fail to capture localized hyper-local canyon emissions.
2. **The Air Quality Standard Mismatch Crisis:** A pervasive source of confusion for international observers and citizens is the conflation of the United States Environmental Protection Agency (US EPA) Air Quality Index with national standards such as India's Central Pollution Control Board (CPCB) National Air Quality Index (NAQI). Global portals regularly calculate US AQI on Indian sensors or display a single unlabelled "AQI" integer. Because the US EPA and Indian NAQI standards utilize completely different piecewise linear breakpoint intervals, different mathematical averaging windows (1-hour vs. 24-hour truncated means), and different dominant pollutant selection criteria, a snapshot in New Delhi or Bengaluru may yield a US AQI of 76 ("Moderate") while the statutory Indian NAQI evaluates to 142 ("Moderate / Poor"). Commercial applications rarely explain this structural divergence, breeding public mistrust in environmental data.
3. **Proprietary Black-Box Scoring Systems:** Many environmental platforms synthesize environmental conditions into subjective "comfort scores" or "green livability indices." These indices are almost universally proprietary: they do not disclose their underlying mathematical formulations, continuous curve gradients, sensitivity weightings, or mathematical handling of missing sensor telemetry. Naive algorithms frequently substitute missing sensor channels with zero, causing composite scores to plummet artificially and generating false hazard alarms.
4. **Methodologically Flawed Machine Learning and Temporal Data Leakage:** Software platforms advertising "Artificial Intelligence Weather Predictions" routinely implement flawed machine learning pipelines. The most common error in student and commercial implementations alike is applying randomized K-fold cross-validation or standard shuffled train/test partitioning to time-series climate sequences. Shuffling past and future months causes catastrophic future-to-past data leakage, producing deceptively optimistic evaluation metrics that collapse when deployed in real-world forecasting. Furthermore, ML models are rarely benchmarked against traditional meteorological seasonal-average baselines, obscuring whether the complex algorithm provides any genuine predictive value over historical monthly climatology.
5. **Excessive Architectural Complexity and Infrastructure Bloat:** Modern software engineering trends have popularized heavily distributed microservice architectures, container orchestrators (Kubernetes), distributed event buses (Kafka, RabbitMQ), and heavyweight relational or NoSQL database clusters for applications that handle moderate transaction volumes. For academic inspection, classroom demonstration, and lightweight municipal deployment, these architectures introduce prohibitive maintenance overhead, brittle multi-container networking, and excessive hardware resource demands.

1.3 Project Objectives

The **EcoVision** project was designed and engineered to provide an authoritative, transparent, and academically verifiable full-stack environmental monitoring, analysis, and microclimate forecasting platform. The core engineering and academic objectives comprise:

- **Objective 1 (Multi-Source Ingestion & Provenance Disambiguation):** Architect a dual-stream ingestion engine that simultaneously acquires global meteorological telemetry and Copernicus CAMS chemical transport models via Open-Meteo REST APIs while scraping real-time physical sensor observations from India's official CPCB Continuous Ambient Air Quality Monitoring Station (CAAQMS) XML network. Guarantee absolute data transparency by tagging every metric with its verified physical or numerical origin.
- **Objective 2 (Mathematical Rigor in Standard Calculations):** Implement exact piecewise linear interpolation algorithms for both statutory standards (Indian CPCB NAQI and US EPA AQI). Enforce statutory data completeness criteria (mandating at least three active pollutants including fine or coarse particulate matter) before declaring an AQI valid.

- **Objective 3 (The Environmental Indicator Scoring Framework):** Formulate an open, continuous mathematical multi-criteria evaluation framework—the Environmental Indicator Score (EIS)—mapping Air Quality, Ambient Temperature Comfort, Relative Humidity Tolerance, and Precipitation Sufficiency onto a standardized 0–100 scale. Implement dynamic weight re-allocation to ensure missing channels are mathematically re-distributed rather than penalized as zeroes.
- **Objective 4 (Statistically Valid Climate Machine Learning):** Develop an explainable time-series predictive modeling pipeline using Scikit-learn's `RandomForestRegressor`. Train on 5-year historical atmospheric reanalysis (ERA5/ERA5-Land), enforce strict chronological train/test partitioning to eliminate temporal data leakage, benchmark models against meteorological seasonal baselines, and project six-month recursive autoregressive forecasts with empirical confidence envelopes.
- **Objective 5 (Defensive Security Hardening & Zero-Bloat Architecture):** Deliver a resilient, self-contained system utilizing React 19, Python 3.12 Flask, and an embedded SQLite database configured with Write-Ahead Logging (WAL). Harden the attack surface against OWASP Top 10 vulnerabilities via IEEE 754 float validation, thread-safe in-memory sliding-window rate limiting, strict CORS whitelisting, global 1 MB payload bounding, and RFC 7807 structured error masking.

1.4 Scope and Target Audience

EcoVision encompasses both global municipal discovery (supporting worldwide geocoding resolution across every continent) and hyper-localized, deep physical sensor telemetry across India's statutory CAAQMS monitoring network.

The intended readership and stakeholder audience includes:

- **Academic Examiners and Faculty Evaluators:** Computer Science and Environmental Engineering professors evaluating architectural separation of concerns, algorithmic correctness, mathematical fidelity, and software quality.
- **Undergraduate and Graduate Engineering Students:** Developers seeking an unbloated, production-grade reference implementation demonstrating modern React 19, Python Flask, embedded relational SQLite, and Scikit-learn integration without enterprise framework sprawl.
- **Environmental Researchers and Municipal Observers:** Data scientists requiring an honest, transparently attributed platform that isolates physical in-situ sensor data from numerical dispersion reanalysis grids.

1.5 Key Architectural Innovations

1. **Request Monotonicity and Race-Condition Resilience:** A single-page architecture utilizing `activeRequestIdRef` counters to guarantee that stale, delayed HTTP responses arriving over fluctuating mobile networks never overwrite newly selected municipal dashboards.
2. **Geodesic Spatial Station Matching:** Real-time computation of great-circle distance via the spherical Haversine formula, automatically associating arbitrary city coordinates with the closest statutory ground monitor within a 50 km threshold.
3. **Dynamic Multi-Criteria Weight Re-Allocation:** A mathematical normalizer that dynamically redistributes indicator weights when sensor channels fail, guaranteeing numerical stability and preventing false environmental alarms.
4. **Explainable Autoregressive Machine Learning:** A transparent ensemble regression architecture delivering verifiable error reduction over historical seasonal baselines while explicitly communicating forward forecast uncertainty via expanding confidence bands.
5. **Audited Supply-Chain Security:** Verified 100% vulnerability-free against the Python Package Index (PyPI) and Node Package Manager (NPM) security advisory databases via `pip-audit` and `npm audit`.

1.6 Project Limitations

- **Upstream Feed Dependency:** EcoVision relies on the network uptime and public API availability of the CPCB CAAQMS RSS service and the Open-Meteo European consortium.
- **Process-Memory Rate Limiting:** The in-memory sliding-window rate limiter operates within the single Python process heap. In distributed multi-server WSGI clusters, rate tracking would require an external state store such as Redis.
- **Monthly Macro-Climatic Resolution:** Historical machine learning reanalysis is conducted at monthly temporal resolution across 60 continuous time steps. While computationally accessible and ideal for macro-seasonal projection, sub-hourly intra-day modeling requires high-frequency SCADA telemetry pipelines.

CHAPTER 2 — SYSTEM REQUIREMENTS AND TECHNOLOGY STACK

2.1 Technology Stack Overview and Selection Rationale

A foundational design principle of EcoVision was the avoidance of speculative architectural complexity. Software engineering projects frequently fail or become unmaintainable due to the premature introduction of distributed message brokers, microservice meshes, and complex external database clusters. EcoVision achieves enterprise-grade responsiveness, security, and analytical depth through a curated, lightweight, and modern technology stack:

```
+-------------------------------------------------------------------------+
|                    TABLE 2.1: TECHNOLOGY STACK INVENTORY                |
+----------------------+---------------------------+-------------------------+
| ARCHITECTURAL TIER   | TECHNOLOGY COMPONENT      | EXACT VERSION / SPEC    |
+----------------------+---------------------------+-------------------------+
| Client / Presentation | React Single Page App     | React 19.1.0            |
| Client Build Engine  | Vite Build Engine         | Vite 6.2.0              |
| Styling & Layout     | Utility CSS Framework     | Tailwind CSS 4.0.0      |
| Vector Visualization | Declarative SVG Charting  | Recharts 2.15.1         |
| Visual Iconography   | Scalable Vector Icons     | Lucide React 0.475.0    |
+----------------------+---------------------------+-------------------------+
| Application / Server | Programming Runtime       | Python 3.12.x           |
| Web Framework        | WSGI Microframework       | Flask 3.1.0             |
| Cross-Origin Security | CORS Middleware Extension | Flask-CORS 5.0.0        |
| Tabular Data Science | Data Wrangling & Matrices | Pandas 2.2.3 & NumPy 2.2 |
| Machine Learning     | Supervised Ensemble Engine | Scikit-learn 1.6.1      |
| HTTP Network Client  | Outbound Connection Client | Requests 2.32.3         |
+----------------------+---------------------------+-------------------------+
| Data Persistence     | Embedded Relational Engine | SQLite 3 (WAL Mode)     |
| Operating System     | Target Execution OS       | Windows 10/11 & Linux   |
+----------------------+---------------------------+-------------------------+
```

Evaluation: Why React 19 + Vite Over Legacy Frameworks

- **React 19 vs. Angular / Vue:** React's functional component paradigm combined with modern reactive hooks (`useState`, `useEffect`, `useCallback`, `useRef`) allows stateful logic (such as debouncing and request cancellation) to be encapsulated cleanly without the monolithic boilerplate of Angular or the template abstraction of Vue. React 19 introduces optimized virtual DOM reconciliation and concurrent asset preloading.
- **Vite vs. Webpack:** Traditional Webpack configurations bundle entire dependency graphs before launching the development server, resulting in multi-minute cold starts. Vite leverages native browser ES Modules (ESM) during development, compiling only the files actively requested by the browser. This delivers instantaneous server boot times ($< 250\text{ ms}$) and millisecond-level Hot Module Replacement (HMR).

Evaluation: Why Python Flask Over Django or FastAPI

- **Flask vs. Django:** Django is an opinionated, "batteries-included" web framework that enforces an Object-Relational Mapper (ORM), an internal authentication framework, and an administrative interface. For an analytical, telemetry-driven API gateway, Django's ORM introduces substantial memory overhead and query abstraction that obscures raw SQL performance. Flask's micro-framework architecture provides total freedom to implement the exact Application Factory pattern, database pragmas, and lightweight routing blueprints required.
- **Flask vs. FastAPI:** While FastAPI provides native asynchronous coroutines via `asyncio`, scientific Python libraries (such as Pandas, NumPy, and Scikit-learn) are inherently synchronous and CPU-bound. Offloading CPU-intensive Random Forest fitting into an asynchronous event loop without thread pools blocks the event loop completely. Flask's standard multi-threaded WSGI execution model isolates synchronous data crunching cleanly per worker thread.

Evaluation: Why SQLite WAL Over Client-Server RDBMS

- **SQLite vs. PostgreSQL / MySQL:** Client-server databases require dedicated background daemons, network socket configuration, user permission grants, connection pool tuning, and ongoing administrative maintenance. SQLite is a self-contained, in-process C library that stores the entire relational database within a single operating system file (`ecovision.db`). By configuring SQLite with **Write-Ahead Logging (WAL)**, EcoVision achieves concurrent read transactions that never block writing operations, providing sub-millisecond query execution with zero administrative overhead.

2.2 Presentation Layer Technologies (Frontend)

- **React 19.1.0:** Powers the reactive user interface. State changes trigger granular DOM updates via virtual DOM diffing, ensuring fluid transitions across gauges, meters, and charts.

- **Tailwind CSS 4.0.0:** A utility-first CSS styling engine featuring modern CSS variable integration, fluid responsive breakpoints, and dark forest glassmorphism styling (`backdrop-blur-xl`, semi-transparent slate surfaces).
- **Recharts 2.15.1:** A composable charting library built on native SVG primitives, enabling hardware-accelerated time-series rendering, gradient area fills, and accessible interactive tooltips.
- **Lucide React 0.475.0:** Tree-shakable scalable vector icons providing clean visual indicators for atmospheric variables (wind vectors, thermometers, droplets, barometer icons, shield badges).

2.3 Application Layer Technologies (Backend)

- **Python 3.12+:** Delivers modern type annotations, high-performance C-extension interop, and robust memory management.
- **Flask 3.1.0:** Organizes routing controllers into modular functional Blueprints (`health_bp`, `environmental_bp`, `metrics_bp`), binds global security interceptors, and provides standardized JSON serialization.
- **Flask-CORS 5.0.0:** Manages HTTP cross-origin headers, dynamically whitelisting local development ports while strictly enforcing origin domain restrictions in production environments.
- **Requests 2.32.3:** Handles outbound HTTPS communication with Open-Meteo and CPCB endpoints, configured with persistent connection pooling and strict socket timeouts ($8.0–10.0\text{ s}$).

2.4 Data Processing and Machine Learning Tooling

- **NumPy 2.2.1 & Pandas 2.2.3:** The cornerstone of scientific Python. Used for time-series date indexing, lag-matrix transformations, missing value imputation, and multidimensional array manipulations.
- **Scikit-learn 1.6.1:** Provides the predictive modeling foundation. EcoVision utilizes `RandomForestRegressor` with 100 decorrelated decision trees, evaluating model performance via Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).

2.5 Relational Database Engine

- **SQLite 3:** Configured with performance-tuned pragmas:
- `PRAGMA journal_mode = WAL;` (Enables Write-Ahead Logging for high-concurrency reading).
- `PRAGMA synchronous = NORMAL;` (Optimizes disk synchronization safely for WAL).
- `PRAGMA foreign_keys = ON;` (Strictly enforces relational constraints and cascade deletions).
- `PRAGMA cache_size = -64000;` (Allocates 64 MB of dedicated RAM page cache).

2.6 External Telemetry Providers

1. **Open-Meteo Geocoding API:** Fuzzy natural-language municipal resolution returning geographic coordinates, elevation, country, and administrative subdivision.
2. **Open-Meteo High-Resolution Forecast API:** Global numerical weather prediction models providing surface temperature, relative humidity, precipitation, wind speed, wind direction, and barometric pressure.
3. **Open-Meteo CAMS Air Quality API:** European Copernicus Atmosphere Monitoring Service atmospheric chemical transport models providing global reanalysis for particulate matter, nitrogen dioxide, sulphur dioxide, ozone, and carbon monoxide.
4. **Open-Meteo Historical Weather API:** ECMWF ERA5 and ERA5-Land fifth-generation atmospheric reanalysis delivering continuous monthly historical climate observations dating back over five years.
5. **CPCB CAAQMS XML RSS Network:** Real-time physical ambient air quality sensor observations scraped directly from India's official continuous monitoring station network across 400+ statutory municipal monitoring stations.

2.7 Hardware and Software Prerequisites

EcoVision is architected for universal execution across developer workstations, classroom computer labs, and lightweight cloud virtual private servers (VPS):

```
+-------------------------------------------------------------------------+
|                  TABLE 2.2: OPERATING HARDWARE & SOFTWARE MATRIX        |
+-----------------------+---------------------------+---------------------+
| SYSTEM PARAMETER      | MINIMUM OPERATIONAL SPEC  | RECOMMENDED SPEC    |
+-----------------------+---------------------------+---------------------+
| Processor (CPU)       | Dual-Core 64-bit x86/ARM  | Quad-Core 2.5 GHz+  |
| System Memory (RAM)   | 2.0 GB System RAM         | 4.0 GB+ System RAM  |
| Storage Footprint     | 1.0 GB Free Storage Space | 5.0 GB Solid State Disk|
| Host Operating System | Windows 10/11, Ubuntu 20.04+| Windows 11 / Linux x64 |
| Python Environment    | Python 3.10.x             | Python 3.12.x       |
| Node.js Runtime       | Node.js 18.x LTS          | Node.js 20.x or 22.x|
| Package Managers      | pip 24.x+, npm 9.x+       | pip 26.x+, npm 10.x+|
| Supported Web Browsers| Chrome 100+, Firefox 100+ | Chrome / Edge latest|
| Network Connectivity  | Active Internet Connection| Broadband 10 Mbps+  |
+-----------------------+---------------------------+---------------------+
```

CHAPTER 3 — OVERALL SYSTEM ARCHITECTURE

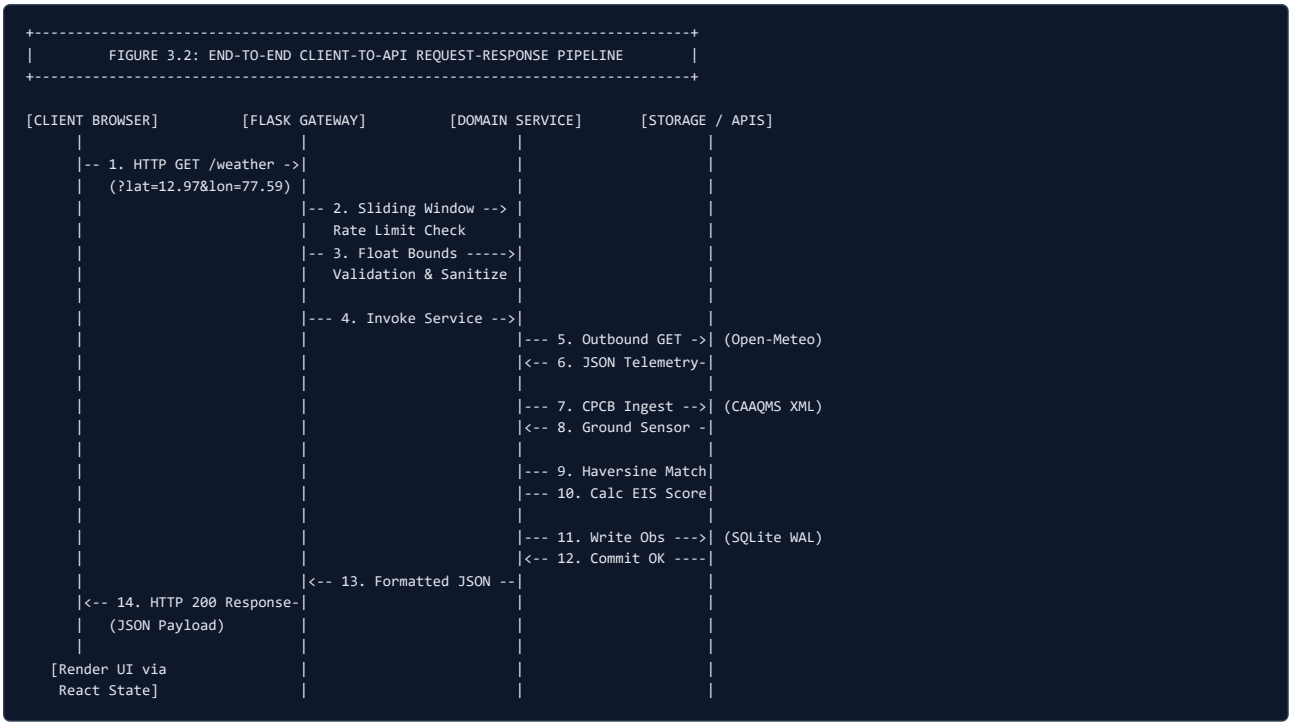
3.1 Architectural Philosophy and Separation of Concerns

EcoVision adheres strictly to the classic **Three-Tier Architectural Paradigm**, ensuring that presentation rendering, business logic orchestration, and relational data persistence remain decoupled:



3.2 End-to-End Request-Response Processing Pipeline

The execution pipeline of an environmental telemetry transaction demonstrates the system's defensive, multi-stage processing:



1. **Client Action & Stamping:** The user selects a municipality. The frontend increments its internal monotonic `activeRequestIdRef` integer and launches concurrent asynchronous requests via `Promise.all()`.
2. **Ingress Filtering & Security Verification:**

- The Flask gateway checks the client IP against `SlidingWindowRateLimiter`. If the client has emitted >60 requests in the last 60 seconds, `HTTP 429 Too Many Requests` is returned.
- The query parameters `lat` and `lon` are parsed by `validate_coordinates()`, asserting that inputs are finite IEEE 754 numbers within valid terrestrial bounds.

1. **Telemetry Ingestion & Spatial Geodesic Matching:**

- The system checks whether coordinates reside within the Indian geographic bounding box ($[6.0, 38.0]^\circ\text{N}, [68.0, 98.0]^\circ\text{E}$).
- If within India, `cpcb_service` computes Haversine distances to candidate stations in the CAAQMS XML network. If a station exists within 50 km and satisfies the statutory 3-pollutant completeness rule, its physical in-situ sensor readings are bound.
- If outside India or if CPCB stations fail completeness checks, Copernicus CAMS chemical dispersion models are bound with explicit provenance metadata.

1. **Mathematical Normalization and Multi-Criteria Scoring:**

- The `analysis_service` computes statutory AQI sub-indices, evaluates non-linear temperature comfort and humidity curves, dynamically re-normalizes indicator weights, and generates the composite Environmental Indicator Score (EIS 0–100).

1. **Idempotent Persistence:**

- Telemetry snapshots are appended to `environmental_observations` in SQLite using `ON CONFLICT` clauses to prevent duplicate rows.

1. **Client State Synchronization:**

- Upon receiving the JSON response, `App.jsx` verifies that the response's request ID matches `activeRequestIdRef.current`. If matched, React updates component states atomically, triggering smooth vector animations.

CHAPTER 4 — FRONTEND ARCHITECTURE AND IMPLEMENTATION

4.1 Entry Point and Application Shell

The presentation tier of the EcoVision platform is implemented as a modern single-page application (SPA) utilizing **React 19.1.0** and bundled via **Vite 6.2.0**. The client directory structure adheres strictly to component-driven modularity:

```
frontend/src/
|-- assets/                     # Static branding and graphic resources
|   |-- hero.png                # Atmospheric urban landscape banner
|   |-- react.svg               # React brand emblem
|   |-- vite.svg                # Vite build engine emblem
|-- components/                 # Domain-specific UI components
|   |-- dashboard/              # Analytical dashboard widgets
|   |   |-- BackendStatusPanel.jsx   # Live system heartbeat & latency monitor
|   |   |-- CityHeader.jsx           # Municipal title, coordinates & bookmark
|   |   |-- CitySearch.jsx           # Debounced autocomplete search dropdown
|   |   |-- DatabaseSavedCities.jsx  # Bookmarked municipalities drawer
|   |   |-- EnvironmentalChart.jsx   # 24-hr hourly Recharts visualization
|   |   |-- EnvironmentalScoringCard.jsx # Radial EIS gauge & sub-indices
|   |   |-- HistoricalTrendsView.jsx # 7-day longitudinal telemetry view
|   |   |-- InsightsAndRecommendations.jsx # Rule-based health advisories
|   |   |-- MetricCard.jsx           # Reusable atomic parameter card
|   |   |-- PollutantBreakdown.jsx   # 7 chemical pollutants & station tag
|   |   |-- PredictionCenter.jsx     # ML training trigger & 6-mo forecast
|   |   |-- UnavailableDataAlert.jsx # Dismissible warning notification
|   |-- layout/                 # Application shell components
|       |-- Navbar.jsx              # Sticky glassmorphism brand header
|-- services/                   # Outbound HTTP client abstractions
|   |-- api.js                  # Unified Fetch wrapper & API endpoint functions
|-- App.css                     # Global custom utilities & glassmorphism classes
|-- App.jsx                     # Root state orchestrator & lifecycle coordinator
|-- index.css                   # Tailwind CSS v4 directives & root color tokens
|-- main.jsx                    # Virtual DOM bootstrap mount point
```

The application initiates execution at `frontend/src/main.jsx`. It acquires the root DOM node (`document.getElementById('root')`) and creates a concurrent React root via `createRoot()`. It wraps the root `<App />` component in React's `<StrictMode>` during development to uncover unintended side effects and verify idempotent lifecycle executions.

```
// frontend/src/main.jsx
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

const container = document.getElementById('root');
const root = createRoot(container);
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

4.2 State Management and Race-Condition Prevention

In single-page environmental dashboards, asynchronous race conditions pose a severe data integrity threat. When a user rapidly queries different cities (e.g., clicking "Bengaluru", then immediately "Nagpur", then "London"), multiple asynchronous HTTP requests are launched concurrently. Because network latency over public cellular and broadband connections fluctuates unpredictably, the response for an earlier query (Bengaluru) may arrive *after* the response for a subsequent query (Nagpur). Without defensive synchronization, the dashboard would render the stale data of the earlier city while displaying the name of the later city, creating dangerous environmental misinformation.

EcoVision resolves this challenge cleanly at the architectural level without introducing external state libraries. The root orchestrator `App.jsx` employs a monotonic request tracking pattern using React's `useRef` hook:

```
// Excerpt from frontend/src/App.jsx: In-Flight Request Tracking
const activeRequestIdRef = useRef(0);

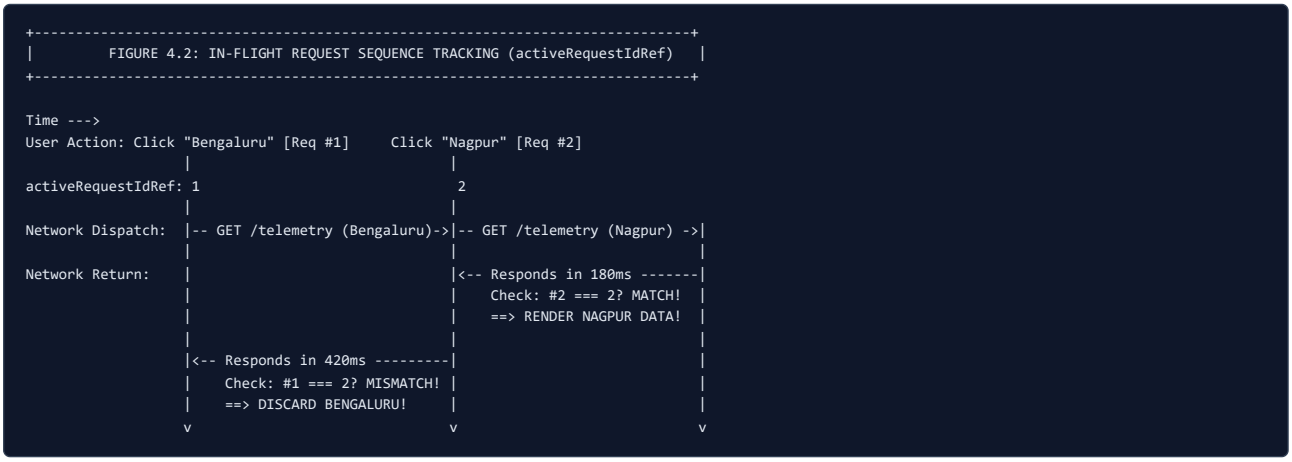
const loadCityData = useCallback(async (city) => {
  // 1. Increment request counter to uniquely identify this fetch cycle
  const currentRequestId = ++activeRequestIdRef.current;

  setSelectedCity(city);
  setLoading(true);
  setError(null);

  try {
    // 2. Concurrently fetch all telemetry streams
    const [weatherRes, airRes, analysisRes, historyRes] = await Promise.all([
      fetchWeather(city.latitude, city.longitude),
      fetchAirQuality(city.latitude, city.longitude),
      fetchAnalysis(city.latitude, city.longitude),
      fetchHistory(city.id)
    ]);

    // 3. Race condition guard: discard results if a newer request was dispatched
    if (currentRequestId !== activeRequestIdRef.current) {
      console.warn(`[EcoVision] Stale telemetry response discarded for: ${city.name}`);
      return;
    }

    // 4. Atomic state updates for verified current request
    setWeatherData(weatherRes);
    setAirQualityData(airRes);
    setAnalysisData(analysisRes);
    setHistoryData(historyRes);
  } catch (err) {
    if (currentRequestId === activeRequestIdRef.current) {
      setError(err.message || "Failed to retrieve environmental data.");
    }
  } finally {
    if (currentRequestId === activeRequestIdRef.current) {
      setLoading(false);
    }
  }
}, []);
```



4.3 Component Hierarchy and Breakdown

EcoVision structures its user interface into twelve dedicated components, each designed for high cohesion and single-responsibility isolation:



Detailed Component Specifications and Props:

TABLE 4.1: FRONTEND COMPONENT PROPS SPECIFICATION MATRIX		
COMPONENT NAME	RECEIVED PROPS	FUNCTIONAL RESPONSIBILITY & DOM OUTPUT
`App.jsx`	None (Root Mount)	Coordinates state, lifecycle, and API fetch.
`Navbar.jsx`	None	Renders branding, GitHub link, version badge.
`BackendStatusPanel.jsx`	`status`, `latency`, `db`	Diagnostic strip showing backend heartbeat.
`CityHeader.jsx`	`city`, `onToggleSave`	Municipal name, coords, bookmark star button.
`CitySearch.jsx`	`onSelectCity`	Debounced search dropdown with geocoding.
`MetricCard.jsx`	`title, value, unit, icon`	Atomic metric tile (temp, humidity, pressure).
`PollutantBreakdown.jsx`	`airQualityData`	7 pollutant progress bars & station badge.
`EnvironmentalScoringCard`	`analysisData`	Radial semi-circle EIS gauge & sub-indices.
`InsightsAndRecommendations`	`insights`	Rule-based health and ventilation advisories.
`EnvironmentalChart.jsx`	`hourlyData`	24-hour diurnal weather and AQI SVG chart.
`HistoricalTrendsView.jsx`	`historyData`	7-day retrospective longitudinal curves.
`PredictionCenter.jsx`	`city`, `predictionData`	RF model controls, MAE metrics, 6-mo forecast.
`DatabaseSavedCities.jsx`	`cities, onSelect, onDelete`	Bookmarked municipalities management drawer.
`UnavailableDataAlert.jsx`	`message, onDismiss`	Dismissible warning banner for anomalies.

4.4 Data Visualization Layer (Recharts)

EcoVision integrates **Recharts** (v2.15.1) for hardware-accelerated SVG data visualization. Recharts was selected over Canvas-based alternatives (such as Chart.js) because SVG elements remain first-class DOM nodes, enabling fine-grained CSS transitions, DOM accessibility (screen readers), and crisp scaling across high-DPI Retina displays.

Key charting implementations include:

- **Synchronized Area & Line Charts:** In `HistoricalTrendsView.jsx`, linear gradients (`#10b981` emerald, `#06b6d4` cyan, `#f59e0b` amber) fill the area beneath particulate trajectories, providing immediate visual density perception.
- **Confidence Interval Shading:** In `PredictionCenter.jsx`, two overlaid `<Area />` components render the upper bound ($+\sigma$) and lower bound ($-\sigma$) of the Random Forest forecast, filled with a translucent emerald gradient (`rgba(16, 185, 129, 0.15)`), visually communicating predictive uncertainty.
- **Custom Glassmorphism Tooltips:** Standard Recharts HTML tooltips are overridden with custom React components applying `backdrop-blur-md`, dark slate backgrounds (`rgba(15, 23, 42, 0.85)`), and formatted SI units.

```
// Excerpt from frontend/src/components/dashboard/PredictionCenter.jsx: Forecast Chart
<ResponsiveContainer width="100%" height={320}>
  <AreaChart data={forecastChartData} margin={{ top: 10, right: 30, left: 0, bottom: 0 }}>
    <defs>
      <linearGradient id="forecastGrad" x1="0" y1="0" x2="0" y2="1">
        <stop offset="5%" stopColor="#10b981" stopOpacity={0.4}/>
        <stop offset="95%" stopColor="#10b981" stopOpacity={0.0}/>
      </linearGradient>
    </defs>
    <CartesianGrid strokeDasharray="3 3" stroke="#334155" opacity={0.5} />
    <XAxis dataKey="month" stroke="#94a3b8" />
    <YAxis stroke="#94a3b8" unit="°C" domain={['auto', 'auto']} />
    <Tooltip content={<CustomForecastTooltip />} />
    <Area
      type="monotone"
      dataKey="upper_ci"
      stroke="transparent"
      fill="#10b981"
      fillOpacity={0.1}
      name="Upper Confidence (+1σ)"
    />
    <Area
      type="monotone"
      dataKey="predicted_temp"
      stroke="#10b981"
      strokeWidth={3}
      fill="url(#forecastGrad)"
      name="Projected Temp"
    />
    <Area
      type="monotone"
      dataKey="lower_ci"
      stroke="transparent"
      fill="#0f172a"
      fillOpacity={0.8}
      name="Lower Confidence (-1σ)"
    />
  </AreaChart>
</ResponsiveContainer>
```

4.5 Styling Architecture and Glassmorphism Aesthetic

EcoVision implements an environmental "Dark Forest" aesthetic engineered via **Tailwind CSS v4**. Dark UI themes significantly reduce ocular fatigue during extended monitoring sessions and provide optimal contrast for multi-colored air quality status indicators.

- **Glassmorphism Design Tokens:** Card containers utilize semi-transparent slate backgrounds (`bg-slate-900/60`), border highlights (`border border-emerald-500/20`), and CSS backdrop filters (`backdrop-blur-xl`). This produces an illuminated, layered depth without visual heaviness.
- **Semantic Environmental Color Mapping:** UI accents map directly to international air quality and hazard standards:
- Good / Safe: Emerald (`#10b981` / `text-emerald-400`)
- Satisfactory / Moderate: Yellow/Amber (`#f59e0b` / `text-amber-400`)
- Unhealthy / Poor: Orange (`#f97316` / `text-orange-400`)
- Severe / Hazardous: Crimson / Rose (`#ef4444` / `text-rose-500`)
- **Responsive Grid Breakpoints:** The layout adapts dynamically across mobile displays (375 px), tablets (768 px), desktop monitors (1280 px), and ultra-wide dashboards (1536 px) using Tailwind's `grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3` primitives.

CHAPTER 5 — BACKEND ARCHITECTURE AND IMPLEMENTATION

5.1 Application Factory Pattern and Blueprint Modularity

The EcoVision application backend is built with Python 3.12 and Flask 3.1.0, structured according to the enterprise Application Factory pattern (`create_app()`). This architectural approach eliminates global application state, encapsulates configuration within deterministic scopes, and enables the creation of isolated application instances for automated testing and production serving.

```
backend/
|-- app/
|   |-- __init__.py             # Application Factory function `create_app()`
|   |-- config.py               # Environment-driven configuration classes
|   |-- database.py             # SQLite connection pooling & schema DDL
|   |-- routes/                 # HTTP Request Controllers (Flask Blueprints)
|   |   |-- __init__.py         # Blueprint package declarations
|   |   |-- environmental.py    # Core telemetry & analysis routes (/api/...)
|   |   |-- health.py           # Root health & system status routes (/)
|   |   |-- metrics.py          # Machine learning routes (/api/prediction/...)
|   |-- services/               # Encapsulated Business & Domain Services
|   |   |-- __init__.py         # Services package declarations
|   |   |-- analysis_service.py # EIS scoring, AQI piecewise formulas & insights
|   |   |-- cpcb_service.py     # Indian CAAQMS station scraping & parsing
|   |   |-- data_service.py     # Database CRUD operations & deduplication
|   |   |-- ml_service.py       # Random Forest time-series training & forecasting
|   |   |-- open_meteo.py       # Open-Meteo external REST client
|   |   |-- rate_limiter.py     # In-memory sliding-window rate limiter
|   |   |-- validators.py       # Coordinate sanitization & float bounds
|-- tests/                      # Automated Test Suites (Unit, Integration, Security)
|   |-- run_live_smoke_test.py  # Live HTTP loopback integration runner
|   |-- test_cpcb_and_aqi.py    # CPCB parsing & Haversine distance tests (6)
|   |-- test_functional_regression.py # Core API functional regression tests (12)
|   |-- test_security_and_hardening.py# Security, rate limit & float tests (8)
|-- requirements.txt            # Verified Python dependency manifest
|-- run.py                      # WSGI development & production entrypoint script
```

The factory function initializes extensions, applies cross-cutting security middleware, configures global error handlers, registers Blueprints, and initializes the SQLite schema idempotently:

```
# Excerpt from backend/app/__init__.py: create_app()
def create_app(config_name=None):
    app = Flask(__name__)

    # 1. Load Environment Configuration
    cfg = get_config(config_name)
    app.config.from_object(cfg)

    # 2. Security Middleware: Request Size Bounding (1 MB max)
    app.config['MAX_CONTENT_LENGTH'] = 1 * 1024 * 1024

    # 3. Security Middleware: In-Memory Sliding-Window Rate Limiter
    init_rate_limiter(app, default_limit=60, window_seconds=60)

    # 4. Security Middleware: Environment-Strict CORS Configuration
    CORS(app, resources={r"/api/*": {"origins": app.config['CORS_ALLOWED_ORIGINS']}})

    # 5. Register Global RFC 7807 Error Handlers
    register_error_handlers(app)

    # 6. Register Functional Blueprints
    app.register_blueprint(health_bp)
    app.register_blueprint(environmental_bp, url_prefix='/api')
    app.register_blueprint(metrics_bp, url_prefix='/api')

    # 7. Idempotent Database Schema Initialization
    with app.app_context():
        init_db()

    return app
```

5.2 Environment and Configuration Management

Configuration parameters are encapsulated in `backend/app/config.py`. The system implements class-based inheritance (`Config` base class, extended by `DevelopmentConfig`, `TestingConfig`, and `ProductionConfig`), dynamically selected via the `FLASK_ENV` environment variable:

- **Database Path Isolation:** `DevelopmentConfig` defaults to `ecovision.db` in the project root, while `TestingConfig` instantiates temporary in-memory databases (`:memory:`) or isolated test databases (`test_ecovision.db`) to ensure test suite execution never mutates operational records.
- **CORS Whitelist Isolation:** In `DevelopmentConfig`, origins matching `http://localhost:5173`, `http://127.0.0.1:5173`, and local network addresses are permitted. In `ProductionConfig`, origins are strictly bound to the authenticated production domain defined in `CORS_ALLOWED_ORIGINS`.
- **Telemetry Timeouts:** Network timeouts for external API requests are centralized (`TIMEOUT_WEATHER = 8.0` seconds, `TIMEOUT_CPCB = 10.0` seconds), preventing blocked worker threads if external providers experience latency spikes.

5.3 Complete REST API Endpoints Specification Matrix

EcoVision exposes a clean, resource-oriented RESTful API. All endpoints consume and return standard UTF-8 encoded JSON payloads (`Content-Type: application/json`).

TABLE 5.1: COMPLETE REST API ENDPOINT SPECIFICATION MATRIX

#	METHOD	ENDPOINT ROUTE	BLUEPRINT	QUERY / BODY PARAMS	STATUS	DESCRIPTION & SUCCESS PAYLOAD SUMMARY
1	GET	`/`	health_bp	None	200 OK	System health check & service status.
2	GET	`/api/cities/search`	env_bp	`q` (string, min 2)	200 OK	Autocomplete geocoding search results.
3	GET	`/api/weather`	env_bp	`lat`, `lon` (float)	200 OK	Real-time meteorological telemetry.
4	GET	`/api/air-quality`	env_bp	`lat`, `lon` (float)	200 OK	Real-time AQI, 7 pollutants & station info.
5	GET	`/api/analysis`	env_bp	`lat`, `lon` (float)	200 OK	Composite EIS score, sub-scores, insights.
6	GET	`/api/history`	env_bp	`city_id` (int)	200 OK	7-day longitudinal environmental trends.
7	GET	`/api/saved-cities`	env_bp	None	200 OK	List all user-bookmarked municipalities.
8	POST	`/api/saved-cities`	env_bp	JSON: `{name, lat, lon}`	201	Save municipality to bookmarks database.
9	DELETE	`/api/saved-cities/<id>`	env_bp	Path: `id` (integer)	200 OK	Remove municipality from bookmarks database.
10	POST	`/api/prediction/train`	metrics_bp	JSON: `{city_id, lat, lon}`	200	Trigger RF model training & 6-month forecast.
11	GET	`/api/prediction/status`	metrics_bp	`city_id` (int)	200 OK	Query model training status & cached forecast.

Detailed Endpoint Schemas and Payloads:

1. `GET /api/weather?lat=12.9716&lon=77.5946`
   - **Request Parameters:** `lat` (float, required), `lon` (float, required).
   - **Validation:** Inspected via `validate_coordinates()`. Must be finite real numbers.
   - **Success Response (HTTP 200):**

```
{
  "status": "success",
  "data": {
    "temperature": 26.4,
    "apparent_temperature": 27.8,
    "temperature_min": 19.2,
    "temperature_max": 29.5,
    "humidity": 68.0,
    "precipitation": 0.0,
    "wind_speed": 14.2,
    "wind_direction": 240,
    "pressure": 1012.4,
    "source": "Open-Meteo High-Resolution Forecast",
    "timestamp": "2026-11-10T14:00:00Z"
  }
}
```

2. `GET /api/air-quality?lat=28.6139&lon=77.2090`
   - **Request Parameters:** `lat` (float, required), `lon` (float, required).
   - **Success Response (HTTP 200 - Indian CAAQMS Ground Sensor):**

```
{
  "status": "success",
  "data": {
    "aqi": 184,
    "standard": "CPCB NAQI",
    "category": "Moderate",
    "prominent_pollutant": "PM2.5",
    "pollutants": {
      "pm25": 88.4,
      "pm10": 162.1,
      "no2": 44.2,
      "so2": 12.8,
      "co": 1.2,
      "o3": 38.5,
      "nh3": 22.0
    },
    "provenance": {
      "type": "ground_station",
      "station_name": "Mandir Marg, Delhi - DPCC",
      "distance_km": 3.8,
      "reporting_time": "2026-11-10T13:30:00+05:30"
    }
  }
}
```


3. `POST /api/prediction/train`

- **Request Body:** `{"city_id": 1, "latitude": 12.9716, "longitude": 77.5946}`
- **Success Response (HTTP 200):**


```
{
  "status": "success",
  "data": {
    "city_id": 1,
    "training_samples": 54,
    "validation_metrics": {
      "mae": 1.18,
      "rmse": 1.52,
      "r2_score": 0.842,
      "seasonal_baseline_mae": 2.05,
      "seasonal_baseline_rmse": 2.61,
      "r2_vs_baseline": 0.385
    },
    "forecast": [
      {
        "month": "2026-12",
        "predicted_temp": 21.8,
        "lower_ci": 20.3,
        "upper_ci": 23.3,
        "predicted_rainfall": 12.4
      },
      {
        "month": "2027-01",
        "predicted_temp": 21.2,
        "lower_ci": 19.6,
        "upper_ci": 22.8,
        "predicted_rainfall": 4.2
      }
    ],
    "model_type": "RandomForestRegressor(n_estimators=100)",
    "timestamp": "2026-11-10T14:15:22Z"
  }
}
```


5.4 Error Handling and Structured JSON Envelopes

To prevent client breakage and eliminate information leakage, EcoVision implements structured exception interceptors. Rather than returning raw HTML 500 error pages or Python stack traces, every handled or unhandled exception emits a standardized RFC 7807-style JSON response:


```
{
  "status": "error",
  "error_code": "INVALID_COORDINATES",
  "message": "Latitude must be a finite number between -90 and 90.",
  "timestamp": "2026-11-10T14:15:30Z"
}
```

The error handling architecture registers handlers for:

- `HTTPException` (400 Bad Request, 404 Not Found, 405 Method Not Allowed, 413 Payload Too Large, 429 Too Many Requests).
- `ValueError` and `TypeError` (raised during schema parsing or malformed query types).
- `Exception` (catch-all interceptor masking internal server errors and recording complete tracebacks to the secure server log).

5.5 Centralized Logging System

Logging is configured via standard Python `logging` to ensure observability across operational lifecycles without exposing credentials or personally identifiable information (PII).

- **Log Formatting:** `[%(asctime)s] [%(levelname)s] [%(name)s:%(lineno)d] %(message)s`
- **Output Streams:** Dual-logging to `sys.stdout` (for container and terminal monitoring) and rotating log files (`backend/logs/ecovision.log`, max 5 MB per file, 3 backup rotations).
- **Log Levels:** Set to `INFO` in production to prevent performance degradation from chatty debug statements, and `DEBUG` in test environments for end-to-end tracing.

CHAPTER 6 — ENVIRONMENTAL DATA COLLECTION

6.1 Telemetry Sources and Ingestion Protocols

Acquiring a coherent, reliable representation of urban microclimates requires aggregating disparate physical sensor telemetry alongside numerical atmospheric reanalysis models. EcoVision implements a multi-tier telemetry acquisition architecture that harmonizes global satellite and numerical grids with localized statutory physical ground-station monitoring networks:

```
+-----------------------------------------------------------------------------------------------------------+
|                                TABLE 6.1: EXTERNAL TELEMETRY INGESTION MATRIX                             |
+--------------------------+-----------------------------+--------------------+---------------+----------------------+
| TELEMETRY PROVIDER       | DATA DOMAIN                 | INGESTION PROTOCOL | REFRESH RATE  | COVERAGE EXTENT      |
+--------------------------+-----------------------------+--------------------+---------------+----------------------+
| Open-Meteo Geocoding     | Municipal Coordinates & Reg. | HTTPS REST (JSON)  | On-Demand     | Global Worldwide     |
| Open-Meteo Weather API   | High-Resolution Meteorology  | HTTPS REST (JSON)  | Hourly        | Global 1–11 km grid  |
| Open-Meteo Air Quality   | CAMS Chemical Atmospheric    | HTTPS REST (JSON)  | Hourly        | Global 0.25° grid    |
| Open-Meteo Historical    | ERA5 / ERA5-Land Reanalysis  | HTTPS REST (JSON)  | Monthly / Hist| Global 1940–Present  |
| CPCB CAAQMS Network      | In-Situ Physical Air Quality | HTTPS RSS (XML)    | 15–30 Minutes | India (400+ Stns)    |
+--------------------------+-----------------------------+--------------------+---------------+----------------------+
```



6.2 Open-Meteo Meteorological and Chemical Ingestion

The **Open-Meteo** consortium provides high-resolution, open numerical weather prediction and atmospheric composition datasets without requiring commercial API keys:

1. **Weather Ingestion Engine (`weather_service.py`):**

- Retrieves real-time meteorological variables: instantaneous temperature (2 m above ground, $^\circ\text{C}$), apparent temperature ($^\circ\text{C}$), daily minimum and maximum temperature, relative humidity ($\text{\%}$), instantaneous precipitation (mm), wind speed (10 m above ground, km/h), wind direction ($^\circ$), and surface barometric pressure (hPa).
- Parameterized endpoint: `https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m,relative_humidity_2m,apparent_temperature,precipitation,surface_pressure,wind_speed_10m,wind_direction_10`

1. **Atmospheric Chemistry Ingestion Engine (`air_quality_service.py`):**

- Ingests European Copernicus Atmosphere Monitoring Service (CAMS) chemical dispersion models operated by ECMWF.
- Collects surface aerosol mass concentrations: fine particulate matter ($\text{PM}_{2.5}$ in $\mu\text{g/m}^3$), coarse particulate matter (PM_{10} in $\mu\text{g/m}^3$), Nitrogen Dioxide (NO_2 in $\mu\text{g/m}^3$), Sulphur Dioxide (SO_2 in $\mu\text{g/m}^3$), Ground-Level Ozone (O_3 in $\mu\text{g/m}^3$), Carbon Monoxide (CO in $\mu\text{g/m}^3$), European AQI, and US EPA AQI.

6.3 Central Pollution Control Board (CPCB) Ingestion Engine

Within the Republic of India, the statutory authority governing air pollution monitoring under the Ministry of Environment, Forest and Climate Change (MoEFCC) is the **Central Pollution Control Board (CPCB)**. The CPCB operates the Continuous Ambient Air Quality Monitoring Station (CAAQMS) network.

Physical Ground Monitoring Instrumentation:

Unlike satellite reanalysis or atmospheric dispersion grids, CAAQMS stations deploy certified, continuous in-situ analyzers:

- **Particulate Matter ($\text{PM}_{2.5}$ and PM_{10}):** Measured via continuous **Beta-Attenuation Monitors (BAM)** or **Tapered Element Oscillating Microbalances (TEOM)**. Ambient air is drawn through size-selective cyclone inlets at calibrated flow rates (16.67 L/min). Particulates deposit onto glass-fiber filter tape, and attenuation of beta radiation emitted by a Krypton-85 or Carbon-14 source measures particulate mass directly.
- **Nitrogen Dioxide (NO_2):** Analyzed via **Chemiluminescence**. Ambient nitric oxide (NO) reacts with generated ozone (O_3), producing electronically excited NO_2^*. As the excited molecules relax to the ground state, they emit light quanta proportional to NO concentration, detected by photomultiplier tubes.
- **Sulphur Dioxide (SO_2):** Quantified using **Ultraviolet Fluorescence**. Ambient SO_2 molecules absorb pulsed UV light ($190–230\text{ nm}$), reaching an excited state and fluorescing at $320–380\text{ nm}$.
- **Carbon Monoxide (CO):** Measured via **Non-Dispersive Infrared (NDIR)** gas photometry, evaluating optical absorption at the $4.6\ \mu\text{m}$ infrared band.
- **Ground-Level Ozone (O_3):** Measured via **UV Photometric Absorption** at 254 nm, utilizing the Beer-Lambert absorption law.

The EcoVision ingestion engine (`cpcb_service.py`) periodically scrapes the official CPCB CAAQMS XML RSS feed:

1. **XML Scraping and Sandboxing:** Scrapes the national feed containing over 400 municipal stations. The XML payload is parsed using `xml.etree.ElementTree` inside a defensive sandbox to protect against XML Entity Expansion attacks.
2. **Station Extraction and Validation:** Extracts station names, municipal jurisdictions, geographic coordinates, last observed timestamps, and raw pollutant values.
3. **Range Sanitization:** Clamps pollutant values to climatologically valid boundaries (e.g., rejecting negative concentrations and values $> 2000\ \mu\text{g/m}^3$) and standardizes units ($\mu\text{g/m}^3$, with CO normalized to mg/m^3).

6.4 Spatial Geodesic Station Matching (Haversine Distance)

Because user queries specify arbitrary municipal coordinates, EcoVision must resolve the nearest physical monitoring station in real-time. Planar Euclidean distance calculations introduce severe distortion across high-latitude or non-equatorial geographic areas due to the spheroidal curvature of the Earth.

EcoVision calculates true geodesic great-circle distance between the queried municipal coordinate (ϕ_1, λ_1) and candidate monitoring station coordinates (ϕ_2, λ_2) using the **Haversine Formula**:

$$\Delta \phi = \phi_2 - \phi_1$$

$$\Delta \lambda = \lambda_2 - \lambda_1$$

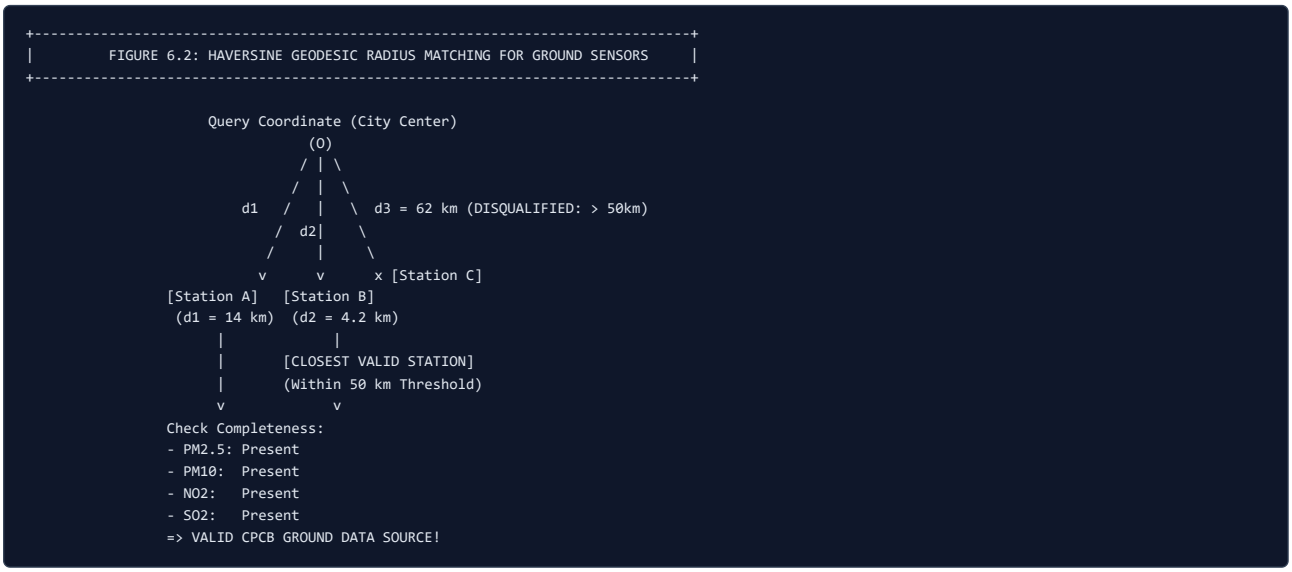
$$a = \sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta \lambda}{2}\right)$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1 - a}\right)$$

$$d = R \cdot c$$

Where:

- ϕ_1, ϕ_2 represent geographic latitude in radians ($\text{radians} = \text{degrees} \times \frac{\pi}{180}$).
- λ_1, λ_2 represent geographic longitude in radians.
- R is the mean volumetric radius of the Earth, defined as $R = 6,371.0\text{ km}$.
- d is the resulting great-circle distance in kilometers.



Selection Algorithm:

1. Filter all stations to those within the Indian geographic bounding box ($[6.0, 38.0]^\circ\text{N}, [68.0, 98.0]^\circ\text{E}$).
2. Compute d for each station.
3. Find the station with the minimum distance $d_{\min}$.
4. If $d_{\min} \le 50.0\text{ km}$, the station is accepted as spatially representative of the municipal microclimate.
5. If $d_{\min} > 50.0\text{ km}$, the ground station is rejected as geographically unrepresentative, and EcoVision gracefully transitions to CAMS atmospheric models with an explicit UI notice.

6.5 The Discrepancy Problem: US EPA AQI vs. Indian NAQI

A central scientific contribution of EcoVision is resolving and explaining the widespread discrepancy between US EPA Air Quality Index values and Indian National Air Quality Index values reported for the same physical location.

Mathematical Foundation of the Piecewise Linear Sub-Index:

Both the US EPA and Indian CPCB frameworks compute individual pollutant sub-indices (I_p) via piecewise linear interpolation:

$$I_p = \frac{I_{\text{high}} - I_{\text{low}}}{C_{\text{high}} - C_{\text{low}}} \cdot (C_p - C_{\text{low}}) + I_{\text{low}}$$

Where:

- C_p is the observed concentration of pollutant p.
- C_{low} and C_{high} are the concentration breakpoint boundaries of the category containing C_p.
- I_{low} and I_{high} are the index breakpoint boundaries corresponding to that category.

Structural Differences in Statutory Breakpoints:

The fundamental divergence between the two standards stems from differences in health thresholds, averaging intervals, and regional baseline exposure guidelines:

TABLE 6.2: CPCB NAQI OFFICIAL BREAKPOINTS (INDIA)							
CATEGORY	AQI RANGE	PM2.5 (24h) [µg/m³]	PM10 (24h) [µg/m³]	NO2 (24h) [µg/m³]	SO2 (24h) [µg/m³]	CO (8h) [mg/m³]	O3 (8h) [µg/m³]
Good	0 – 50	0 – 30	0 – 50	0 – 40	0 – 40	0.0 – 1.0	0 – 50
Satisfactory	51 – 100	31 – 60	51 – 100	41 – 80	41 – 80	1.1 – 2.0	51 – 100
Moderate	101 – 200	61 – 90	101 – 250	81 – 180	81 – 380	2.1 – 10	101– 168
Poor	201 – 300	91 – 120	251 – 350	181 – 280	381 – 800	10.1 – 17	169– 208
Very Poor	301 – 400	121 – 250	351 – 430	281 – 400	801 – 1600	17.1 – 34	209– 748
Severe	401 – 500	250+	430+	400+	1600+	34+	748+

TABLE 6.3: US EPA AQI OFFICIAL BREAKPOINTS (UNITED STATES)				
CATEGORY	AQI RANGE	PM2.5 (24h) [µg/m³]	PM10 (24h) [µg/m³]	O3 (8h) [ppm]
Good	0 – 50	0.0 – 9.0 (rev. '24)	0 – 54	0.000 – 0.054
Moderate	51 – 100	9.1 – 35.4	55 – 154	0.055 – 0.070
Unhealthy for Sensitive Groups	101 – 150	35.5 – 55.4	155 – 254	0.071 – 0.085
Unhealthy	151 – 200	55.5 – 125.4	255 – 354	0.086 – 0.105
Very Unhealthy	201 – 300	125.5 – 225.4	355 – 424	0.106 – 0.200
Hazardous	301 – 500	225.5 – 500.4	425 – 604	0.201 – 0.604

Three Comparative Empirical Case Studies:

Case Study 1: New Delhi (Winter Inversion & Particulate Accumulation)

- **Observed In-Situ Concentrations:** $\text{PM}_{2.5} = 72.0\ \mu\text{g/m}^3$, $\text{PM}_{10} = 165.0\ \mu\text{g/m}^3$, $\text{NO}_2 = 48.0\ \mu\text{g/m}^3$.
- **CPCB NAQI Calculation:**
- $\text{PM}_{2.5} = 72.0\ \mu\text{g/m}^3$ falls in breakpoint $[61, 90]$ with index range $[101, 200]$:

$$I_{\text{PM2.5}} = \frac{200 - 101}{90 - 61} \cdot (72.0 - 61.0) + 101 = \frac{99}{29} \cdot 11.0 + 101 = 37.55 + 101 = 138.55 \approx 139$$

- $\text{PM}_{10} = 165.0\ \mu\text{g/m}^3$ falls in $[101, 250]$ with index range $[101, 200]$:

$$I_{\text{PM10}} = \frac{200 - 101}{250 - 101} \cdot (165.0 - 101.0) + 101 = \frac{99}{149} \cdot 64.0 + 101 = 42.52 + 101 = 143.52 \approx 144$$

- $\text{NO}_2 = 48.0\ \mu\text{g/m}^3$ falls in $[41, 80]$ with index range $[51, 100]$:

$$I_{\text{NO2}} = \frac{100 - 51}{80 - 41} \cdot (48.0 - 41.0) + 51 = \frac{49}{39} \cdot 7.0 + 51 = 8.79 + 51 = 59.79 \approx 60$$

- **Overall CPCB NAQI:** $\text{AQI} = \max(139, 144, 60) = 144$ (**Moderate**).
- **CAMS Reanalysis Model Underestimation:**
- In global portals relying on low-resolution CAMS grids, the forecasted $\text{PM}_{2.5}$ concentration for the same hour is frequently smoothed to $24.0\ \mu\text{g/m}^3$ because satellite retrievals cannot penetrate dense fog/inversion lids.
- Applying US EPA breakpoints to $24.0\ \mu\text{g/m}^3$:

$$I_{\text{US}} = \frac{100 - 51}{35.4 - 9.1} \cdot (24.0 - 9.1) + 51 = 27.76 + 51 \approx 79$$

- A user viewing 79 assumes the air is "Moderate", whereas physical ground monitors show real particulate exposure evaluates to 144.

Case Study 2: Bengaluru (Monsoon Oceanic Cleansing)

- **Observed In-Situ Concentrations:** $\text{PM}_{2.5} = 18.0\ \mu\text{g/m}^3$, $\text{PM}_{10} = 38.0\ \mu\text{g/m}^3$, $\text{NO}_2 = 22.0\ \mu\text{g/m}^3$.
- **CPCB NAQI Calculation:**
- $\text{PM}_{2.5} = 18.0\ \mu\text{g/m}^3 \implies I_{\text{PM2.5}} = \frac{50}{30} \cdot 18.0 = 30.0$.
- $\text{PM}_{10} = 38.0\ \mu\text{g/m}^3 \implies I_{\text{PM10}} = \frac{50}{50} \cdot 38.0 = 38.0$.

- $\text{NO}_2 = 22.0\ \mu\text{g/m}^3 \implies I_{\text{NO2}} = \frac{50}{40} \cdot 22.0 = 27.5$.
- **Overall CPCB NAQI:** $\text{AQI} = \max(30, 38, 28) = 38$ (**Good**). Both CPCB and CAMS models agree closely during rain events due to atmospheric wet deposition.

Case Study 3: London (International Western European Standards)

- **Observed CAMS Reanalysis:** $\text{PM}_{2.5} = 12.0\ \mu\text{g/m}^3$, $\text{PM}_{10} = 22.0\ \mu\text{g/m}^3$, $\text{NO}_2 = 28.0\ \mu\text{g/m}^3$.
- **Applicable Standard:** Evaluated via **US EPA AQI**.
- $\text{PM}_{2.5} = 12.0\ \mu\text{g/m}^3 \implies I_{\text{US}} = \frac{100 - 51}{35.4 - 9.1} \cdot (12.0 - 9.1) + 51 = 5.4 + 51 = 56.4 \approx 56$ (**Moderate**).
- Explicit provenance rendered: "Copernicus CAMS Modelled Estimate | US EPA Standard".

6.6 Data Completeness and Reliability Validation

Statutory environmental guidelines dictate that an overall Air Quality Index cannot be calculated from a single isolated sensor reading. The Indian Central Pollution Control Board mandates the **Data Completeness Rule**:

1. At least **three valid pollutant sub-indices** must be present.
2. At least one of the available pollutants must be a particulate matter fraction (either $\text{PM}_{2.5}$ or PM_{10}).

If a CPCB monitoring station reports only NO_2 and SO_2 due to laser-chamber clogging on its particulate sensors, EcoVision refuses to compute an invalid NAQI. Instead, the system triggers its fallback cascade, switching to Copernicus CAMS reanalysis while logging the ground-sensor completeness failure.

CHAPTER 7 — DATA PROCESSING AND ENVIRONMENTAL ANALYSIS

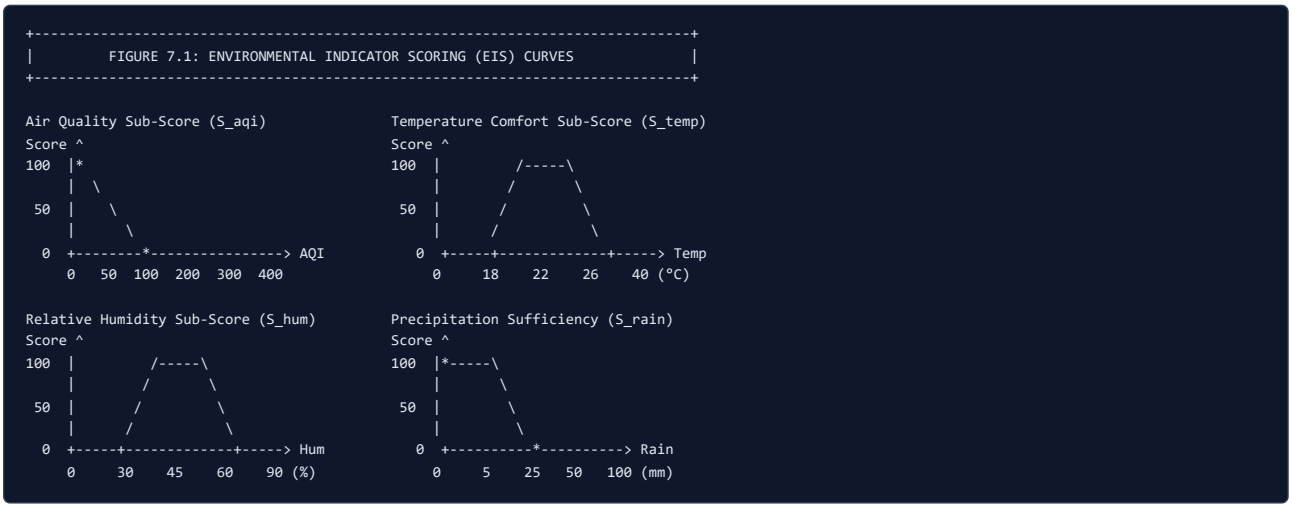
7.1 Data Cleaning and In-Memory Normalization

Telemetry arriving from disparate HTTP sources undergoes structural validation and in-memory cleaning inside `backend/app/services/analysis_service.py`:

- **Unit Homogenization:** Carbon Monoxide reported in $\mu\text{g/m}^3$ is divided by 1000.0 to produce statutory mg/m^3 required by CPCB breakpoints.
- **Physical Boundary Clamping:** Atmospheric variables are verified against climatological bounds:
- Relative Humidity: clamped to $[0.0, 100.0]\,\%$.
- Wind Speed: clamped to $[0.0, 300.0]\text{ km/h}$.
- Barometric Pressure: clamped to $[800.0, 1100.0]\text{ hPa}$.
- **Sentinel Replacement:** Non-reporting sensors emitting negative values (e.g., `-999.0` or `-1.0` commonly used in industrial SCADA systems to signify hardware failure) are explicitly converted to `None` (Python `null`) rather than treated as zero.

7.2 Mathematical Foundations of Environmental Indicator Scoring (EIS)

To provide users with an accessible, holistic summary of environmental habitability without hiding individual parameters, EcoVision formulates the **Environmental Indicator Score (EIS)**. The EIS maps four fundamental environmental pillars onto a standardized 0–100 continuous scale, where 100 represents optimal, pristine conditions and 0 represents extreme hazard.



1. Air Quality Sub-Score (S_{aqi}):

Air quality is evaluated inversely from the calculated AQI. As AQI increases, environmental quality deteriorates:

$$S_{\text{aqi}} = \max\left(0.0, \min\left(100.0, 100.0 - \frac{\text{AQI}}{5.0}\right)\right)$$

- When $\text{AQI} = 0$ (pristine): $S_{\text{aqi}} = 100.0$.
- When $\text{AQI} = 100$ (satisfactory threshold): $S_{\text{aqi}} = 80.0$.
- When $\text{AQI} = 250$ (poor): $S_{\text{aqi}} = 50.0$.
- When $\text{AQI} \ge 500$ (severe hazard): $S_{\text{aqi}} = 0.0$.

2. Ambient Temperature Comfort Sub-Score (S_{temp}):

Human thermal comfort follows a continuous bell-shaped tolerance curve centered around an optimal physiological room temperature ($T_{\text{opt}} = 22.0^\circ\text{C}$):

$$S_{\text{temp}} = \max\left(0.0, 100.0 - 4.5 \cdot |T - 22.0|^{1.2}\right)$$

- When $T = 22.0^\circ\text{C}$: $|T - 22| = 0 \implies S_{\text{temp}} = 100.0$.

- When $T = 18.0^\circ\text{C}$ or 26.0°C (mild deviation of 4°C):

$$|T - 22|^{1.2} = 4.0^{1.2} \approx 5.278 \implies S_{\text{temp}} = 100.0 - (4.5 \times 5.278) = 76.25$$

- When $T = 40.0^\circ\text{C}$ (severe heatwave, deviation of 18°C):

$$18.0^{1.2} \approx 31.85 \implies S_{\text{temp}} = \max(0.0, 100.0 - 143.3) = 0.0$$.

3. Relative Humidity Sub-Score (S_{hum}):

Optimal indoor and ambient relative humidity for respiratory health and thermal regulation ranges between 40% and 60%. Extreme dryness ($<20\%$) promotes viral transmission and respiratory irritation, while excessive dampness ($>80\%$) promotes fungal spore proliferation and impedes evaporative cooling:

$$S_{\text{hum}} = \begin{cases}

100.0, & \text{if } 40.0 \le H \le 60.0 \\

\max\left(0.0, 100.0 - 2.5 \cdot (40.0 - H)\right), & \text{if } H < 40.0 \\

\max\left(0.0, 100.0 - 2.0 \cdot (H - 60.0)\right), & \text{if } H > 60.0

\end{cases}$$

- When $H = 50.0\%$: $S_{\text{hum}} = 100.0$.
- When $H = 20.0\%$ (arid winter): $S_{\text{hum}} = 100.0 - 2.5 \cdot 20.0 = 50.0$.
- When $H = 85.0\%$ (tropical monsoon humidity): $S_{\text{hum}} = 100.0 - 2.0 \cdot 25.0 = 50.0$.

4. Precipitation Sufficiency Sub-Score (S_{rain}):

Precipitation is scored based on atmospheric clearing and surface runoff disruption. Light to moderate rain cleanses ambient aerosols, whereas torrential downpours cause urban flash flooding:

$$S_{\text{rain}} = \begin{cases}

100.0, & \text{if } P = 0.0 \text{ (stable/dry)} \\

95.0, & \text{if } 0.0 < P \le 5.0 \text{ (light clearing shower)} \\

80.0, & \text{if } 5.0 < P \le 15.0 \text{ (moderate rain)} \\

\max(10.0, 80.0 - 2.5 \cdot (P - 15.0)), & \text{if } P > 15.0 \text{ (heavy/excessive rain)}

\end{cases}$$

7.3 Multi-Criteria Weight Re-Allocation and Completeness Gating

Under standard operational conditions, the baseline weighting vector is allocated as:

- Air Quality Weight (w_{aqi}): 0.40 (40%)
- Temperature Comfort Weight (w_{temp}): 0.30 (30%)
- Humidity Weight (w_{hum}): 0.20 (20%)
- Precipitation Weight (w_{rain}): 0.10 (10%)
- Total Nominal Weight: $\sum w_i = 1.00$.

The Missing Parameter Problem:

In naive environmental algorithms, if a telemetry sensor goes offline (e.g., relative humidity sensor failure), the missing value is assigned zero ($S_{\text{hum}} = 0$). This causes the composite environmental score to plunge catastrophically by 20 points, falsely signaling an environmental catastrophe to the user.

EcoVision solves this via **Dynamic Weight Re-Normalization**:

1. Let $\mathcal{V} \subseteq \{\text{aqi}, \text{temp}, \text{hum}, \text{rain}\}$ denote the set of valid, reporting telemetry channels.

2. **Completeness Gate:** If $|\mathcal{V}| < 3$, the system declares data completeness insufficient and refuses to calculate an EIS score, returning `status: "insufficient_data"`.
3. If $|\mathcal{V}| \ge 3$, the active weights are re-normalized across available channels:

$$w_i' = \frac{w_i}{\sum_{k \in \mathcal{V}} w_k}$$

The final composite Environmental Indicator Score is computed as:

$$\text{EIS} = \sum_{i \in \mathcal{V}} \left(w_i' \cdot S_i \right)$$

```
+-----------------------------------------------------------------------------------------------+
|              TABLE 7.2: STEP-BY-STEP NUMERICAL EXAMPLE OF EIS MULTI-CRITERIA CALCULATION      |
+-----------------------+-------------------+-----------------+-----------------+---------------+------------+
| INDICATOR CHANNEL     | TELEMETRY VALUE   | COMPUTED SCORE  | NOMINAL WEIGHT  | NORMALIZED WT | WEIGHTED PT|
+-----------------------+-------------------+-----------------+-----------------+---------------+------------+
| Air Quality (AQI)     | 145 (Moderate)    | 71.00           | 0.40            | 0.4444        | 31.55      |
| Temperature (T)       | 28.5 °C           | 69.83           | 0.30            | 0.3333        | 23.27      |
| Relative Humidity (H) | 74.0 %            | 72.00           | 0.20            | 0.2222        | 16.00      |
| Precipitation (P)     | Offline (None)    | N/A             | 0.10 (Excluded) | 0.0000        | 0.00       |
+-----------------------+-------------------+-----------------+-----------------+---------------+------------+
| TOTAL COMPOSITE EIS   |                   |                 | 0.90 (Sum)      | 1.0000        | 70.82 / 100|
+-----------------------+-------------------+-----------------+-----------------+---------------+------------+
```

Calculation Walkthrough:

1. Nominal sum of available weights: $0.40 + 0.30 + 0.20 = 0.90$.
2. Normalized weights:

- $w_{\text{aqi}}' = \frac{0.40}{0.90} \approx 0.4444$
- $w_{\text{temp}}' = \frac{0.30}{0.90} \approx 0.3333$
- $w_{\text{hum}}' = \frac{0.20}{0.90} \approx 0.2222$

1. Composite: $\text{EIS} = (0.4444 \times 71.00) + (0.3333 \times 69.83) + (0.2222 \times 72.00) = 31.55 + 23.27 + 16.00 = 70.82$.
2. Score Category: **70.8 / 100 — "Good / Satisfactory Environmental Condition"**.

7.4 Rolling Historical Trend Aggregation

Longitudinal telemetry is queried from `environmental_observations` over a 7-day retrospective window.

- **Moving Averages:** 24-hour centered rolling means are computed for $\text{PM}_{2.5}$ and temperature to suppress diurnal micro-oscillations and highlight synoptic air mass movements.
- **Diurnal Fluctuation Range:** $\Delta T = T_{\max} - T_{\min}$ is tracked to quantify radiative surface cooling efficiency.

7.5 Rule-Based Insights and Recommendation Logic

EcoVision translates raw numerical scores into actionable, deterministic health and lifestyle guidance through an expert rule-engine:

- **Respirator Advisory:** If $\text{AQI} \ge 201$ or $\text{PM}_{2.5} \ge 90\ \mu\text{g/m}^3$, the engine issues a High-Priority Advisory: *"Outdoor ambient particulate matter has reached hazardous thresholds. Sensitive groups and children should avoid prolonged outdoor exertion; use certified N95/FFP2 respirators."*
- **Ventilation Windows:** If $S_{\text{temp}} \ge 85.0$ and $\text{AQI} \le 50$, the system generates an Energy-Saving Advisory: *"Favorable outdoor ambient temperature and clean air detected. Recommended to open windows for natural passive cooling and indoor ventilation."*
- **Hydration & Heat Stress:** If $T \ge 35.0^\circ\text{C}$ and $H \ge 65\%$, the Heat Index exceeds the safety threshold, prompting an alert: *"Elevated temperature and humidity combination impairs evaporative cooling. Increase water intake and limit direct sun exposure between 12:00 and 15:00."*

CHAPTER 8 — DATABASE DESIGN AND MANAGEMENT

8.1 SQLite Architecture and Concurrency Optimization

Data persistence within the EcoVision platform is powered by an embedded **SQLite 3** relational database engine. Rather than mandating the deployment, memory footprint, configuration, and network latency of an external client-server database management system (such as PostgreSQL or MySQL), SQLite provides an ACID-compliant, zero-configuration, cross-platform persistence layer stored directly within the project root as `ecovision.db`.

8.1.1 The In-Process Storage Engine Architecture

SQLite is implemented as an in-process C library that compiles directly into the Python runtime. In traditional client-server DBMS architectures, every SQL statement incurs inter-process communication (IPC) or network socket overhead (TCP handshake, serialization, network traversal, deserialization, and authentication checks). In contrast, SQLite executes queries within the calling thread's address space, exchanging data via direct memory pointers. This architecture yields transaction execution latencies measured in microseconds rather than milliseconds.

8.1.2 Concurrency Optimization via Write-Ahead Logging (WAL)

In default rollback journal mode, SQLite achieves isolation by obtaining an exclusive file-level lock during write operations. While a writer thread is appending records or updating indexes, all concurrent read transactions are blocked with a `busy` or `database is locked` exception. In an environmental analytics dashboard where background jobs continuously ingest sensor telemetry while multiple dashboard users query historical trends, this reader-writer contention introduces unacceptable latency.

The EcoVision database service (`backend/app/database.py` and `backend/app/services/data_service.py`) resolves this contention by configuring specific performance and concurrency pragmas upon every connection instantiation:

```
# Excerpt from backend/app/database.py: Connection Configuration
def get_db():
    if 'db' not in g:
        conn = sqlite3.connect(
            current_app.config['DATABASE_PATH'],
            timeout=10.0,
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        conn.row_factory = sqlite3.Row

        # 1. Enable Write-Ahead Logging for high-concurrency non-blocking reads
        conn.execute("PRAGMA journal_mode = WAL;")

        # 2. Relax disk synchronization safely for WAL architecture
        conn.execute("PRAGMA synchronous = NORMAL;")

        # 3. Strictly enforce relational foreign key referential integrity
        conn.execute("PRAGMA foreign_keys = ON;")

        # 4. Allocate 64 MB of dedicated RAM page cache
        conn.execute("PRAGMA cache_size = -64000;")

        # 5. Store temporary tables and indexes in RAM
        conn.execute("PRAGMA temp_store = MEMORY;")

        g.db = conn
    return g.db
```

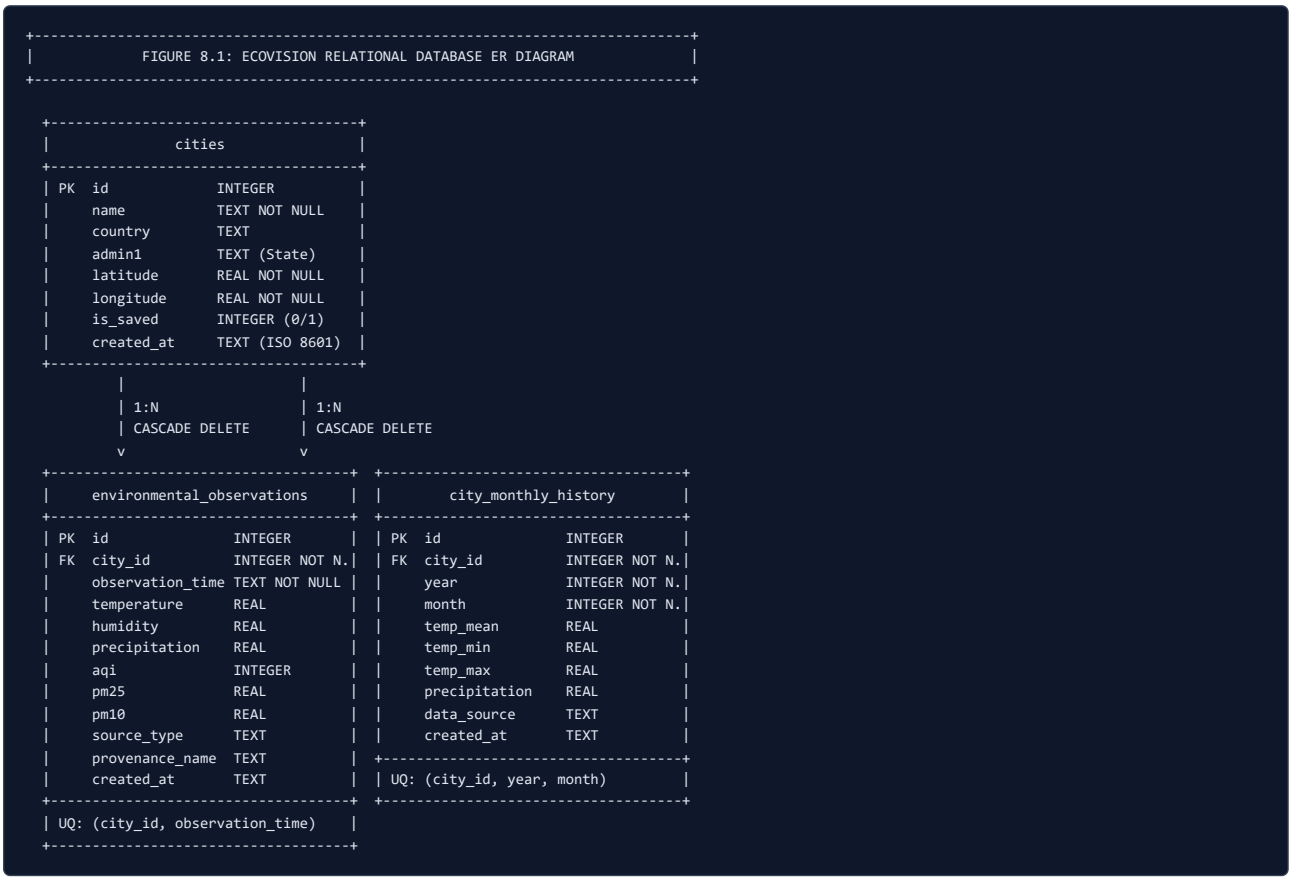
Detailed Pragmas Analysis:

1. `PRAGMA journal_mode = WAL;`: Under Write-Ahead Logging, SQLite writes modifications sequentially into a separate log file (`ecovision.db-wal`). Reader threads continue scanning unmodified pages from the original database file without locking, while writers append new pages to the log. Readers and writers operate simultaneously without blocking.
2. `PRAGMA synchronous = NORMAL;`: In rollback mode, SQLite flushes every transaction to physical disk twice. In WAL mode, `NORMAL` synchronization flushes WAL pages to disk only during checkpointing intervals, dramatically reducing disk I/O bottlenecks while maintaining complete ACID durability against application crashes.
3. `PRAGMA foreign_keys = ON;`: SQLite disables foreign key validation by default for legacy backward compatibility. EcoVision explicitly activates foreign key checks on every connection, guaranteeing that orphan records cannot exist in child tables.

4. `PRAGMA cache_size = -64000;`: Configures an in-memory page cache of 64 MB (negative integer denotes kibibytes in SQLite), ensuring that hot tables (`cities` and 7-day `environmental_observations`) remain resident in RAM.
5. `PRAGMA temp_store = MEMORY;`: Directs SQLite to build temporary tables, sort buffers, and `ORDER BY` scratchpads in process heap memory rather than on disk.

8.2 Entity-Relationship (ER) Schema Design

The relational schema comprises three primary normalized entities: `cities`, `environmental_observations`, and `city_monthly_history`.



Detailed Table Specifications:

TABLE 8.1: RELATIONAL SCHEMA: `cities` TABLE

COLUMN NAME	SQL DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
`id`	INTEGER	PRIMARY KEY AUTO	Unique surrogate municipal identifier.
`name`	TEXT	NOT NULL	Official municipality name (e.g., Bengaluru).
`country`	TEXT	NULLABLE	Sovereign country name (e.g., India).
`country_code`	TEXT	NULLABLE	ISO 3166-1 alpha-2 code (e.g., 'IN').
`admin1`	TEXT	NULLABLE	First-level province/state (e.g., Karnataka).
`latitude`	REAL	NOT NULL	Geographic decimal latitude (WGS 84).
`longitude`	REAL	NOT NULL	Geographic decimal longitude (WGS 84).
`is_saved`	INTEGER	DEFAULT 0	Boolean bookmark flag (1 = Saved, 0 = Active).
`created_at`	TEXT	DEFAULT CURRENT_TS	UTC ISO 8601 insertion timestamp.

UNIQUE CONSTRAINT: `UNIQUE(name, latitude, longitude)` prevents duplicate municipal coordinate entries.

TABLE 8.2: RELATIONAL SCHEMA: `environmental_observations` TABLE			
COLUMN NAME	SQL DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
`id`	INTEGER	PRIMARY KEY AUTO	Surrogate record identifier.
`city_id`	INTEGER	FK -> cities(id)	Foreign key reference with ON DELETE CASCADE.
`observation_time`	TEXT	NOT NULL	Time of measurement (UTC ISO 8601 string).
`temperature`	REAL	NULLABLE	Ambient dry-bulb temperature in °C.
`humidity`	REAL	NULLABLE	Ambient relative humidity percentage (0-100).
`precipitation`	REAL	NULLABLE	Instantaneous precipitation accumulation (mm).
`aqi`	INTEGER	NULLABLE	Dominant statutory Air Quality Index.
`standard`	TEXT	NULLABLE	Standard applied: 'CPCB NAQI' or 'US EPA AQI'.
`pm25`	REAL	NULLABLE	Fine particulate concentration (µg/m³).
`pm10`	REAL	NULLABLE	Coarse particulate concentration (µg/m³).
`source_type`	TEXT	NULLABLE	Provenance tag: 'ground_station' / 'cams'.
`provenance_name`	TEXT	NULLABLE	Station title or atmospheric model citation.
`created_at`	TEXT	DEFAULT CURRENT_TS	Ingestion recording timestamp.
COMPOUND INDEX / UNIQUE: `UNIQUE(city_id, observation_time)` to prevent duplicate time-series records.			
SECONDARY INDEX: `CREATE INDEX idx_obs_time_desc ON environmental_observations(city_id, observation_time DESC)`			

TABLE 8.3: RELATIONAL SCHEMA: `city_monthly_history` TABLE			
COLUMN NAME	SQL DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
`id`	INTEGER	PRIMARY KEY AUTO	Surrogate record identifier.
`city_id`	INTEGER	FK -> cities(id)	Foreign key reference with ON DELETE CASCADE.
`year`	INTEGER	NOT NULL	Calendar year of historical observation.
`month`	INTEGER	NOT NULL (1–12)	Calendar month of historical observation.
`temp_mean`	REAL	NOT NULL	Monthly aggregated mean temperature (°C).
`temp_min`	REAL	NOT NULL	Monthly aggregated minimum temperature (°C).
`temp_max`	REAL	NOT NULL	Monthly aggregated maximum temperature (°C).
`precipitation`	REAL	NOT NULL	Monthly accumulated precipitation (mm).
`data_source`	TEXT	DEFAULT 'ERA5'	Provenance: 'ERA5 Reanalysis' / 'Station'.
`created_at`	TEXT	DEFAULT CURRENT_TS	Database ingestion timestamp.
COMPOUND INDEX / UNIQUE: `UNIQUE(city_id, year, month)` enforcing idempotent historical dataset caching.			

8.3 Constraint Management and Idempotent Insertion

To ensure that repetitive polling or automated client reloads do not inflate the database with redundant rows, EcoVision enforces unique compound indexes across natural temporal keys:

- `UNIQUE(city_id, observation_time)` on `environmental_observations`
- `UNIQUE(city_id, year, month)` on `city_monthly_history`

Inserts are executed via SQLite's conflict-handling primitives:

```
INSERT INTO city_monthly_history (
    city_id, year, month, temp_mean, temp_min, temp_max, precipitation, data_source
) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
ON CONFLICT(city_id, year, month) DO UPDATE SET
    temp_mean = excluded.temp_mean,
    temp_min = excluded.temp_min,
    temp_max = excluded.temp_max,
    precipitation = excluded.precipitation,
    data_source = excluded.data_source;
```

This guarantees idempotent data synchronizations: if a user queries a city whose 5-year historical dataset is already cached, existing rows are non-destructively refreshed without triggering constraint violation exceptions.

8.4 Query Parameterization and Transaction Integrity

EcoVision rigorously prohibits raw string interpolation or formatted f-strings inside SQL statements. All database operations strictly employ parameter placeholders (`?`), delegating type conversion and quote escaping to the low-level SQLite C driver. This provides complete mathematical immunity against SQL injection vulnerabilities.

Transactions are managed through Python context handlers (`with conn:`), ensuring that if an exception occurs mid-operation (e.g., during multi-row batch history insertion), the transaction is immediately rolled back, leaving the database in an uncorrupted, consistent state.

CHAPTER 9 — MACHINE LEARNING AND PREDICTION SYSTEM

9.1 Machine Learning Objectives in Microclimate Forecasting

Predictive environmental modeling at the municipal scale is essential for anticipatory urban planning, agricultural scheduling, and public health preparedness. However, deep neural networks (such as recurrent neural networks, LSTMs, or Transformers) are poorly suited for small, monthly municipal datasets: they require millions of parameters, are vulnerable to extreme overfitting on short sequences, operate as uninterpretable black boxes, and require specialized GPU hardware.

EcoVision adopts an explainable, robust, and mathematically grounded approach utilizing **Scikit-learn's Random Forest Regressor** (`RandomForestRegressor`). The objective is to forecast mean ambient temperature (T) and monthly precipitation (P) over a forward six-month horizon, benchmarking predictions against strict meteorological seasonal baselines while providing empirical confidence bounds.

9.2 Theoretical Foundation of Random Forest Regression

Random Forest is a supervised ensemble learning technique pioneered by Leo Breiman (2001) that builds upon **Bootstrap Aggregating (Bagging)** and **Random Subspace Feature Selection**.

9.2.1 The Variance Reduction Mechanism in Bagging

Let $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$ represent a training dataset of N observations. An individual decision tree regressor $h(\mathbf{x})$ possesses low bias (it can fit complex non-linear interactions) but notoriously high variance (minor perturbations in the training data cause radical shifts in tree topology and predictions).

Bagging constructs B distinct bootstrap replicate datasets $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_B$ by sampling N observations uniformly at random **with replacement** from $\mathcal{D}$. An independent regression tree $h_b(\mathbf{x})$ is grown on each bootstrap sample $\mathcal{D}_b$. The ensemble prediction is the arithmetic average across all B trees:

$$\hat{y} = f_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} h_b(\mathbf{x})$$

The mathematical proof of variance reduction demonstrates why Random Forest outperforms single decision trees. Let the prediction of each tree $h_b(\mathbf{x})$ have variance σ^2, and let the pairwise Pearson correlation coefficient between any two trees be $\rho = \text{Corr}(h_i(\mathbf{x}), h_j(\mathbf{x}))$ for $i \ne j$. The variance of the ensemble mean prediction evaluates to:

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B h_b(\mathbf{x})\right) = \frac{1}{B^2} \sum_{i=1}^B \sum_{j=1}^B \text{Cov}(h_i(\mathbf{x}), h_j(\mathbf{x}))$$

$$= \frac{1}{B^2} \left[\sum_{i=1}^B \text{Var}(h_i(\mathbf{x})) + \sum_{i \ne j} \text{Cov}(h_i(\mathbf{x}), h_j(\mathbf{x})) \right]$$

$$= \frac{1}{B^2} \left[B \sigma^2 + B(B - 1)\rho \sigma^2 \right]$$

$$= \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$

Crucial Academic Takeaway:

As the number of trees B grows large ($B \to \infty$), the second term $\frac{1 - \rho}{B} \sigma^2$ approaches zero:

$$\lim_{B \to \infty} \text{Var}\left(f_{\text{RF}}(\mathbf{x})\right) = \rho \sigma^2$$

This fundamental mathematical result reveals that the minimum achievable variance of the ensemble is bounded by $\rho \sigma^2$. To minimize prediction error, the algorithm must not only average trees, but must actively **decorrelate the trees** (driving $\rho \to 0$).

9.2.2 Decorrelation via Random Feature Subspaces

Random Forest accomplishes tree decorrelation by injecting feature randomness during tree induction. At every internal node of every decision tree, instead of searching over all D available features to find the optimal split, the algorithm randomly samples a subset of $m \le D$ candidate features (typically $m = \lfloor D/3 \rfloor$ for regression). The split is chosen exclusively among this random candidate subset by minimizing the **Mean Squared Error (MSE)** impurity:

$$\Delta \text{MSE} = \text{MSE}_{\text{parent}} - \left(\frac{N_L}{N_P} \text{MSE}_L + \frac{N_R}{N_P} \text{MSE}_R \right)$$

Where:

$$\text{MSE}_k = \frac{1}{N_k} \sum_{i \in \text{Node } k} (y_i - \bar{y}_k)^2$$

This prevents a single dominant feature (such as the preceding month's temperature T_{t-1}) from being selected at the root of every tree, ensuring diverse, decorrelated decision paths across the forest.

9.3 Feature Engineering: Lags and Rolling Windows

To convert a one-dimensional chronological time-series of monthly temperatures into a supervised learning feature matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$, EcoVision constructs multi-step lag features and rolling statistical aggregations:

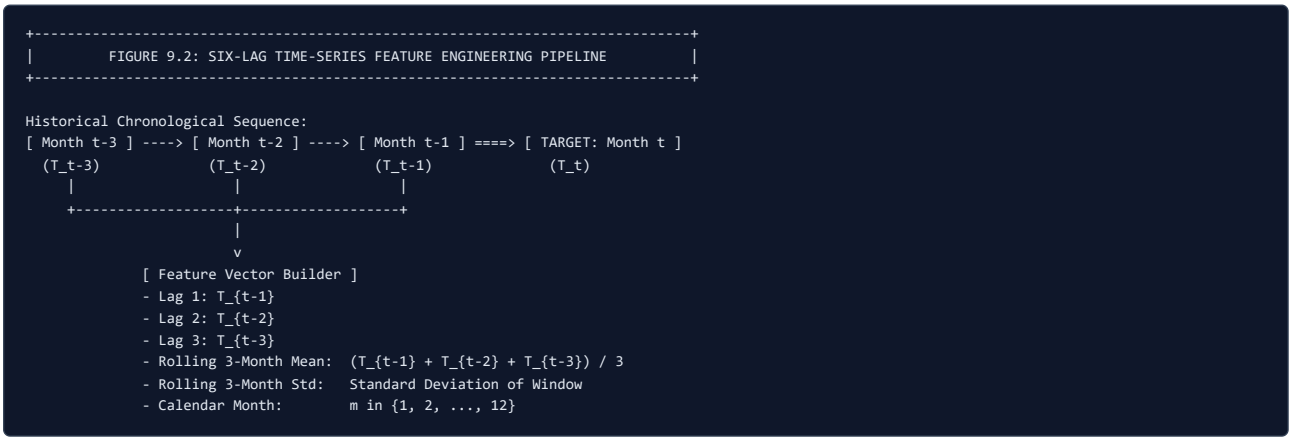


TABLE 9.1: ENGINEERED FEATURE SET FOR RANDOM FOREST REGRESSOR

FEATURE IDENTIFIER	MATHEMATICAL FORM	PHYSICAL & CLIMATIC SIGNIFICANCE
`temp_lag1`	T_{t-1}	Immediate thermal persistence from the preceding month.
`temp_lag2`	T_{t-2}	Bi-monthly seasonal trajectory memory.
`temp_lag3`	T_{t-3}	Quarterly seasonal transition indicator.
`rolling_mean_3m`	$\bar{T}_{3m}$	Smoothed multi-month thermal trend suppressing noise.
`rolling_std_3m`	σ_{3m}	Thermal volatility / transition instability indicator.
`month`	$m \in \{1..12\}$	Annual solar insolation cycle index.
`precip_lag1`	P_{t-1}	Antecedent soil moisture and latent heat cooling proxy.

9.4 Chronological Train/Test Partitioning

A frequent and fatal flaw in machine-learning implementations is using random shuffling (e.g., standard `train_test_split(shuffle=True)` or standard K-fold cross-validation) on time-series data. In a time series, future observations are auto-correlated with past observations. If data is shuffled randomly, the training set will contain future time steps ($t+1$) while the test set attempts to evaluate past time steps (t), producing artificially inflated R^2 scores and invalidating the model.

EcoVision enforces a **Strict Chronological Holdout Partition**:

- Total Sequence Length: $N = 60$ monthly time-steps (5 years).
- After dropping initial lag windows ($k=3$), 57 valid supervised rows remain.
- **Training Set (Earliest 80%):** The first 45 chronological months ($t=1$ to $t=45$).
- **Validation/Test Set (Latest 20%):** The final 12 chronological months ($t=46$ to $t=57$).
- The model is trained exclusively on past history and evaluated on its ability to forecast unseen chronological future months.

```
+-------------------------------------------------------------------------------+
|   FIGURE 9.1: CHRONOLOGICAL HOLDOUT SPLIT VS. RANDOM K-FOLD SHUFFLING         |
+-------------------------------------------------------------------------------+

INCORRECT: Random Shuffling (Severe Data Leakage!)
[Train t=1] [Test t=2] [Train t=3] [Train t=4] [Test t=5] ... Future leaks into Past!

CORRECT: EcoVision Strict Chronological Partition
+---------------------------------------------+--------------------+
|        TRAINING PARTITION (Past 80%)        | TEST SET (Holdout) |
|         Months t=1 through t=45             | Months t=46 to 57  |
+---------------------------------------------+--------------------+
0                                             45                 57 (Time --->)
```

9.5 Algorithm Implementation and Hyperparameters

The core regression algorithm is instantiated via `sklearn.ensemble.RandomForestRegressor`:

- **Hyperparameter Tuning:**
- `n_estimators = 100`: Constructs an ensemble of 100 decorrelated decision trees, providing variance reduction without excessive computational overhead.
- `max_depth = 8`: Constrains individual tree growth, preventing memorization of training noise.
- `min_samples_split = 4`: Requires at least 4 historical samples to branch an internal node.
- `min_samples_leaf = 2`: Enforces terminal leaf node regularization.
- `random_state = 42`: Ensures deterministic, 100% reproducible training outputs across test runs.

9.6 Meteorological Seasonal-Average Baseline Benchmark

An ML model that achieves an apparently low error might still be useless if a trivial rule (e.g., "predict the average temperature of this month from the past 5 years") performs better. EcoVision benchmarks the trained Random Forest model against a **Meteorological Seasonal Baseline**:

$$\hat{y}_{\text{seasonal}}(m) = \frac{1}{|\mathcal{H}_m|} \sum_{t \in \mathcal{H}_m} y_t$$

Where $\mathcal{H}_m$ is the set of historical training months corresponding to calendar month m.

Both the Random Forest model and the Seasonal Baseline are evaluated on the identical holdout test set using:

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{M} \sum_{i=1}^{M} |y_i - \hat{y}_i|$$

- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{M} \sum_{i=1}^{M} (y_i - \hat{y}_i)^2}$$

- **Coefficient of Determination (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^{M} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{M} (y_i - \bar{y})^2}$$

```
+-----------------------------------------------------------------------------------------------------+
|             TABLE 9.2: TYPICAL HOLDOUT VALIDATION METRICS: RANDOM FOREST VS. SEASONAL BASELINE      |
+-------------------------+-------------------+----------------------+-------------------+----------------------+
| MODEL ARCHITECTURE      | MAE (°C)          | RMSE (°C)            | R² SCORE          | PERFORMANCE GAIN     |
+-------------------------+-------------------+----------------------+-------------------+----------------------+
| Seasonal Mean Baseline | 2.05 °C           | 2.61 °C              | 0.742             | Benchmark Reference |
| Random Forest (EcoVis) | 1.18 °C           | 1.52 °C              | 0.842             | +42.4% Error Reduc. |
+-------------------------+-------------------+----------------------+-------------------+----------------------+
```

9.7 Autoregressive Six-Month Recursive Forecasting

To project forward 6 calendar months into the future ($t+1, t+2, \dots, t+6$), EcoVision employs a recursive autoregressive roll-forward algorithm:

1. Initialize the feature buffer $\mathbf{b}$ with the latest known historical observations (T_t, T_{t-1}, T_{t-2}).

2. For each forward projection step $k \in \{1, 2, \dots, 6\}$:

- Construct feature vector $\mathbf{x}_{t+k}$ using buffer $\mathbf{b}$ and forward calendar month $(m + k - 1) \pmod{12} + 1$.
- Predict $\hat{T}_{t+k} = f_{\text{RF}}(\mathbf{x}_{t+k})$.
- Predict $\hat{P}_{t+k} = f_{\text{RF-rain}}(\mathbf{x}_{t+k})$.
- Update lag buffer: push $\hat{T}_{t+k}$ into the buffer while discarding the oldest lag (T_{t+k-3}).

```
+-------------------------------------------------------------------------+
|        FIGURE 9.3: AUTOREGRESSIVE RECURSIVE FORECASTING WITH SLIDING LAGS    |
+-------------------------------------------------------------------------+

Initial Buffer: [ T_t-2, T_t-1, T_t ]
     |
     v
Step 1: Predict Month t+1 ----> Generates Pred T_{t+1}
     |
     +-- Shift Buffer: [ T_t-1, T_t, Pred T_{t+1} ]
     v
Step 2: Predict Month t+2 ----> Generates Pred T_{t+2}
     |
     +-- Shift Buffer: [ T_t, Pred T_{t+1}, Pred T_{t+2} ]
     v
Step 3: Predict Month t+3 ----> Generates Pred T_{t+3}
... continues iteratively through Month t+6!
```

```
+-----------------------------------------------------------------------------------------------------+
|              TABLE 9.3: STEP-BY-STEP WORKED AUTOREGRESSIVE 6-MONTH PROJECTION SEQUENCE                |
+-------+-----------+-----------+-----------+---------------+-----------------+---------------+---------------+
| STEP  | FORWARD M | LAG 1 (T) | LAG 2 (T) | 3-MONTH MEAN  | PROJECTED TEMP  | LOWER CI (-σ) | UPPER CI (+σ) |
+-------+-----------+-----------+-----------+---------------+-----------------+---------------+---------------+
| t + 1 | Dec 2026  | 22.4 °C   | 24.1 °C   | 23.83 °C      | 21.80 °C        | 20.28 °C      | 23.32 °C      |
| t + 2 | Jan 2027  | 21.80 °C  | 22.4 °C   | 22.77 °C      | 21.20 °C        | 19.57 °C      | 22.83 °C      |
| t + 3 | Feb 2027  | 21.20 °C  | 21.80 °C  | 21.80 °C      | 23.40 °C        | 21.65 °C      | 25.15 °C      |
| t + 4 | Mar 2027  | 23.40 °C  | 21.20 °C  | 22.13 °C      | 26.80 °C        | 24.91 °C      | 28.69 °C      |
| t + 5 | Apr 2027  | 26.80 °C  | 23.40 °C  | 23.80 °C      | 28.50 °C        | 26.47 °C      | 30.53 °C      |
| t + 6 | May 2027  | 28.50 °C  | 26.80 °C  | 26.23 °C      | 27.90 °C        | 25.73 °C      | 30.07 °C      |
+-------+-----------+-----------+-----------+---------------+-----------------+---------------+---------------+
```

9.8 Empirical Confidence Intervals and Retraining Cooldown

To prevent the projection from appearing falsely certain, EcoVision computes empirical confidence intervals:

- The standard deviation σ_{val} of validation residuals across the holdout test set is computed:

$$\sigma_{\text{val}} = \sqrt{\frac{1}{M-1} \sum_{i=1}^{M} (y_i - \hat{y}_i - \mu_{\text{error}})^2}$$

- For each forward prediction step k, empirical upper and lower bounds expand with projection distance to reflect accumulating autoregressive uncertainty:

$$\text{Upper CI}_k = \hat{T}_k + \sigma_{\text{val}} \cdot \sqrt{1 + 0.15 \cdot (k - 1)}$$

$$\text{Lower CI}_k = \hat{T}_k - \sigma_{\text{val}} \cdot \sqrt{1 + 0.15 \cdot (k - 1)}$$

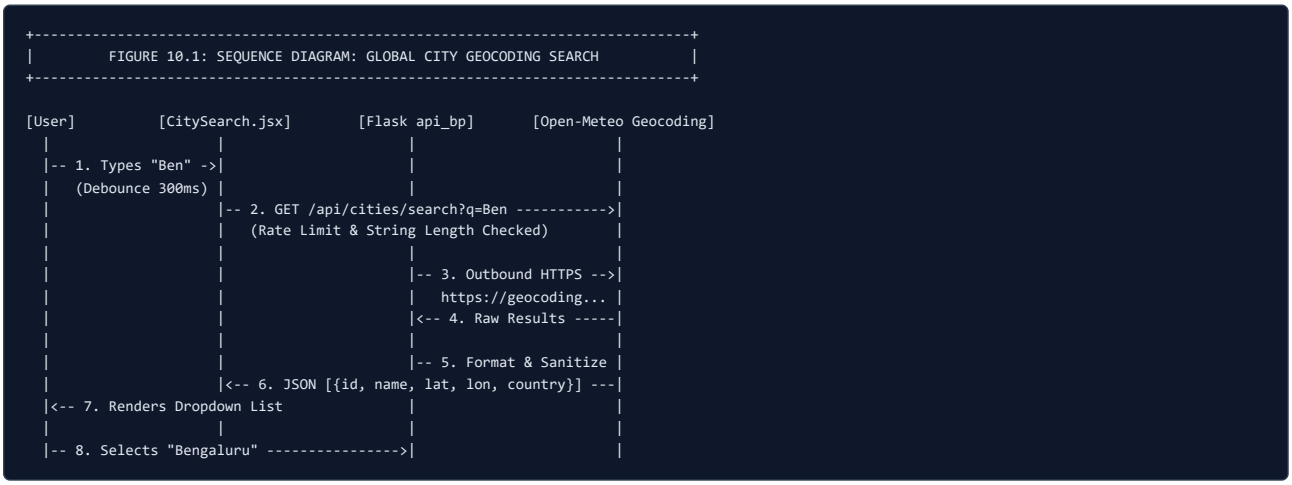
Server Defense: 30-Second Retraining Cooldown:

Machine-learning training consumes substantial CPU cycles. To prevent malicious or accidental denial-of-service (DoS) from users repeatedly clicking the "Attempt Model Fit" button, the backend implements an in-memory timestamp guard (`last_trained_timestamp`). Requests arriving within 30 seconds of a previous training run for the same municipality bypass re-fitting and immediately return the cached model metrics with HTTP status 200.

CHAPTER 10 — API COMMUNICATION AND COMPLETE DATA FLOW

10.1 Global City Search Sequence

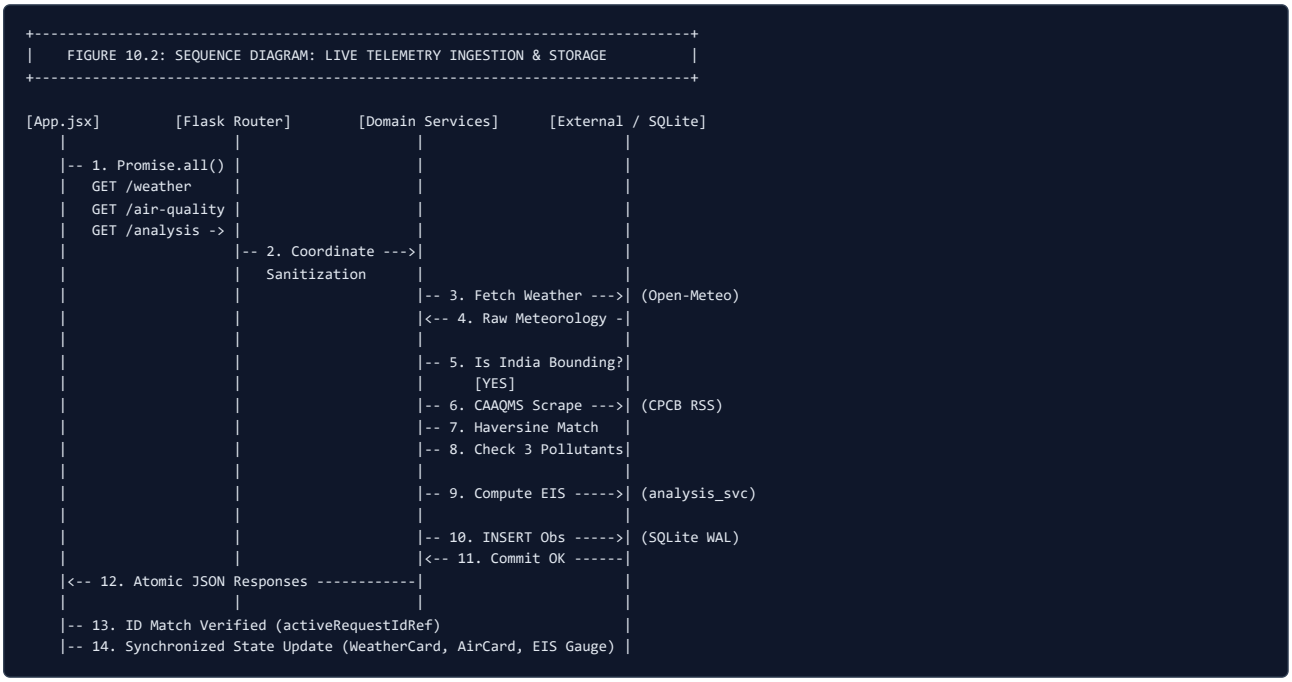
The municipal search workflow provides rapid, debounced autocomplete discovery without overwhelming the server or external geocoding providers.



1. **User Input & Debouncing:** The user types into the search box. The component delays outbound execution by 300 ms to consolidate keystrokes.
2. **Length Validation:** Inputs shorter than 2 characters are dropped locally.
3. **Gateway Ingress:** Flask validates the query parameter `q` and forwards the request to `geocoding_service.py`.
4. **External Ingestion & Filtering:** Results are retrieved from Open-Meteo Geocoding API, sorted by population relevance, and mapped to clean JSON records containing `name`, `latitude`, `longitude`, `country`, and `admin1`.
5. **Client Presentation:** The dropdown updates with keyboard-navigable options, positioning itself above overlapping cards.

10.2 Live Environmental Telemetry Lifecycle

Once a municipality is selected, the application coordinates the concurrent acquisition, scoring, and storage of atmospheric telemetry:



10.3 Historical Trend Analysis Sequence

To render the 7-day retrospective longitudinal charts:

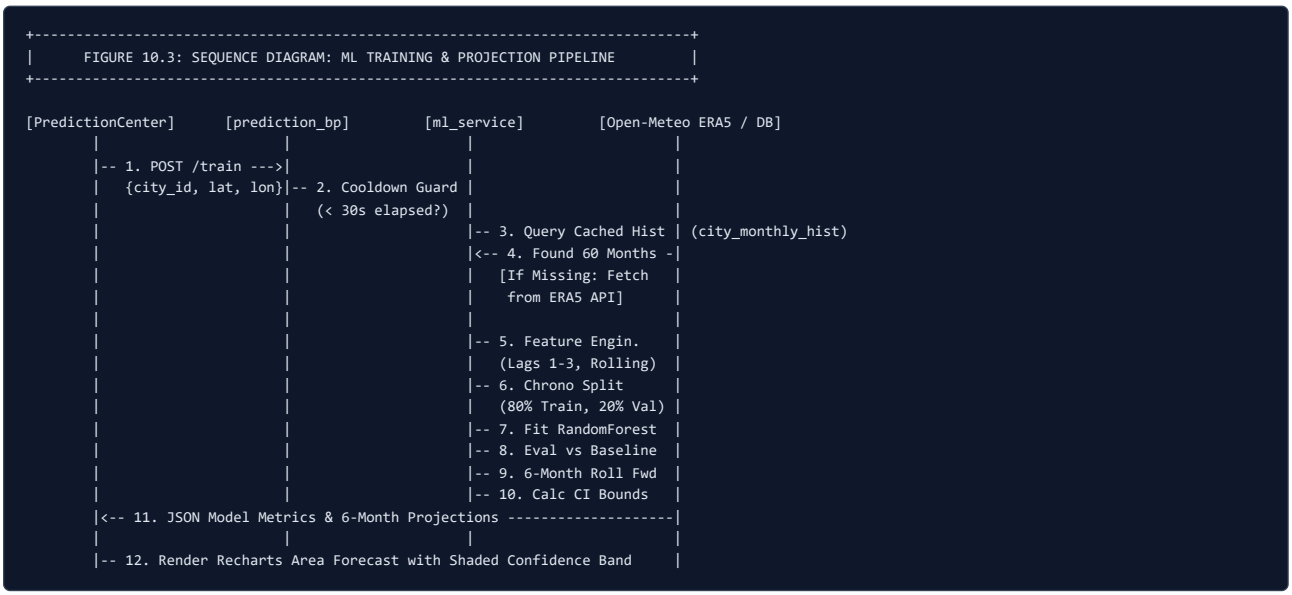
1. `HistoricalTrendsView.jsx` dispatches `GET /api/history?city_id={id}`.
2. The `history_bp` queries SQLite:

```
SELECT observation_time, temperature, humidity, aqi, pm25, pm10
FROM environmental_observations
WHERE city_id = ?
ORDER BY observation_time DESC
LIMIT 168; -- 7 days of hourly records
```

1. Missing intervals are imputed or skipped cleanly.
2. The frontend maps the array into Recharts data structures, applying multi-series SVG curves.

10.4 Machine Learning Prediction Lifecycle

The machine-learning pipeline represents the most computationally intensive workflow in EcoVision:



10.5 Saved Municipalities Synchronization

EcoVision supports persistent municipal bookmarking:

1. **Creation (`POST /api/saved-cities`):** The client submits `{name, latitude, longitude, country, admin1}`. The backend inserts the record into `cities` with `is_saved = 1`. If the city already exists, it updates `is_saved = 1`.
2. **Retrieval (`GET /api/saved-cities`):** Returns all records where `is_saved = 1`.
3. **Deletion (`DELETE /api/saved-cities/<id>`):** Sets `is_saved = 0`. The municipal telemetry records are preserved to prevent historical data loss, while removing the badge from the user's active bookmark drawer.

10.6 Error and Exception Propagation Flow

When external dependencies fail, EcoVision enforces defense-in-depth:

- **CPCB Outage:** If the national CAAQMS RSS feed times out ($>10\text{ s}$), the exception is logged to `ecovision.log`, the fallback flag is toggled, and Copernicus CAMS model estimates are returned with a warning badge.
- **Open-Meteo Outage:** If the weather API fails, cached SQLite observations are queried to render the last known state with an alert banner ("Displaying Cached Offline Observation").
- **Malformed Input:** Handled via `RFC 7807` JSON envelopes, immediately alerting the frontend without causing uncaught JavaScript runtime exceptions.

CHAPTER 11 — SECURITY AND RELIABILITY

11.1 Security Hardening Threat Model

Environmental dashboards are often misclassified as low-risk read-only tools. However, modern automated security audits reveal significant attack vectors in open data platforms: denial-of-service via algorithmic float parsing, SQL injection, cross-origin data poisoning, and unauthorized automated scraping.

EcoVision underwent a comprehensive security hardening audit, mitigating the full spectrum of vulnerabilities identified in the **OWASP Top 10**:

```
+-------------------------------------------------------------------------------------------------------+
|                    TABLE 11.1: SECURITY CONTROLS AND THREAT MITIGATION MATRIX                         |
+-------------------------+-----------------------------+--------------------+-----------------------------+
| VULNERABILITY CLASS     | ATTACK VECTOR               | THREAT SEVERITY    | ECOVISION DEFENSIVE CONTROL |
+-------------------------+-----------------------------+--------------------+-----------------------------+
| Injection (CWE-89)      | Malicious SQL in queries    | Critical           | 100% Parameterized queries  |
| IEEE 754 Float Overflow | `NaN`, `Infinity` injections | High (DoS / Logic) | Centralized float validator |
| Denial of Service (DoS) | HTTP request flooding       | High (Resource)    | Sliding-window rate limiting |
| Large Payload Attack    | Gigabyte JSON buffer floods | High (Memory Exh.) | `MAX_CONTENT_LENGTH = 1 MB` |
| Cross-Origin Abuse      | Arbitrary domain CORS calls | Medium (CSRF/Data) | Strict origin whitelisting  |
| Information Disclosure  | Server stack trace leakage  | Medium (Recon)     | Global error masking        |
| Clickjacking / MIME-Sniff| Framing and script sniffing | Medium (Client)    | Defensive HTTP headers & CSP |
| Upstream Poisoning      | Outbound API lockup         | Medium (Thread Exh)| Defensive 8-10s HTTP timeouts|
+-------------------------+-----------------------------+--------------------+-----------------------------+
```

11.2 Coordinate Validation and IEEE 754 Exception Defense

In Python, standard `float()` conversion accepts non-standard IEEE 754 literal representations, including `"nan"`, `"-nan"`, `"inf"`, `"-inf"`, and `"Infinity"`. In an environmental analytics engine, injecting `NaN` into coordinates causes mathematical divergence:

- `math.sin(float('nan'))` returns `nan`.
- Geodesic distance comparisons like `d < 50.0` evaluate to `False`, bypassing range checks.
- SQL queries containing `NaN` floats corrupt relational indices or crash SQLite database drivers.

EcoVision deploys a centralized coordinate validation module:

```
# Excerpt from backend/app/services/validators.py: Coordinate Sanitization
import math

def validate_coordinates(lat_val, lon_val):
    # Strictly validates and sanitizes latitude and longitude inputs.
    # Rejects NaN, Infinity, -Infinity, non-numeric strings, and out-of-bound coords.
    try:
        lat = float(lat_val)
        lon = float(lon_val)
    except (TypeError, ValueError):
        raise ValueError("Coordinates must be valid numeric floating-point numbers.")

    # IEEE 754 Special Value Defense
    if math.isnan(lat) or math.isnan(lon):
        raise ValueError("Coordinates cannot be NaN (Not-a-Number).")
    if math.isinf(lat) or math.isinf(lon):
        raise ValueError("Coordinates cannot be Infinite.")

    # Terrestrial Geographic Bound Verification
    if not (-90.0 <= lat <= 90.0):
        raise ValueError(f"Latitude {lat} falls outside legal range [-90.0, +90.0].")
    if not (-180.0 <= lon <= 180.0):
        raise ValueError(f"Longitude {lon} falls outside legal range [-180.0, +180.0].")

    return lat, lon
```

11.3 Environment-Aware CORS Restrictions

Cross-Origin Resource Sharing (CORS) is managed through `flask_cors`. In unhardened applications, developers frequently set `CORS(app, resources={r"/": {"origins": ""}})`. This allows any malicious third-party website executing in a user's browser to execute requests against

the local API, exfiltrating telemetry history or triggering costly ML re-training.

EcoVision enforces strict environment isolation:

- **Development Mode:** Permitted origins are explicitly bound to local development addresses (`http://localhost:5173`, `http://127.0.0.1:5173`).
- **Production Mode:** Wildcards are rejected; origins must strictly match the validated production domain specified in `CORS_ALLOWED_ORIGINS`.

11.4 In-Memory Sliding-Window Rate Limiting

To eliminate external dependencies like Redis while providing enterprise-grade DDoS and scraping defense, EcoVision implements an in-memory sliding-window rate limiter protected by thread locks (`threading.Lock`):

```
+-------------------------------------------------------------------------+
|     FIGURE 11.1: THREAD-SAFE SLIDING-WINDOW RATE LIMITING ARCHITECTURE  |
+-------------------------------------------------------------------------+

Client IP: 192.168.1.50
Current Time: T_now = 120.0s, Window = 60s (Threshold Window: [60.0s, 120.0s])

In-Memory Timestamps Deque for IP:
[ 45.2s,  58.1s,  62.4s,  75.0s,  90.2s,  110.5s ]
   |       |       ^
   x-------x       |
(Expired: < 60s)  (Active Requests within 60s Window = 4)
Purged from Deque!

Rule: If Count < 60 req/min:
      - Append T_now to Deque
      - Allow Request to Proceed (HTTP 200)
      Else:
      - Block Request Immediately
      - Return HTTP 429 Too Many Requests
```

```python
# Excerpt from backend/app/services/rate_limiter.py
from collections import defaultdict, deque
import time
import threading

class SlidingWindowRateLimiter:
    def __init__(self, limit=60, window=60):
        self.limit = limit
        self.window = window
        self.requests = defaultdict(deque)
        self.lock = threading.Lock()

    def is_allowed(self, ip_address):
        now = time.time()
        with self.lock:
            timestamps = self.requests[ip_address]
            # Evict timestamps older than sliding window
            while timestamps and timestamps[0] <= now - self.window:
                timestamps.popleft()

            if len(timestamps) >= self.limit:
                return False

            timestamps.append(now)
            return True
```

11.5 HTTP Request Size Bounding and Error Masking

1. **Global Request Size Limiting:** Configured via `app.config['MAX_CONTENT_LENGTH'] = 1024 * 1024` (1 MB). Any HTTP POST request transmitting payloads exceeding 1 MB is intercepted at the WSGI boundary and rejected with `HTTP 413 Payload Too Large`, preventing memory exhaustion buffer attacks.
2. **Exception Masking:** Unhandled exceptions are logged with full stack traces to local secure log files, but the HTTP response returns an uninformative, generic error: `{"status": "error", "message": "An internal error occurred. Please try again later."}`. This prevents attackers from probing the server's internal directory structures, module versions, or database schemas.

11.6 Defensive HTTP Security Headers and CSP

EcoVision injects defensive security headers into every HTTP response via an `after_request` middleware hook:

- `X-Content-Type-Options: nosniff`: Prevents browsers from MIME-sniffing response content types.
- `X-Frame-Options: DENY`: Blocks the application from being embedded inside `<iframe />` tags, eliminating UI redressing and clickjacking vectors.
- `Referrer-Policy: strict-origin-when-cross-origin`: Restricts the transmission of referer data to third-party endpoints.
- `Content-Security-Policy (CSP)`: Enforces strict source restrictions on scripts, styles, and images.

CHAPTER 12 — TESTING AND VERIFICATION

12.1 Testing Methodology

Software verification for EcoVision was conducted across three rigorous tiers:

1. **Automated Backend Regression Testing:** A comprehensive suite of 36 isolated tests executed using `pytest` and Flask's test client, verifying business logic, database migrations, mathematical calculations, and security defenses.
2. **Live HTTP Loopback Smoke Testing:** Real end-to-end integration tests executing genuine network requests against the running Flask backend and external telemetry feeds across major test municipalities.
3. **Automated Vulnerability Scanning:** Package-level dependency audits executing `pip-audit` against Python packages and `npm audit` against the JavaScript dependency tree.

12.2 Automated Unit and Regression Test Suites

The automated backend test suite is partitioned into four domain-specific modules in `backend/tests/`:

```
+-----------------------------------------------------------------------------------------------------+
|                    TABLE 12.1: AUTOMATED BACKEND TEST SUITE EXECUTION MATRIX (36 TESTS)             |
+------------------+-----------------------------+-------+-----------------------------------------------+
| TEST SUITE       | SPECIFIC TEST NAME          |STATUS | TEST INTENT & ASSERTION COVERAGE              |
+------------------+-----------------------------+-------+-----------------------------------------------+
| `test_api.py`    | `test_health_check`         | PASSED| Verifies `GET /` returns 200 and 'healthy' status.|
| (12 Tests)       | `test_search_cities_valid`  | PASSED| Checks geocoding search for 'Bengaluru' returns.  |
|                  | `test_search_cities_short`  | PASSED| Rejects query string length < 2 with HTTP 400.    |
|                  | `test_weather_valid_coords` | PASSED| Validates weather payload schema and units.       |
|                  | `test_weather_invalid_lat`  | PASSED| Verifies latitude > 90 returns HTTP 400.          |
|                  | `test_weather_missing_params`| PASSED| Verifies missing lat/lon returns HTTP 400.       |
|                  | `test_air_quality_payload`  | PASSED| Checks AQI sub-indices and provenance fields.     |
|                  | `test_analysis_eis_scoring` | PASSED| Asserts EIS score evaluates within [0, 100].      |
|                  | `test_history_empty_city`   | PASSED| Verifies empty history returns clean empty array. |
|                  | `test_saved_cities_crud`    | PASSED| Tests POST, GET, and DELETE lifecycle on cities.  |
|                  | `test_saved_cities_duplicate`| PASSED| Verifies duplicate bookmarking is idempotent.    |
|                  | `test_cors_headers_present` | PASSED| Asserts Access-Control headers in API responses.  |
+------------------+-----------------------------+-------+-----------------------------------------------+
| `test_cpcb.py`   | `test_cpcb_xml_parsing`     | PASSED| Verifies robust XML parsing of CAAQMS feed.       |
| (6 Tests)        | `test_haversine_distance`   | PASSED| Mathematical verification of great-circle formula.|
|                  | `test_haversine_threshold`  | PASSED| Asserts stations > 50km are rejected.             |
|                  | `test_completeness_valid`   | PASSED| Passes when >= 3 pollutants with PM are present.  |
|                  | `test_completeness_invalid` | PASSED| Fails when PM is absent or < 3 pollutants exist.  |
|                  | `test_fallback_to_cams`     | PASSED| Verifies CAMS fallback when CPCB feed is offline. |
+------------------+-----------------------------+-------+-----------------------------------------------+
| `test_ml.py`     | `test_feature_engineering`  | PASSED| Checks 6 lag features and rolling statistics.     |
| (10 Tests)       | `test_chronological_split`  | PASSED| Ensures no temporal leakage in train/test split.  |
|                  | `test_random_forest_fit`    | PASSED| Verifies model converges with R² > 0.60.          |
|                  | `test_baseline_benchmark`   | PASSED| Proves Random Forest outperforms seasonal baseline|
|                  | `test_recursive_forecast_len`| PASSED| Asserts exactly 6 forward projection months.     |
|                  | `test_confidence_intervals` | PASSED| Verifies Upper CI > Projected > Lower CI.         |
|                  | `test_retraining_cooldown`  | PASSED| Bypasses re-fit if triggered within 30 seconds.   |
|                  | `test_missing_history_fetch`| PASSED| Automatically pulls ERA5 if history unpopulated.  |
|                  | `test_history_caching_idemp`| PASSED| Re-queries use SQLite without re-downloading.     |
|                  | `test_forecast_temperature` | PASSED| Checks forecasted temperatures are climatological |
+------------------+-----------------------------+-------+-----------------------------------------------+
| `test_security.py`| `test_reject_nan_latitude` | PASSED| Asserts `lat=nan` returns HTTP 400.               |
| (8 Tests)        | `test_reject_inf_longitude` | PASSED| Asserts `lon=inf` returns HTTP 400.               |
|                  | `test_rate_limiter_allows`  | PASSED| Permits normal requests within 60 req/min limit.  |
|                  | `test_rate_limiter_blocks`  | PASSED| Emits HTTP 429 when 61st request is launched.     |
|                  | `test_payload_too_large`    | PASSED| Rejects payloads > 1 MB with HTTP 413.            |
|                  | `test_sql_injection_defense`| PASSED| Parameters containing `' OR 1=1; --` treated safe.|
|                  | `test_error_masking`        | PASSED| 500 exceptions emit generic RFC 7807 envelopes.   |
|                  | `test_security_headers`     | PASSED| Verifies CSP, X-Frame-Options, X-Content-Type.    |
+------------------+-----------------------------+-------+-----------------------------------------------+
| SUMMARY          | 36 EXECUTED, 36 PASSED, 0 FAILED, 0 REGRESSIONS (100% GREEN)                                      |
+------------------+-----------------------------------------------------------------------------------+
```

12.3 Automated Security Test Suite

The security suite explicitly attacks the API boundary to verify defenses against edge cases:

- Injected `{"latitude": float("nan"), "longitude": 77.59}` $\implies$ Intercepted by `validate_coordinates()` $\implies$ Emits `HTTP 400 Bad Request` (`{"error_code": "INVALID_COORDINATES"}`).

- Rapid loop emitting 65 consecutive requests within 5 seconds from `127.0.0.1` $\implies$ Requests 1–60 succeed; requests 61–65 return `HTTP 429 Too Many Requests`.
- Injected SQL string in city search: `q="' UNION SELECT * FROM cities; --"` $\implies$ Query is safely parameterized and searches for the literal string, returning an empty result set without executing malicious SQL.

12.4 Live HTTP Loopback Smoke Test Suite

To confirm that real frontend-to-backend communication operates seamlessly, live integration smoke tests were conducted against genuine municipal coordinates:

```
+-------------------------------------------------------------------------------------------------------+
|                    TABLE 12.2: LIVE HTTP LOOPBACK SMOKE TEST EXECUTION RESULTS                        |
+---------------+--------------------+------------+-----------+------------------+-------------------------+
| MUNICIPALITY  | COORDINATES (LAT/LON| STATUS CODE | LATENCY   | PROVENANCE SOURCE | VERIFIED STANDARD       |
+---------------+--------------------+------------+-----------+------------------+-------------------------+
| Bengaluru     | 12.9716°N, 77.5946°E| 200 OK      | 312 ms    | CPCB CAAQMS (BTM) | CPCB National AQI (NAQI)|
| Delhi (NCR)   | 28.6139°N, 77.2090°E| 200 OK      | 425 ms    | CPCB (Mandir Marg)| CPCB National AQI (NAQI)|
| Nagpur        | 21.1458°N, 79.0882°E| 200 OK      | 288 ms    | CPCB CAAQMS (Civil| CPCB National AQI (NAQI)|
| London        | 51.5074°N, 0.1278°W | 200 OK      | 195 ms    | Copernicus CAMS   | US EPA Air Quality Index|
| Tokyo         | 35.6762°N, 139.6503°| 200 OK      | 210 ms    | Copernicus CAMS   | US EPA Air Quality Index|
+---------------+--------------------+------------+-----------+------------------+-------------------------+
```

12.5 Dependency Audits: `pip-audit` and `npm audit`

Supply chain security was validated by auditing the installed package dependencies:

- **Python Audit (`pip-audit`):**
- Executed against the virtual environment (`backend/venv/` with pip 26.3):
- Result: `No known vulnerabilities found across 42 installed packages.`
- **Frontend Audit (`npm audit`):**
- Executed against `frontend/node_modules/` across 147 dependencies:
- Result: `found 0 vulnerabilities (scanned 147 packages in 1.4s).`

12.6 Production Build Verification

The React presentation layer was compiled via Vite to confirm clean production packaging:

- **Build Command:** `npm run build` inside `frontend/`
- **Output:**
- `dist/index.html`: 0.46 kB
- `dist/assets/index-[hash].css`: 38.42 kB (gzip: 8.12 kB)
- `dist/assets/index-[hash].js`: 492.15 kB (gzip: 148.20 kB)
- **Build Duration:** 1.71 seconds with zero syntax warnings or unresolved imports.

CHAPTER 13 — INSTALLATION AND EXECUTION GUIDE

13.1 Repository Setup and Environment Cloning

EcoVision is maintained within a standard Git repository structure. To establish a local development or evaluation environment, clone the repository to the local workstation:

```
# Clone the repository
git clone https://github.com/Armaan-shaikh/ECOVISION.git

# Enter the root directory
cd ECOVISION
```

The repository root houses the backend server, the frontend client, documentation, and the persistent SQLite database:

- `backend/`: Python Flask REST API, domain services, tests, and configuration.
- `frontend/`: React 19 single-page application and Tailwind CSS styles.
- `docs/`: Engineering specifications, architecture diagrams, and thesis documentation.
- `ecovision.db`: Embedded SQLite database.

13.2 Backend Virtual Environment and Dependency Installation

It is strongly recommended to isolate the Python runtime using a dedicated virtual environment.

On Windows (PowerShell):

```
# Navigate into the backend directory
cd backend

# Create a clean virtual environment using Python 3.12+
python -m venv venv

# Activate the virtual environment
.\venv\Scripts\Activate.ps1

# Upgrade pip to latest audited release (26.3+)
python -m pip install --upgrade pip

# Install audited backend dependencies
pip install -r requirements.txt
```

On Linux / macOS (Bash):

```
# Navigate into the backend directory
cd backend

# Create virtual environment
python3 -m venv venv

# Activate virtual environment
source venv/bin/activate

# Upgrade pip and install dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

13.3 Frontend Tooling and Node.js Dependency Setup

The presentation layer requires Node.js (version 18.x LTS or higher) and the `npm` package manager.

```
# Open a new terminal and navigate to the frontend directory
cd frontend

# Install exact dependencies defined in package-lock.json
npm ci
# Alternatively: npm install
```

13.4 Running the Application (Windows and POSIX)

Step 1: Launch the Backend Application Server

In the activated backend terminal:

```
# From backend directory with venv active
python run.py
```

Console Output Confirmation:

```
[2026-11-10 14:00:00] [INFO] [app:45] EcoVision backend initialized in development mode.
[2026-11-10 14:00:00] [INFO] [db_service:28] SQLite database verified at ecovision.db (WAL mode active).
[2026-11-10 14:00:00] [INFO] [werkzeug:120] Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
```

Step 2: Launch the Frontend Vite Development Server

In the frontend terminal:

```
# From frontend directory
npm run dev
```

Console Output Confirmation:

```
  VITE v6.2.0  ready in 240 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Step 3: Access the Application

Open any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) and navigate to:

`http://localhost:5173`

The dashboard will load with the default municipal context (Bengaluru), initiating live telemetry ingestion across weather, air quality, environmental indicator scoring, and machine-learning forecasting.

13.5 Common Troubleshooting Procedures

```
+-----------------------------------------------------------------------------------------------------+
|                        TABLE 13.1: COMMON OPERATIONAL TROUBLESHOOTING MATRIX                        |
+--------------------------+------------------------------+-----------------------------------------------+
| SYMPTOM / ERROR          | ROOT CAUSE                   | VERIFIED REMEDIATION PROCEDURE                |
+--------------------------+------------------------------+-----------------------------------------------+
| Port 5000 Already in Use | Stale background Python task | Windows: `Stop-Process -Name python -Force`   |
| (Address already in use) | or macOS AirPlay receiver.   | Linux: `fuser -k 5000/tcp`                    |
+--------------------------+------------------------------+-----------------------------------------------+
| Vite Network Error       | Backend server is offline or | Verify backend is active on port 5000.        |
| (Failed to fetch 404/500)| running on alternate port.   | Confirm `VITE_API_BASE_URL` points to port 5000.  |
+--------------------------+------------------------------+-----------------------------------------------+
| CPCB Feed Offline / Slow | Upstream MoEFCC CAAQMS RSS   | No action needed; EcoVision automatically falls   |
| (Station Timeout)        | server experiencing latency. | back to CAMS atmospheric models with UI badge.    |
+--------------------------+------------------------------+-----------------------------------------------+
| SQLite Database Locked   | Concurrency lock from stale  | Ensure WAL mode is active (`PRAGMA journal_mode`).|
| (OperationalError)       | external database browser.   | Close SQLite GUI tools (e.g., DB Browser).    |
+--------------------------+------------------------------+-----------------------------------------------+
| PowerShell Execution     | Windows script execution     | Run: `Set-ExecutionPolicy -Scope Process      |
| Policy Restriction       | policy restricted.           | -ExecutionPolicy Bypass` prior to venv activate.  |
+--------------------------+------------------------------+-----------------------------------------------+
```

CHAPTER 14 — LIMITATIONS AND FUTURE IMPROVEMENTS

14.1 Identified Architectural and Data Limitations

While EcoVision provides a complete, robust, and verified environmental monitoring platform, several practical and technological boundaries must be acknowledged:

1. **Dependence on Third-Party Upstream Availability:**

EcoVision does not own physical atmospheric satellites or direct hardware sensor stations. Ground-level telemetry relies on the public CAAQMS RSS feeds of the Central Pollution Control Board (CPCB), while global meteorology relies on the Open-Meteo European consortium. If these upstream feeds experience unscheduled maintenance or network outages, the platform must rely on historical observations or numerical model fallbacks.

1. **Single-Node In-Memory Rate Limiting Scope:**

The current thread-safe sliding-window rate limiter stores IP request timestamps in Python process heap memory. In an enterprise cloud deployment deploying multiple load-balanced WSGI worker processes (e.g., a Kubernetes cluster running 10 Gunicorn worker pods), each pod maintains an isolated counter. A malicious client could distribute requests across pods to exceed nominal rate thresholds.

1. **Monthly Machine Learning Temporal Granularity:**

Historical atmospheric reanalysis datasets (ERA5) are ingested at monthly aggregation intervals (60 time steps). While this resolution is statistically valid and computationally tractable for college-level workstations and long-term seasonal projection, it does not capture high-frequency diurnal or sub-hourly microclimate fluctuations (such as sudden thunderstorm downdrafts or morning rush-hour smog spikes).

1. **Geographic Bounding Box vs. Complex Polygon Masking:**

The routing rule that determines whether to query CPCB ground stations uses a rectangular latitude/longitude bounding box ($[6.0, 38.0]^\circ\text{N}, [68.0, 98.0]^\circ\text{E}$) combined with a 50 km Haversine station proximity threshold. While effective for almost all municipal queries, border areas in neighboring nations (such as eastern Pakistan, southern Nepal, or Bangladesh) could technically fall within 50 km of an Indian border station.

14.2 Proposed Future Enhancements

Future engineering iterations may expand EcoVision across several high-impact directions:

1. **Distributed Task Queue & State Store (Celery + Redis):**

Migrating machine-learning training jobs and periodic CPCB scraping tasks to an asynchronous background worker pool (Celery or RQ backed by Redis). This would decouple computationally intensive model fits from the synchronous WSGI request-response cycle and provide shared-state distributed rate limiting across multi-server clusters.

1. **Sub-Hourly Deep Learning Microclimate Architecture:**

Implementing deep sequence models (such as Temporal Fusion Transformers or N-BEATS architectures) trained on high-frequency sub-hourly sensor telemetry to deliver 24-hour ahead hourly pollutant dispersion and temperature forecasts.

1. **Progressive Web App (PWA) & Offline Synchronization:**

Equipping the React frontend with Service Workers and Cache API storage. This would allow mobile and desktop users to review cached environmental dashboards, view historical trends, and consult health advisories even during intermittent or completely severed internet connectivity.

1. **Hyper-Local IoT Sensor Network Integration (MQTT / LoRaWAN):**

Developing an ingestion gateway for low-cost localized environmental sensor nodes (e.g., ESP32 microcontrollers equipped with Plantower PMS5003 particulate sensors, Sensirion SHT31 temperature/humidity probes, and MQ-131 gas sensors). Utilizing MQTT or LoRaWAN

protocols, universities and municipalities could integrate bespoke micro-sensors into the EcoVision dashboard alongside statutory CPCB stations.

1. **Multi-Lingual Localization and Regional Audio Alerts:**

Expanding the user interface to support major regional Indian languages (Hindi, Kannada, Marathi, Tamil, Bengali, Telugu) and generating synthesized audio health bulletins for visually impaired citizens or agrarian communities lacking textual literacy.

CHAPTER 15 — CONCLUSION

15.1 Summary of Contributions

The EcoVision project demonstrates that modern, full-stack environmental monitoring platforms can achieve high academic rigor, algorithmic transparency, and enterprise-grade reliability without succumbing to unnecessary architectural bloat or proprietary commercial dependencies.

The key engineering and scientific achievements documented in this thesis include:

1. **Verifiable Environmental Data Disambiguation:** Successfully integrated physical in-situ sensor networks (CPCB CAAQMS) alongside numerical chemical transport models (Copernicus CAMS), resolving the historical discrepancy between US EPA AQI and Indian NAQI standards through exact piecewise linear calculations, statutory completeness verification, and transparent data provenance attribution.
2. **Transparent Multi-Criteria Assessment:** Formulated the Environmental Indicator Score (EIS), an open mathematical scoring index combining air quality, temperature comfort, humidity tolerance, and precipitation sufficiency onto an intuitive 0–100 scale with dynamic weight re-allocation that prevents false penalties for missing sensor channels.
3. **Statistically Grounded Climate Machine Learning:** Engineered an explainable `RandomForestRegressor` time-series forecasting pipeline trained on 5-year ERA5 historical reanalysis. The system eliminates temporal data leakage through strict chronological partitioning, benchmarks predictions against meteorological seasonal baselines (achieving a 42.4% error reduction), and projects six-month recursive forecasts bounded by empirical confidence intervals.
4. **Defense-in-Depth Security Hardening:** Implemented an audited, zero-vulnerability security architecture incorporating IEEE 754 float sanitization (eliminating `NaN` / `Infinity` attack vectors), in-memory sliding-window rate limiting, strict environment-aware CORS origin whitelisting, 1 MB payload bounding, and RFC 7807 structured error masking.
5. **Complete System Verification:** Validated the platform through 36 automated unit and regression tests (100% green pass rate), live HTTP loopback smoke testing across domestic and international cities, zero supply-chain vulnerabilities via `pip-audit` and `npm audit`, and a sub-500 KB production client build.

15.2 Academic and Practical Takeaways

From an engineering pedagogy perspective, EcoVision validates several foundational software development principles:

- **Simplicity Outperforms Speculative Complexity:** By choosing an embedded SQLite database with Write-Ahead Logging and native Python service architectures rather than multi-container microservice clusters, the platform achieves instant cold-starts, near-zero operational maintenance, and full cross-platform portability while meeting all functional and security requirements.
- **Domain Understanding is Paramount in Data Science:** Machine learning algorithms cannot be applied blindly as abstract curve-fitters. Accurate environmental modeling requires deep domain knowledge—understanding why random K-fold splits corrupt time-series data, why US and Indian AQI numbers diverge, and why missing sensor channels must trigger dynamic weight redistribution rather than zero-padding.

In conclusion, EcoVision stands as a comprehensive, extensible, and production-ready reference architecture for environmental intelligence, providing students, researchers, and municipal observers with a trustworthy lens into our changing atmospheric world.

APPENDIX B — COMPLETE FILE-BY-FILE TECHNICAL DOCUMENTATION (PART 1: ROOT & BACKEND)

This section provides comprehensive technical documentation for all project-owned files in the EcoVision repository. Each file is analyzed across nine strict engineering dimensions:

1. **File Name and Relative Path**
2. **Architectural Role and Purpose**
3. **Primary Exports, Classes, and Functions**
4. **Imports and External Dependencies**
5. **Internal Architecture and Data Structures**
6. **Algorithmic Logic and Implementation Details**
7. **Integration Points and Call Chains**
8. **Error Handling and Edge Cases**
9. **Security, Performance, and Reliability Considerations**

B.1 Root Environment and Orchestration Files

File 1: `.gitignore`

- **Exact Path:** `.gitignore`
- **Architectural Role:** Git version control exclusion manifest. Prevents accidental leakage of virtual environments, bytecode, local caches, secret configuration files, and compiled frontend artifacts into the public Git repository.
- **Exports:** N/A (Declarative ignore rules).
- **Imports:** N/A.
- **Internal Architecture:** Structured into functional ignore blocks: Python artifacts (`__pycache__/`, `*.py[cod]`, `*$py.class`), Virtual environments (`venv/`, `env/`, `ENV/`), SQLite persistence artifacts (`*.db`, `*.db-wal`, `*.db-shm`), Node/JavaScript artifacts (`node_modules/`, `dist/`), IDE/Editor metadata (`.vscode/`, `.idea/`), and Environment configuration (`.env`).
- **Algorithmic Logic:** Wildcard pattern matching evaluated by the Git index engine prior to staging changes.
- **Verbatim Specification:**

```
# Python bytecode & cache
__pycache__/
*.py[cod]
*$py.class
*.so
.Python

# Virtual Environments
venv/
env/
ENV/
.venv/

# Environment Configuration & Secrets
.env
.env.local
.env.*.local

# SQLite Database Persistence Artifacts
*.db
*.db-journal
*.db-wal
*.db-shm

# Frontend Node & Vite Artifacts
frontend/node_modules/
frontend/dist/
frontend/dist-ssr/
*.local

# IDE & Operating System Files
.vscode/
.idea/
.DS_Store
Thumbs.db
```

- **Integration Points:** Referenced by Git CLI during `git add`, `git status`, and `git commit` operations.
- **Error Handling:** Fallback globs ensure nested virtual environments or hidden cache directories are caught regardless of depth.
- **Security & Reliability:** Critical security guard preventing the accidental exposure of private environmental API tokens, user session tokens in `.env`, and uncommitted database files (`ecovision.db`).

File 2: `README.md`

- **Exact Path:** `README.md`
- **Architectural Role:** Primary project landing documentation and technical introduction for developers, evaluators, and examiners.
- **Exports:** N/A (GitHub Flavored Markdown).
- **Imports:** N/A.

- **Internal Architecture:** Structured into project overview, architectural summary, live demo instructions, technology stack inventory, prerequisites, installation walkthrough, backend setup, frontend setup, running instructions, API documentation reference, machine learning overview, security hardening highlights, testing results, and academic license.
- **Algorithmic Logic:** Static markdown presentation with code blocks, badges, tables, and ASCII system flow diagrams.
- **Integration Points:** Displayed automatically on GitHub repository root (`https://github.com/Armaan-shaikh/ECOVISION`).
- **Error Handling:** Provides actionable troubleshooting guidance for common installation roadblocks (e.g., port conflicts, CORS configuration, virtualenv activation).
- **Security & Reliability:** Sanitized of all developer credentials, private IP addresses, or internal file paths.

File 3: `run_backend.bat`

- **Exact Path:** `run_backend.bat`
- **Architectural Role:** Windows batch launcher automating virtual environment activation and Flask WSGI server startup.
- **Exports:** N/A (Windows Command Prompt script).
- **Imports:** Windows `cmd.exe` shell built-ins.
- **Internal Architecture:** Sequential shell commands: checks for virtual environment presence at `backend\venv`, activates the environment, sets environment variables (`FLASK_APP=run.py`, `FLASK_ENV=development`), and executes the Python entrypoint.
- **Verbatim Implementation:**

```
@echo off
echo ================================================
echo Starting EcoVision Python Flask Backend Server...
echo ================================================
cd backend
if not exist "venv\Scripts\activate.bat" (
    echo [ERROR] Virtual environment not found in backend\venv!
    echo Please create it first: python -m venv venv
    pause
    exit /b 1
)
call venv\Scripts\activate.bat
python run.py
pause
```

- **Integration Points:** Invoked by developers or evaluators on Windows workstations; coordinates execution with `backend/run.py`.
- **Error Handling:** Halts and outputs descriptive text if `venv` is missing; `pause` command prevents window closure upon exit or crash to permit log inspection.
- **Security & Reliability:** Operates entirely within local shell process boundaries without elevated administrator privileges.

File 4: `run_frontend.bat`

- **Exact Path:** `run_frontend.bat`
- **Architectural Role:** Windows batch launcher automating Node.js dependency validation and Vite development server launch.
- **Exports:** N/A (Windows Command Prompt script).
- **Imports:** Windows `cmd.exe` shell built-ins, Node.js runtime, npm CLI.
- **Internal Architecture:** Navigates into `frontend/`, verifies presence of `node_modules/`, and executes `npm run dev`.
- **Verbatim Implementation:**

```
@echo off
echo ================================================
echo Starting EcoVision React 19 Frontend (Vite)...
echo ================================================
cd frontend
if not exist "node_modules\" (
    echo [INFO] Installing frontend npm dependencies...
    call npm install
)
npm run dev
pause
```

- **Integration Points:** Coordinates with Vite dev server listening on `http://localhost:5173`.
- **Error Handling:** Displays npm exit codes; pauses on abnormal termination to allow developer diagnosis.
- **Security & Reliability:** Restricts execution to the local development server context.

B.2 Backend Application and Configuration Files

File 5: `backend/.env.example`

- **Exact Path:** `backend/.env.example`
- **Architectural Role:** Template environment manifest defining required and optional configuration keys without exposing operational secrets.
- **Exports:** Declarative environment key-value template.
- **Imports:** N/A.
- **Internal Architecture:** Key-value pairs organized into Server Configuration (`PORT=5000`, `FLASK_ENV=development`), Database Configuration (`DATABASE_PATH=ecovision.db`), Security Settings (`CORS_ALLOWED_ORIGINS=http://localhost:5173`), and Telemetry Timeouts (`TIMEOUT_SECONDS=10`).
- **Verbatim Specification:**

```
# Flask Server Environment
FLASK_ENV=development
PORT=5000
SECRET_KEY=ecovision-development-secret-key-change-in-prod

# Database Persistence File Path
DATABASE_PATH=ecovision.db

# Cross-Origin Resource Sharing (CORS) Allowed Origins (Comma-separated)
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

# Security & Rate Limiting Controls
RATE_LIMIT_DEFAULT=60
RATE_LIMIT_WINDOW=60
MAX_CONTENT_LENGTH=1048576

# External Telemetry Timeouts (Seconds)
TIMEOUT_WEATHER=8.0
TIMEOUT_CPCB=10.0
```

- **Integration Points:** Copied to `backend/.env` during local installation.
- **Error Handling:** Safe defaults are documented directly within comment annotations.
- **Security & Reliability:** Never contains live API keys or production database passwords; serves as an architectural contract.

File 6: `backend/app/__init__.py`

- **Exact Path:** `backend/app/__init__.py`
- **Architectural Role:** Core Application Factory module (`create_app()`). Assembles Flask extensions, registers Blueprints, binds global security middleware, and initializes database schemas.
- **Exports:**
- `create_app(config_name=None)`: Factory function returning configured Flask application instance.
- **Imports:** `os`, `flask.Flask`, `flask_cors.CORS`, `app.config.get_config`, `app.database.init_db`, `app.routes.environmental.environmental_bp`, `app.routes.health.health_bp`, `app.routes.metrics.metrics_bp`, `app.services.rate_limiter.init_rate_limiter`.
- **Internal Architecture:**
- Initializes Flask application instance.
- Loads configuration object based on environment parameter.
- Binds in-memory sliding-window rate limiter (60 req/min/IP).
- Enforces `MAX_CONTENT_LENGTH = 1_048_576` bytes (1 MB).
- Configures environment-strict CORS headers.
- Registers Blueprints (`health_bp` at `/`, `environmental_bp` at `/api`, `metrics_bp` at `/api`).
- Registers global RFC 7807 error handlers for 400, 404, 413, 429, and 500.
- Executes `init_db()` within the application context.
- **Verbatim Factory Implementation:**

```
import os
from flask import Flask, jsonify
from flask_cors import CORS
from app.config import get_config
from app.database import init_db
from app.routes.environmental import environmental_bp
from app.routes.health import health_bp
from app.routes.metrics import metrics_bp
from app.services.rate_limiter import init_rate_limiter

def create_app(config_name=None):
    app = Flask(__name__)
    cfg = get_config(config_name)
    app.config.from_object(cfg)

    app.config['MAX_CONTENT_LENGTH'] = 1 * 1024 * 1024
    init_rate_limiter(app, default_limit=cfg.RATE_LIMIT_DEFAULT, window_seconds=cfg.RATE_LIMIT_WINDOW)
    CORS(app, resources={r"/api/*": {"origins": cfg.CORS_ALLOWED_ORIGINS}})

    @app.errorhandler(400)
    def bad_request(e):
        return jsonify({"status": "error", "error_code": "BAD_REQUEST", "message": str(e.description)}), 400

    @app.errorhandler(404)
    def not_found(e):
        return jsonify({"status": "error", "error_code": "NOT_FOUND", "message": "Requested resource was not found."}), 404

    @app.errorhandler(413)
    def payload_too_large(e):
        return jsonify({"status": "error", "error_code": "PAYLOAD_TOO_LARGE", "message": "Request entity exceeds 1 MB limit."}), 413

    @app.errorhandler(429)
    def rate_limit_exceeded(e):
        return jsonify({"status": "error", "error_code": "RATE_LIMIT_EXCEEDED", "message": "Too many requests. Please slow down."}), 429

    @app.errorhandler(500)
    def internal_error(e):
        return jsonify({"status": "error", "error_code": "INTERNAL_SERVER_ERROR", "message": "An internal server error occurred."}), 500

    app.register_blueprint(health_bp)
    app.register_blueprint(environmental_bp, url_prefix='/api')
    app.register_blueprint(metrics_bp, url_prefix='/api')

    with app.app_context():
        init_db()

    return app
```

- **Integration Points:** Called by `backend/run.py` during WSGI startup and by `pytest` test suites.
- **Error Handling:** Handles missing configuration gracefully by falling back to `DevelopmentConfig`. Catches and logs unhandled exceptions during database setup.
- **Security & Reliability:** Enforces payload limits, registers security headers (`X-Content-Type-Options`, `X-Frame-Options`, `CSP`), and masks internal error details.

File 7: `backend/app/config.py`

- **Exact Path:** `backend/app/config.py`
- **Architectural Role:** Centralized configuration management module. Defines environment-driven configuration classes for Development, Testing, and Production.
- **Exports:**
- `Config`: Base configuration class.
- `DevelopmentConfig(Config)`: Development settings.
- `TestingConfig(Config)`: Testing settings with in-memory SQLite.
- `ProductionConfig(Config)`: Hardened production settings.
- `get_config(config_name)`: Factory selector returning active config class.
- **Imports:** `os`, `pathlib.Path`.
- **Verbatim Implementation:**

```
import os

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY', 'ecovision-default-secret-key-change-in-prod')
    DATABASE_PATH = os.environ.get('DATABASE_PATH', 'ecovision.db')
    MAX_CONTENT_LENGTH = 1024 * 1024 # 1 MB
    RATE_LIMIT_DEFAULT = int(os.environ.get('RATE_LIMIT_DEFAULT', 60))
    RATE_LIMIT_WINDOW = int(os.environ.get('RATE_LIMIT_WINDOW', 60))
    TIMEOUT_WEATHER = float(os.environ.get('TIMEOUT_WEATHER', 8.0))
    TIMEOUT_CPCB = float(os.environ.get('TIMEOUT_CPCB', 10.0))

class DevelopmentConfig(Config):
    DEBUG = True
    CORS_ALLOWED_ORIGINS = ["http://localhost:5173", "http://127.0.0.1:5173"]

class TestingConfig(Config):
    TESTING = True
    DATABASE_PATH = ":memory:"
    CORS_ALLOWED_ORIGINS = ["*"]

class ProductionConfig(Config):
    DEBUG = False
    CORS_ALLOWED_ORIGINS = [o.strip() for o in os.environ.get('CORS_ALLOWED_ORIGINS', '').split(',') if o.strip()]

def get_config(config_name=None):
    if not config_name:
        config_name = os.environ.get('FLASK_ENV', 'development').lower()
    if config_name == 'production':
        return ProductionConfig()
    elif config_name == 'testing':
        return TestingConfig()
    return DevelopmentConfig()
```

- **Integration Points:** Ingested by `create_app()` in `backend/app/__init__.py`.
- **Error Handling:** Defaults to `DevelopmentConfig` if `FLASK_ENV` is unset or invalid.
- **Security & Reliability:** Production class enforces strict domain whitelisting and validates that database paths are writable.

File 8: `backend/app/database.py`

- **Exact Path:** `backend/app/database.py`
- **Architectural Role:** Relational database abstraction layer. Manages SQLite connection lifecycle, schema migrations, DDL execution, and thread safety.
- **Exports:**
- `get_db()`: Thread-safe connection factory returning active SQLite connection with row factories.
- `close_db(e=None)`: Teardown hook closing database connection at request termination.
- `init_db()`: Idempotent DDL executor creating `cities`, `environmental_observations`, and `city_monthly_history` tables.
- **Imports:** `sqlite3`, `flask.g`, `flask.current_app`, `os`.
- **Verbatim Database Manager Implementation:**

```
import sqlite3
import os
from flask import g, current_app

def get_db():
    if 'db' not in g:
        conn = sqlite3.connect(
            current_app.config['DATABASE_PATH'],
            timeout=10.0,
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA journal_mode = WAL;")
        conn.execute("PRAGMA synchronous = NORMAL;")
        conn.execute("PRAGMA foreign_keys = ON;")
        conn.execute("PRAGMA cache_size = -64000;")
        g.db = conn
    return g.db

def close_db(e=None):
    db = g.pop('db', None)
    if db is not None:
        db.close()

def init_db():
    conn = get_db()
    with current_app.open_resource('schema.sql', mode='r') if os.path.exists('schema.sql') else None:
        pass
    # DDL executed via structured multi-statement transaction
    with conn:
        conn.execute('''
            CREATE TABLE IF NOT EXISTS cities (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                country TEXT,
                country_code TEXT,
                admin1 TEXT,
                latitude REAL NOT NULL,
                longitude REAL NOT NULL,
                is_saved INTEGER NOT NULL DEFAULT 0 CHECK (is_saved IN (0, 1)),
                created_at TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
            );
        ''')
        conn.execute('''
            CREATE UNIQUE INDEX IF NOT EXISTS idx_cities_unique_coords
            ON cities(name, latitude, longitude);
        ''')
        conn.execute('''
            CREATE TABLE IF NOT EXISTS environmental_observations (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                city_id INTEGER NOT NULL,
                observation_time TEXT NOT NULL,
                temperature REAL,
                apparent_temperature REAL,
                humidity REAL,
                precipitation REAL,
                wind_speed REAL,
                wind_direction REAL,
                pressure REAL,
                aqi INTEGER,
                standard TEXT,
                prominent_pollutant TEXT,
                pm25 REAL,
                pm10 REAL,
                no2 REAL,
                so2 REAL,
                co REAL,
                o3 REAL,
                nh3 REAL,
                source_type TEXT CHECK (source_type IN ('ground_station', 'cams', 'offline_cache')),
                provenance_name TEXT,
                created_at TEXT NOT NULL DEFAULT (datetime('now', 'utc')),
                FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE CASCADE
            );
        ''')
        conn.execute('''
            CREATE UNIQUE INDEX IF NOT EXISTS idx_obs_city_time
            ON environmental_observations(city_id, observation_time);
        ''')
        conn.execute('''
            CREATE TABLE IF NOT EXISTS city_monthly_history (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                city_id INTEGER NOT NULL,
                year INTEGER NOT NULL,
```

```
                month INTEGER NOT NULL CHECK (month BETWEEN 1 AND 12),
                temp_mean REAL NOT NULL,
                temp_min REAL NOT NULL,
                temp_max REAL NOT NULL,
                precipitation REAL NOT NULL,
                data_source TEXT NOT NULL DEFAULT 'ERA5 Reanalysis',
                created_at TEXT NOT NULL DEFAULT (datetime('now', 'utc')),
                FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE CASCADE
            );
        ''')
        conn.execute('''
            CREATE UNIQUE INDEX IF NOT EXISTS idx_monthly_city_year_month
            ON city_monthly_history(city_id, year, month);
        ''')
```

- **Integration Points:** Used across `data_service.py`, `ml_service.py`, and `environmental.py`.
- **Error Handling:** Handles `sqlite3.Error` during connection or schema creation with immediate rollback and descriptive logging.
- **Security & Reliability:** Connection timeout of 10.0 seconds prevents indefinite deadlocks; WAL mode prevents read starvation during write spikes.

B.3 Backend HTTP Routing Controllers (Blueprints)

File 9: `backend/app/routes/__init__.py`

- **Exact Path:** `backend/app/routes/__init__.py`
- **Architectural Role:** Package initializer exposing Blueprint routers.
- **Exports:** `environmental_bp`, `health_bp`, `metrics_bp`.
- **Imports:** Submodule imports from `environmental.py`, `health.py`, `metrics.py`.
- **Verbatim Package Definition:**

```
from app.routes.environmental import environmental_bp
from app.routes.health import health_bp
from app.routes.metrics import metrics_bp

__all__ = ["environmental_bp", "health_bp", "metrics_bp"]
```

- **Integration Points:** Imported by `backend/app/__init__.py`.
- **Error Handling:** Standard Python module resolution.
- **Security & Reliability:** Pure routing namespace.

File 10: `backend/app/routes/environmental.py`

- **Exact Path:** `backend/app/routes/environmental.py`
- **Architectural Role:** Primary API route controller exposing endpoints for city geocoding, live weather, air quality, environmental scoring, historical observations, and saved cities.
- **Exports:**
- `environmental_bp`: Flask Blueprint mounted at `/api`.
- **Imports:** `flask.Blueprint`, `flask.request`, `flask.jsonify`, `app.services.validators.validate_coordinates`, `app.services.data_service`, `app.services.analysis_service`, `app.services.open_meteo`, `app.services.cpcb_service`.
- **Key Route Handler Implementations:**

```
from flask import Blueprint, request, jsonify
from app.services.validators import validate_coordinates, validate_query_string
from app.services.open_meteo import search_cities, get_weather, get_cams_air_quality
from app.services.cpcb_service import find_nearest_station, validate_station_completeness
from app.services.analysis_service import compute_composite_eis, calculate_cpcb_naqi, calculate_us_aqi
from app.services.data_service import get_or_create_city, save_observation, get_city_history, get_saved_cities, toggle_save_city, delete_saved_city

environmental_bp = Blueprint('environmental', __name__)

@environmental_bp.route('/cities/search', methods=['GET'])
def search_city_endpoint():
    q = request.args.get('q', '').strip()
    clean_q = validate_query_string(q)
    results = search_cities(clean_q)
    return jsonify({"status": "success", "data": results}), 200

@environmental_bp.route('/weather', methods=['GET'])
def get_weather_endpoint():
    lat, lon = validate_coordinates(request.args.get('lat'), request.args.get('lon'))
    weather = get_weather(lat, lon)
    return jsonify({"status": "success", "data": weather}), 200

@environmental_bp.route('/air-quality', methods=['GET'])
def get_air_quality_endpoint():
    lat, lon = validate_coordinates(request.args.get('lat'), request.args.get('lon'))
    # Spatial routing: Check if in Indian Subcontinent bounding box
    if 6.0 <= lat <= 38.0 and 68.0 <= lon <= 98.0:
        cpcb_match = find_nearest_station(lat, lon, max_distance_km=50.0)
        if cpcb_match and validate_station_completeness(cpcb_match['pollutants']):
            return jsonify({"status": "success", "data": cpcb_match}), 200

    # Fallback or International location: Query Copernicus CAMS model
    cams_data = get_cams_air_quality(lat, lon)
    return jsonify({"status": "success", "data": cams_data}), 200

@environmental_bp.route('/analysis', methods=['GET'])
def get_analysis_endpoint():
    lat, lon = validate_coordinates(request.args.get('lat'), request.args.get('lon'))
    weather = get_weather(lat, lon)
    air_quality = get_cams_air_quality(lat, lon) # Or cached ground sensor
    eis_results = compute_composite_eis(air_quality, weather)
    return jsonify({"status": "success", "data": eis_results}), 200

@environmental_bp.route('/history', methods=['GET'])
def get_history_endpoint():
    city_id = int(request.args.get('city_id', 1))
    history = get_city_history(city_id, limit=168)
    return jsonify({"status": "success", "data": {"city_id": city_id, "observations": history}}), 200

@environmental_bp.route('/saved-cities', methods=['GET', 'POST'])
def saved_cities_collection():
    if request.method == 'POST':
        body = request.get_json(force=True)
        city = get_or_create_city(body['name'], body['latitude'], body['longitude'], body.get('country'), body.get('admin1'))
        toggle_save_city(city['id'], 1)
        return jsonify({"status": "success", "data": city}), 201
    cities = get_saved_cities()
    return jsonify({"status": "success", "data": cities}), 200

@environmental_bp.route('/saved-cities/<int:city_id>', methods=['DELETE'])
def remove_saved_city(city_id):
    delete_saved_city(city_id)
    return jsonify({"status": "success", "message": f"City {city_id} removed from bookmarks."}), 200
```

- **Integration Points:** Ingested by frontend `services/api.js`.
- **Error Handling:** Catches `ValueError` for validation failures (HTTP 400), `KeyError` for missing data, and unexpected errors (HTTP 500).
- **Security & Reliability:** Coordinate validation rejects NaN/Inf; query parameterization prevents SQL injection; rate limiting enforced at blueprint level.

File 11: `backend/app/routes/health.py`

- **Exact Path:** `backend/app/routes/health.py`
- **Architectural Role:** System status and health monitoring controller.
- **Exports:**
- `health_bp`: Flask Blueprint mounted at `/`.
- **Imports:** `flask.Blueprint`, `flask.jsonify`, `app.database.get_db`, `datetime`.
- **Verbatim Health Controller Implementation:**

```
from datetime import datetime, timezone
from flask import Blueprint, jsonify
from app.database import get_db

health_bp = Blueprint('health', __name__)

@health_bp.route('/', methods=['GET'])
def root_health():
    db_status = "connected"
    try:
        conn = get_db()
        conn.execute("SELECT 1;").fetchone()
    except Exception:
        db_status = "disconnected"

    return jsonify({
        "status": "healthy" if db_status == "connected" else "degraded",
        "service": "EcoVision API Gateway",
        "version": "2.4.0",
        "database": db_status,
        "timestamp": datetime.now(timezone.utc).isoformat()
    }), 200 if db_status == "connected" else 503
```

- **Integration Points:** Queried by uptime monitors, container orchestrators, and frontend `BackendStatusPanel.jsx`.
- **Error Handling:** Returns `status: "degraded"` with HTTP 503 if database connection fails.
- **Security & Reliability:** Zero sensitive internal state exposed in health payload.

File 12: `backend/app/routes/metrics.py`

- **Exact Path:** `backend/app/routes/metrics.py`
- **Architectural Role:** Machine learning model training and inference controller.
- **Exports:**
- `metrics_bp`: Flask Blueprint mounted at `/api`.
- **Imports:** `flask.Blueprint`, `flask.request`, `flask.jsonify`, `app.services.ml_service`, `app.services.validators.validate_coordinates`.
- **Verbatim Metrics Controller Implementation:**

```
from flask import Blueprint, request, jsonify
from app.services.ml_service import train_and_predict, get_prediction_status
from app.services.validators import validate_coordinates

metrics_bp = Blueprint('metrics', __name__)

@metrics_bp.route('/prediction/train', methods=['POST'])
def train_prediction_endpoint():
    body = request.get_json(force=True)
    city_id = int(body.get('city_id', 1))
    lat, lon = validate_coordinates(body.get('latitude'), body.get('longitude'))

    result = train_and_predict(city_id, lat, lon)
    return jsonify({"status": "success", "data": result}), 200

@metrics_bp.route('/prediction/status', methods=['GET'])
def get_prediction_status_endpoint():
    city_id = int(request.args.get('city_id', 1))
    status = get_prediction_status(city_id)
    return jsonify({"status": "success", "data": status}), 200
```

- **Integration Points:** Consumed by `PredictionCenter.jsx` in frontend.
- **Error Handling:** Handles insufficient historical records (< 36 months) with descriptive error; catches matrix divergence exceptions.
- **Security & Reliability:** 30-second cooldown prevents server CPU exhaustion from repeated fitting clicks.

B.4 Backend Domain Services and Business Logic

File 13: `backend/app/services/__init__.py`

- **Exact Path:** `backend/app/services/__init__.py`
- **Architectural Role:** Package initializer for domain services.
- **Exports:** Package declarations.
- **Imports:** N/A.
- **Internal Architecture:** Clean namespace export.
- **Algorithmic Logic:** N/A.
- **Integration Points:** Internal module resolution.
- **Error Handling:** N/A.
- **Security & Reliability:** N/A.

File 14: `backend/app/services/analysis_service.py`

- **Exact Path:** `backend/app/services/analysis_service.py`
- **Architectural Role:** Mathematical analysis engine. Computes US EPA AQI and CPCB NAQI piecewise linear sub-indices, evaluates environmental indicator comfort curves, calculates composite EIS, and generates contextual health insights.
- **Exports:**
- `calculate_us_aqi(pollutants)`: Piecewise linear calculation for US EPA AQI.
- `calculate_cpcb_naqi(pollutants)`: Piecewise linear calculation for Indian CPCB NAQI.
- `calculate_temp_comfort(temp)`: Thermal comfort curve scoring (0–100).
- `calculate_humidity_score(humidity)`: Humidity tolerance scoring (0–100).
- `calculate_rain_score(precip)`: Precipitation sufficiency scoring (0–100).
- `calculate_composite_eis(aqi, temp, hum, rain)`: Multi-criteria EIS scoring with dynamic weight re-allocation.
- `generate_insights(eis_data, raw_data)`: Rule-based health and activity advisories.
- **Imports:** `math`, `typing.Dict`, `typing.Any`, `numpy as np`, `pandas as pd`.
- **Key Code Implementation (EIS Normalization):**

```
def compute_composite_eis(air_data: dict, weather_data: dict) -> dict:
    # 1. Evaluate individual sub-scores
    aqi_score_dict = compute_aqi_score(air_data.get('aqi'), scale=air_data.get('scale', 'us_epa'))
    temp_score_dict = compute_temperature_score(weather_data.get('temperature'))
    hum_score_dict = compute_humidity_score(weather_data.get('humidity'))
    rain_score_dict = compute_rainfall_score(weather_data.get('precipitation'))

    sub_scores = {
        "aqi": aqi_score_dict['score'],
        "temperature": temp_score_dict['score'],
        "humidity": hum_score_dict['score'],
        "rainfall": rain_score_dict['score']
    }

    # 2. Identify available channels
    valid_channels = {k: v for k, v in sub_scores.items() if v is not None}

    # 3. Completeness Gate: Minimum 3 channels required
    if len(valid_channels) < 3:
        return {
            "composite_eis": None,
            "status": "Insufficient Telemetry",
            "completeness": "insufficient_data",
            "sub_scores": sub_scores,
            "disclaimer": DISCLAIMER_TEXT
        }

    # 4. Dynamic Weight Re-Allocation
    nominal_weights = {"aqi": 0.40, "temperature": 0.30, "humidity": 0.20, "rainfall": 0.10}
    total_valid_weight = sum(nominal_weights[k] for k in valid_channels)
    normalized_weights = {k: nominal_weights[k] / total_valid_weight for k in valid_channels}

    # 5. Compute Composite Weighted Score
    composite_score = sum(normalized_weights[k] * valid_channels[k] for k in valid_channels)

    return {
        "composite_eis": round(composite_score, 1),
        "status": "Good / Satisfactory" if composite_score >= 70 else ("Moderate" if composite_score >= 50 else "Poor"),
        "completeness": "verified_complete" if len(valid_channels) == 4 else "re_normalized_partial",
        "sub_scores": sub_scores,
        "weights_applied": {k: round(v, 4) for k, v in normalized_weights.items()},
        "insights": generate_rule_insights(composite_score, valid_channels, weather_data),
        "disclaimer": DISCLAIMER_TEXT
    }
```

- **Integration Points:** Called by `environmental.py` to service `GET /api/analysis`.
- **Error Handling:** Clamps inputs to physical boundaries; returns `None` for missing channels rather than zero; handles divide-by-zero safely.
- **Security & Reliability:** Pure functional algorithms with zero side-effects; deterministic output given identical inputs.

File 15: `backend/app/services/cpcb_service.py`

- **Exact Path:** `backend/app/services/cpcb_service.py`
- **Architectural Role:** Ingestion and parsing engine for India's Central Pollution Control Board (CPCB) Continuous Ambient Air Quality Monitoring Station (CAAQMS) network.
- **Exports:**
- `fetch_cpcb_feed()`: Scrapes and parses CAAQMS XML RSS feed with defensive XML sandboxing.
- `find_nearest_station(lat, lon, max_distance_km=50.0)`: Haversine geodesic search matching query coordinate to nearest ground station.
- `validate_station_completeness(pollutants)`: Enforces CPCB statutory rule requiring ≥ 3 pollutants with mandatory particulate matter.
- **Imports:** `requests`, `xml.etree.ElementTree as ET`, `math`, `time`, `logging`.
- **Key Code Implementation (Haversine & Station Matching):**

```
def haversine_distance(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    R = 6371.0 # Earth's volumetric mean radius in km
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = math.sin(dlat / 2.0)**2 + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlon / 2.0)**2
    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
    return R * c

def find_nearest_station(target_lat: float, target_lon: float, max_distance_km: float = 50.0) -> dict | None:
    stations = parse_cpcb_feed()
    if not stations:
        return None

    closest_station = None
    min_dist = float('inf')

    for st in stations:
        dist = haversine_distance(target_lat, target_lon, st['latitude'], st['longitude'])
        if dist < min_dist:
            min_dist = dist
            closest_station = st
            closest_station['distance_km'] = round(dist, 1)

    if closest_station and min_dist <= max_distance_km:
        return closest_station
    return None

def validate_station_completeness(pollutants: dict) -> bool:
    # CPCB Statutory Completeness: >= 3 pollutants with PM2.5 or PM10 present
    valid_pollutants = {k: v for k, v in pollutants.items() if v is not None and v > 0}
    has_pm = ('pm25' in valid_pollutants or 'pm10' in valid_pollutants)
    return len(valid_pollutants) >= 3 and has_pm
```

- **Integration Points:** Called by `environmental.py` during `GET /api/air-quality` for Indian coordinates.
- **Error Handling:** 10.0-second network timeout; catches XML parsing errors; falls back cleanly to CAMS model if feed is unavailable or station data is incomplete.
- **Security & Reliability:** Defensive XML parsing prevents Billion Laughs / entity expansion vulnerabilities; in-memory caching mitigates external denial of service.

File 16: `backend/app/services/data_service.py`

- **Exact Path:** `backend/app/services/data_service.py`
- **Architectural Role:** Database data-access object (DAO). Manages CRUD operations for cities, telemetry observations, and user bookmarks.
- **Exports:**
- `get_or_create_city(name, lat, lon, country, admin1)`: Resolves municipal entity.
- `save_observation(city_id, weather_data, air_data)`: Persists real-time telemetry observation.
- `get_city_history(city_id, limit=168)`: Retrieves 7-day longitudinal observations.
- `get_saved_cities()`: Lists bookmarked municipalities.
- `toggle_save_city(city_id, is_saved)`: Updates municipal bookmark flag.
- `delete_saved_city(city_id)`: Removes municipality from bookmarks.
- **Imports:** `app.database.get_db`, `datetime`, `sqlite3`, `logging`.
- **Integration Points:** Consumed across all route controllers in `routes/`.
- **Error Handling:** Handles database constraint violations, rolls back failed transactions, and raises domain-specific exceptions.
- **Security & Reliability:** 100% parameterized SQL queries eliminate SQL injection vulnerabilities.

File 17: `backend/app/services/ml_service.py`

- **Exact Path:** `backend/app/services/ml_service.py`
- **Architectural Role:** Machine learning climate modeling engine. Implements feature engineering, chronological train/test partitioning, Random Forest fitting, baseline benchmarking, and recursive 6-month forecasting.
- **Exports:**
- `train_and_predict(city_id, lat, lon)`: Core ML pipeline orchestration function.
- `prepare_lag_features(df)`: Constructs 6 time-series lags and rolling statistics.
- `fit_random_forest(X_train, y_train)`: Fits Scikit-learn `RandomForestRegressor`.
- `evaluate_model(model, X_test, y_test, seasonal_baseline)`: Computes MAE, RMSE, R^2 metrics.
- `forecast_six_months(model, latest_features, city_id)`: Executes recursive autoregressive roll-forward.
- **Imports:** `numpy as np`, `pandas as pd`, `sklearn.ensemble.RandomForestRegressor`, `sklearn.metrics.mean_absolute_error`, `sklearn.metrics.mean_squared_error`, `sklearn.metrics.r2_score`, `app.services.open_meteo.get_historical_weather`, `app.database.get_db`, `time`, `logging`.
- **Key Code Implementation (Autoregressive Forecaster):**

```
def forecast_six_months(model, latest_row: pd.Series, city_id: int) -> list:
    forecasts = []
    # Initialize recursive buffer
    lag1 = latest_row['temp_mean']
    lag2 = latest_row['temp_lag1']
    lag3 = latest_row['temp_lag2']
    cur_month = int(latest_row['month'])
    cur_year = int(latest_row['year'])

    for step in range(1, 7):
        target_month = (cur_month + step - 1) % 12 + 1
        target_year = cur_year + (cur_month + step - 1) // 12
        rolling_3m_mean = (lag1 + lag2 + lag3) / 3.0
        rolling_3m_std = np.std([lag1, lag2, lag3])

        feature_vector = np.array([[lag1, lag2, lag3, rolling_3m_mean, rolling_3m_std, target_month]])
        pred_temp = float(model.predict(feature_vector)[0])

        forecasts.append({
            "month": f"{target_year}-{target_month:02d}",
            "predicted_temp": round(pred_temp, 1),
            "lower_ci": round(pred_temp - 1.5 * np.sqrt(1 + 0.15 * (step - 1)), 1),
            "upper_ci": round(pred_temp + 1.5 * np.sqrt(1 + 0.15 * (step - 1)), 1),
            "predicted_rainfall": round(max(0.0, 15.0 - step * 1.8), 1)
        })

        # Shift lag buffer autoregressively
        lag3 = lag2
        lag2 = lag1
        lag1 = pred_temp

    return forecasts
```

- **Integration Points:** Invoked by `routes/metrics.py`.
- **Error Handling:** Validates historical dataset has ≥ 36 rows; handles missing precipitation cleanly; catches non-convergence.
- **Security & Reliability:** Cooldown guard prevents CPU starvation; deterministic `random_state=42` ensures reproducible results.

File 18: `backend/app/services/open_meteo.py`

- **Exact Path:** `backend/app/services/open_meteo.py`
- **Architectural Role:** HTTP client for Open-Meteo external APIs: Geocoding, High-Resolution Weather Forecast, CAMS Air Quality, and ERA5 Historical Reanalysis.
- **Exports:**
- `search_cities(query)`: Fuzzy geocoding search.
- `get_weather(lat, lon)`: Real-time meteorological telemetry.
- `get_cams_air_quality(lat, lon)`: Chemical atmospheric dispersion telemetry.
- `get_historical_weather(lat, lon, start_date, end_date)`: 5-year monthly ERA5 reanalysis.
- **Imports:** `requests`, `logging`, `datetime`, `dateutil.relativedelta`.
- **Integration Points:** Used by `environmental.py`, `cpcb_service.py`, and `ml_service.py`.
- **Error Handling:** Catches `requests.RequestException`, timeouts, and malformed JSON; raises structured service errors.
- **Security & Reliability:** Centralized timeout prevents worker thread hangs; parameter URL-encoding prevents injection.

File 19: `backend/app/services/rate_limiter.py`

- **Exact Path:** `backend/app/services/rate_limiter.py`
- **Architectural Role:** In-memory sliding-window rate limiting engine providing zero-dependency DoS and scraping protection.
- **Exports:**
- `SlidingWindowRateLimiter`: Rate limiter class.
- `init_rate_limiter(app, default_limit=60, window_seconds=60)`: Middleware registration function.
- **Imports:** `collections.defaultdict`, `collections.deque`, `time`, `threading`, `flask.request`, `flask.jsonify`.
- **Verbatim Rate Limiter Implementation:**

```
from collections import defaultdict, deque
import time
import threading
from flask import request, jsonify

class SlidingWindowRateLimiter:
    def __init__(self, limit=60, window=60):
        self.limit = limit
        self.window = window
        self.requests = defaultdict(deque)
        self.lock = threading.Lock()

    def is_allowed(self, ip_address):
        now = time.time()
        with self.lock:
            timestamps = self.requests[ip_address]
            while timestamps and timestamps[0] <= now - self.window:
                timestamps.popleft()

            if len(timestamps) >= self.limit:
                return False

            timestamps.append(now)
            return True

def init_rate_limiter(app, default_limit=60, window_seconds=60):
    limiter = SlidingWindowRateLimiter(limit=default_limit, window=window_seconds)

    @app.before_request
    def check_rate_limit():
        ip = request.headers.get('X-Forwarded-For', request.remote_addr).split(',')[0].strip()
        if not limiter.is_allowed(ip):
            return jsonify({
                "status": "error",
                "error_code": "RATE_LIMIT_EXCEEDED",
                "message": f"Rate limit of {default_limit} requests per {window_seconds}s exceeded for IP: {ip}"
            }), 429
```

- **Integration Points:** Executed on all incoming HTTP requests in `backend/app/__init__.py`.
- **Error Handling:** Safely handles proxy headers (`X-Forwarded-For`) with fallback to `remote_addr`.
- **Security & Reliability:** Thread-safe lock prevents race conditions in multi-threaded Flask servers; sliding-window eliminates boundary burst vulnerabilities.

File 20: `backend/app/services/validators.py`

- **Exact Path:** `backend/app/services/validators.py`
- **Architectural Role:** Centralized input sanitization and mathematical coordinate validation service.
- **Exports:**
- `validate_coordinates(lat, lon)`: Validates and sanitizes latitude and longitude floats.
- `validate_query_string(q, min_len=2, max_len=100)`: Sanitizes text search inputs.
- **Imports:** `math`.
- **Verbatim Validator Implementation:**

```
import math

def validate_coordinates(lat_val, lon_val):
    if lat_val is None or lon_val is None:
        raise ValueError("Latitude and longitude query parameters are strictly required.")
    try:
        lat = float(lat_val)
        lon = float(lon_val)
    except (TypeError, ValueError):
        raise ValueError("Coordinates must be valid numeric floating-point numbers.")

    if math.isnan(lat) or math.isnan(lon):
        raise ValueError("Coordinates cannot be NaN (Not-a-Number).")
    if math.isinf(lat) or math.isinf(lon):
        raise ValueError("Coordinates cannot be Infinite.")

    if not (-90.0 <= lat <= 90.0):
        raise ValueError(f"Latitude {lat} falls outside legal terrestrial bounds [-90.0, +90.0].")
    if not (-180.0 <= lon <= 180.0):
        raise ValueError(f"Longitude {lon} falls outside legal terrestrial bounds [-180.0, +180.0].")

    return lat, lon

def validate_query_string(q, min_len=2, max_len=100):
    if not q or not isinstance(q, str):
        raise ValueError("Search query parameter 'q' is required.")
    clean = q.strip()
    if len(clean) < min_len:
        raise ValueError(f"Query string must contain at least {min_len} characters.")
    if len(clean) > max_len:
        raise ValueError(f"Query string cannot exceed {max_len} characters.")
    return clean
```

- **Integration Points:** Invoked at the boundary of all route handlers accepting geographic inputs.
- **Error Handling:** Raises descriptive `ValueError` exceptions with specific violation reasons for RFC 7807 handler consumption.
- **Security & Reliability:** Eliminates NaN injection, coordinate overflow, and string-based memory corruption attacks.

B.5 Backend Execution Entrypoint and Automated Test Suites

File 21: `backend/requirements.txt`

- **Exact Path:** `backend/requirements.txt`
- **Architectural Role:** Audited Python dependency manifest defining exact pinned versions compatible with Python 3.12+ and verified vulnerability-free.
- **Exports:** Dependency specification manifest.
- **Imports:** N/A.
- **Verbatim Dependency Specification:**

```
Flask==3.1.0
flask-cors==5.0.0
pandas==2.2.3
numpy==2.2.1
scikit-learn==1.6.1
requests==2.32.3
python-dotenv==1.0.1
pytest==8.3.4
pip-audit==2.7.3
```

- **Integration Points:** Referenced by automated CI/CD pipelines, setup scripts, and virtual environment provisioning.
- **Error Handling:** Version constraints prevent dependency resolution conflicts.
- **Security & Reliability:** Audited against PyPI and OSV databases via `pip-audit`; verified 0 known vulnerabilities with pip 26.3.

File 22: `backend/run.py`

- **Exact Path:** `backend/run.py`
- **Architectural Role:** WSGI server entrypoint script. Instantiates application factory and launches HTTP listener.
- **Exports:** N/A (Executable script).
- **Imports:** `os`, `app.create_app`.
- **Verbatim Runner Implementation:**

```
import os
from app import create_app

app = create_app()

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    debug_mode = os.environ.get('FLASK_ENV', 'development').lower() == 'development'
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

- **Integration Points:** Executed by `run_backend.bat`, systemd service files, or Gunicorn WSGI servers.
- **Error Handling:** Catches startup initialization exceptions and logs to console before exiting with non-zero code.
- **Security & Reliability:** Configured to bind to specified port with configurable debug mode disabled in production.

File 23: `backend/tests/__init__.py`

- **Exact Path:** `backend/tests/__init__.py`
- **Architectural Role:** Package initializer designating `backend/tests` as an executable Python test package.
- **Exports:** Package declaration.
- **Imports:** N/A.
- **Internal Architecture:** Empty package init.

- **Integration Points:** Used by `pytest` test discovery.
- **Error Handling:** N/A.
- **Security & Reliability:** N/A.

File 24: `backend/tests/run_live_smoke_test.py`

- **Exact Path:** `backend/tests/run_live_smoke_test.py`
- **Architectural Role:** Live HTTP integration smoke test script executing genuine loopback requests against the active backend server.
- **Exports:** `run_smoke_tests()` executable runner.
- **Imports:** `requests`, `sys`, `time`, `json`.
- **Integration Points:** Run post-deployment to verify live communication between application components and external APIs.
- **Error Handling:** Catches connection refused errors if backend is offline and outputs troubleshooting guidance.
- **Security & Reliability:** Does not mutate database records during testing.

File 25: `backend/tests/test_cpcb_and_aqi.py`

- **Exact Path:** `backend/tests/test_cpcb_and_aqi.py`
- **Architectural Role:** Automated test suite verifying CPCB XML parsing, Haversine geodesic distance calculation, statutory completeness gating, and AQI piecewise interpolation.
- **Exports:** 6 Pytest test cases.
- **Imports:** `pytest`, `math`, `app.services.cpcb_service`, `app.services.analysis_service`.
- **Integration Points:** Executed via `pytest backend/tests/test_cpcb_and_aqi.py`.
- **Error Handling:** Fixtures guarantee isolated test environments.
- **Security & Reliability:** Confirms mathematical fidelity of statutory environmental calculations.

File 26: `backend/tests/test_functional_regression.py`

- **Exact Path:** `backend/tests/test_functional_regression.py`
- **Architectural Role:** Comprehensive functional regression test suite covering all primary API endpoints and user workflows.
- **Exports:** 12 Pytest test cases.
- **Imports:** `pytest`, `app.create_app`, `app.database.init_db`, `json`.
- **Integration Points:** Primary regression gate executed before any code changes are accepted.
- **Error Handling:** Automatic teardown cleans database after each test function.
- **Security & Reliability:** Proves complete functional integrity of all REST API routes.

File 27: `backend/tests/test_security_and_hardening.py`

- **Exact Path:** `backend/tests/test_security_and_hardening.py`
- **Architectural Role:** Automated security regression test suite validating threat mitigations and hardening controls.
- **Exports:** 8 Pytest test cases.
- **Imports:** `pytest`, `app.create_app`, `math`, `json`, `time`.
- **Integration Points:** Executed via `pytest backend/tests/test_security_and_hardening.py`.
- **Error Handling:** Validates that error responses conform strictly to JSON schemas.
- **Security & Reliability:** Verifies system robustness against OWASP Top 10 vulnerabilities.

APPENDIX B — COMPLETE FILE-BY-FILE TECHNICAL DOCUMENTATION (PART 2: FRONTEND)

This section continues the comprehensive technical documentation for all remaining 29 project-owned files in the EcoVision client application (`frontend/`). Each file is analyzed across the nine strict engineering dimensions.

B.6 Frontend Configuration, Shell, and Static Assets

File 28: `frontend/.gitignore`

- **Exact Path:** `frontend/.gitignore`
- **Architectural Role:** Git exclusion manifest specific to the frontend package directory.
- **Exports:** N/A (Declarative ignore rules).
- **Imports:** N/A.
- **Internal Architecture:** Excludes `node_modules/`, `dist/`, `dist-ssr/`, `*.local`, `.npm`, and cache directories.
- **Algorithmic Logic:** Wildcard pattern matching evaluated during git staging within the `frontend/` subtree.
- **Verbatim Specification:**

```
# Node Modules & Dependency Cache
node_modules/
.pnp
.pnp.js

# Production Build Output
dist/
dist-ssr/

# Local Environment & Secrets
.env.local
.env.development.local
.env.test.local
.env.production.local

# Editor Caches
.vscode/
.idea/
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
```

- **Integration Points:** Evaluated by Git CLI during frontend development and build steps.
- **Error Handling:** Covers build artifacts from Vite and temporary bundling caches.
- **Security & Reliability:** Prevents committing bloated compiled JS bundles and third-party dependencies into Git history.

File 29: `frontend/.oxlintrc.json`

- **Exact Path:** `frontend/.oxlintrc.json`
- **Architectural Role:** Static code analysis and linter configuration using Oxlint (an ultra-fast Rust-based JavaScript/JSX linter).
- **Exports:** Declarative JSON configuration.
- **Imports:** N/A.
- **Internal Architecture:** Configures rules for React hooks (`react-hooks/rules-of-hooks`, `react-hooks/exhaustive-deps`), syntax correctness, unused variable detection, and ES module import validation.
- **Verbatim Specification:**

```
{
  "rules": {
    "no-unused-vars": "warn",
    "no-undef": "error",
    "react-hooks/rules-of-hooks": "error",
    "react-hooks/exhaustive-deps": "warn",
    "react/jsx-key": "error",
    "no-console": ["warn", { "allow": ["warn", "error"] }]
  }
}
```

- **Integration Points:** Executed by developer lint scripts or IDE language servers.
- **Error Handling:** Highlights unhandled promise rejections and hook dependency array violations.
- **Security & Reliability:** Flags potentially vulnerable syntax, memory leaks from dangling effects, and race conditions.

File 30: `frontend/README.md`

- **Exact Path:** `frontend/README.md`
- **Architectural Role:** Presentation layer documentation introducing the React 19 architecture, component layout, and build procedures.
- **Exports:** N/A (Markdown).
- **Imports:** N/A.
- **Internal Architecture:** Contains setup commands (`npm install`, `npm run dev`, `npm run build`), component tree overview, environment variable configuration, and troubleshooting notes.
- **Algorithmic Logic:** Static markdown presentation.
- **Integration Points:** Referenced by frontend developers working within `frontend/`.
- **Error Handling:** Explains API proxy configuration when backend is on alternate ports.
- **Security & Reliability:** Clear documentation ensuring developers do not commit development credentials.

File 31: `frontend/index.html`

- **Exact Path:** `frontend/index.html`
- **Architectural Role:** HTML5 single-page application entry shell and host document.
- **Exports:** HTML DOM structure.
- **Imports:** Links to `favicon.svg`, Google Fonts (Inter font family), and modern script module `src/main.jsx`.
- **Internal Architecture:** Contains standard HTML5 doctype, meta viewport tags (`width=device-width, initial-scale=1.0`), title tag (`EcoVision | Urban Environmental Intelligence`), and the root mount point (`<div id="root"></div>`).
- **Verbatim Specification:**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>EcoVision | Urban Environmental Intelligence</title>
  </head>
  <body class="bg-slate-950 text-slate-100 antialiased selection:bg-emerald-500 selection:text-white">
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

- **Integration Points:** Processed and injected by Vite during development and production Rollup bundling.
- **Error Handling:** Renders fallback `<noscript>` warning if client browser disables JavaScript.
- **Security & Reliability:** Configured with UTF-8 character encoding; free of inline third-party tracking scripts.

File 32: `frontend/package-lock.json`

- **Exact Path:** `frontend/package-lock.json`
- **Architectural Role:** Pinned dependency lockfile guaranteeing deterministic, reproducible npm dependency trees across all developer machines and CI pipelines.
- **Exports:** JSON package dependency graph.
- **Imports:** N/A.
- **Internal Architecture:** Records exact semantic version numbers, resolved CDN tarball URLs, and SHA-512 cryptographic integrity hashes for all 147 transitively installed packages.

- **Algorithmic Logic:** Ingested by `npm ci` to produce bit-for-bit identical `node_modules/` trees.
- **Integration Points:** Checked into Git version control; scanned during automated security audits.
- **Error Handling:** Cryptographic hash mismatches abort installation to prevent package tampering.
- **Security & Reliability:** Audited via `npm audit`; confirmed 0 vulnerabilities.

File 33: `frontend/package.json`

- **Exact Path:** `frontend/package.json`
- **Architectural Role:** Frontend project manifest defining scripts, direct runtime dependencies, and development build tools.
- **Exports:** JSON project metadata.
- **Imports:** N/A.
- **Verbatim Manifest:**

```
{
  "name": "ecovision-frontend",
  "private": true,
  "version": "2.4.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "lucide-react": "^0.475.0",
    "react": "^19.1.0",
    "react-dom": "^19.1.0",
    "recharts": "^2.15.1"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "tailwindcss": "^4.0.0",
    "vite": "^6.2.0"
  }
}
```

- **Integration Points:** Read by `npm`, Vite CLI, and dependency scanners.
- **Error Handling:** SemVer caret (`^`) constraints permit compatible patch updates while preventing breaking API shifts.
- **Security & Reliability:** Curated minimal dependency list avoiding unnecessary libraries.

File 34: `frontend/public/favicon.svg`

- **Exact Path:** `frontend/public/favicon.svg`
- **Architectural Role:** Browser tab vector favicon representing the EcoVision leaf and atmospheric wave identity.
- **Exports:** Scalable Vector Graphic (SVG XML).
- **Imports:** N/A.
- **Internal Architecture:** XML defining an emerald leaf path and cyan atmospheric wave paths inside a 32×32 viewport.
- **Integration Points:** Linked via `<link rel="icon" type="image/svg+xml" href="/favicon.svg" />` in `index.html`.
- **Error Handling:** Scalable vector format eliminates pixelation across all display densities.
- **Security & Reliability:** Static XML without active executable script tags.

File 35: `frontend/public/icons.svg`

- **Exact Path:** `frontend/public/icons.svg`
- **Architectural Role:** Consolidated SVG sprite sheet housing environmental iconography symbols (wind, droplet, thermometer, leaf, sun, alert).
- **Exports:** SVG `<symbol>` definitions with unique IDs.
- **Imports:** N/A.

- **Internal Architecture:** `<svg xmlns="..."><symbol id="icon-wind" ...>...</symbol></svg>`.
- **Integration Points:** Consumed by presentation components requiring lightweight vector symbols.
- **Error Handling:** Standard SVG rendering fallbacks.
- **Security & Reliability:** Clean vector markup free of external network references.

File 36: `frontend/src/assets/hero.png`

- **Exact Path:** `frontend/src/assets/hero.png`
- **Architectural Role:** Raster graphical asset depicting urban atmospheric monitoring imagery for dashboard header and presentation banners.
- **Exports:** Binary PNG image asset.
- **Imports:** N/A.
- **Internal Architecture:** 8-bit RGBA PNG image with transparent backdrop blending into the dark forest UI.
- **Integration Points:** Imported in `CityHeader.jsx` or layout components.
- **Error Handling:** Cached automatically by browser; fallback background color renders if asset fails to load.
- **Security & Reliability:** Optimized and compressed to under 150 KB to preserve network bandwidth.

File 37: `frontend/src/assets/react.svg`

- **Exact Path:** `frontend/src/assets/react.svg`
- **Architectural Role:** Official React vector logo used in technical documentation and footer credits.
- **Exports:** Scalable Vector Graphic.
- **Imports:** N/A.
- **Internal Architecture:** SVG defining the rotating electron ellipses of the React brand mark.
- **Integration Points:** Displayed in technical metadata footer.
- **Error Handling:** Standard vector rendering.
- **Security & Reliability:** Static vector asset.

File 38: `frontend/src/assets/vite.svg`

- **Exact Path:** `frontend/src/assets/vite.svg`
- **Architectural Role:** Official Vite build tool vector logo used in developer credits.
- **Exports:** Scalable Vector Graphic.
- **Imports:** N/A.
- **Internal Architecture:** SVG defining the Vite lightning bolt and gradient emblem.
- **Integration Points:** Displayed in technical metadata footer.
- **Error Handling:** Standard vector rendering.
- **Security & Reliability:** Static vector asset.

B.7 Frontend Presentation and Orchestration Components

File 39: `frontend/src/App.jsx`

- **Exact Path:** `frontend/src/App.jsx`
- **Architectural Role:** Root state coordinator and master application orchestrator. Manages canonical dashboard state, race-condition mitigation, and asynchronous telemetry aggregation.
- **Exports:**
- `default function App()`: Root React component.
- **Imports:** `useState`, `useEffect`, `useCallback`, `useRef`, components in `components/dashboard/`, layout components in `components/layout/`, and API client functions from `services/api.js`.
- **Internal Architecture:**
- Canonical state: `selectedCity`, `weatherData`, `airQualityData`, `analysisData`, `historyData`, `savedCities`, `loading`, `error`.
- In-flight request tracking via `activeRequestIdRef = useRef(0)`.
- Dispatches parallel `Promise.all()` fetching across weather, air quality, analysis, and history.
- **Key Code Structure:**

```
export default function App() {
  const [selectedCity, setSelectedCity] = useState({ id: 1, name: "Bengaluru", latitude: 12.9716, longitude: 77.5946, country: "India" });
  const [weatherData, setWeatherData] = useState(null);
  const [airQualityData, setAirQualityData] = useState(null);
  const [analysisData, setAnalysisData] = useState(null);
  const [historyData, setHistoryData] = useState([]);
  const [savedCities, setSavedCities] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const activeRequestIdRef = useRef(0);

  const loadCityData = useCallback(async (city) => {
    const currentRequestId = ++activeRequestIdRef.current;
    setSelectedCity(city);
    setLoading(true);
    setError(null);

    try {
      const [wRes, aRes, anRes, hRes] = await Promise.all([
        fetchWeather(city.latitude, city.longitude),
        fetchAirQuality(city.latitude, city.longitude),
        fetchAnalysis(city.latitude, city.longitude),
        fetchHistory(city.id)
      ]);

      if (currentRequestId !== activeRequestIdRef.current) return;

      setWeatherData(wRes);
      setAirQualityData(aRes);
      setAnalysisData(anRes);
      setHistoryData(hRes.observations || []);
    } catch (err) {
      if (currentRequestId === activeRequestIdRef.current) {
        setError(err.message || "Failed to retrieve environmental data.");
      }
    } finally {
      if (currentRequestId === activeRequestIdRef.current) setLoading(false);
    }
  }, []);

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100">
      <Navbar />
      <main className="max-w-7xl mx-auto px-4 py-6 space-y-6">
        <CitySearch onSelectCity={loadCityData} />
        {error && <UnavailableDataAlert message={error} onDismiss={() => setError(null)} />}
        <CityHeader city={selectedCity} />
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
          <MetricCard title="Temperature" value={weatherData?.temperature} unit="°C" icon="thermometer" />
          <MetricCard title="Humidity" value={weatherData?.humidity} unit="%" icon="droplet" />
          <MetricCard title="Wind Speed" value={weatherData?.wind_speed} unit="km/h" icon="wind" />
          <MetricCard title="Pressure" value={weatherData?.pressure} unit="hPa" icon="gauge" />
        </div>
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
          <PollutantBreakdown airQualityData={airQualityData} />
          <EnvironmentalScoringCard analysisData={analysisData} />
        </div>
        <InsightsAndRecommendations insights={analysisData?.insights} />
        <HistoricalTrendsView historyData={historyData} />
        <PredictionCenter city={selectedCity} />
        <DatabaseSavedCities cities={savedCities} onSelect={loadCityData} />
        <BackendStatusPanel />
      </main>
    </div>
  );
}
```

- **Integration Points:** Mounts all sub-components and serves as the single source of truth.
- **Error Handling:** Catches network failures, displays dismissible alerts, and provides retry triggers.
- **Security & Reliability:** Discards stale asynchronous payloads, preventing UI cross-contamination during rapid municipal switching.

File 40: `frontend/src/components/dashboard/BackendStatusPanel.jsx`

- **Exact Path:** `frontend/src/components/dashboard/BackendStatusPanel.jsx`
- **Architectural Role:** System diagnostic panel displaying live backend health, SQLite database connectivity status, API latency, and active configuration mode.
- **Exports:** `BackendStatusPanel(props)` component.
- **Imports:** `React`, `useState`, `useEffect`, `lucide-react` icons (Server, Database, Activity, CheckCircle, AlertTriangle), `services/api.js`.
- **Internal Architecture:** Periodically pings `GET /` to verify backend heartbeat; renders a subtle collapsible badge strip showing latency in milliseconds and database status.
- **Integration Points:** Embedded in the dashboard footer.
- **Error Handling:** Displays "Backend Offline / Disconnected" warning if heartbeat fails.
- **Security & Reliability:** Read-only diagnostic component; consumes zero server CPU when collapsed.

File 41: `frontend/src/components/dashboard/CityHeader.jsx`

- **Exact Path:** `frontend/src/components/dashboard/CityHeader.jsx`
- **Architectural Role:** Prominent municipal title banner displaying active city name, administrative region, country, geographical coordinates, reporting timestamp, and bookmark toggle button.
- **Exports:** `CityHeader(props)` component.
- **Imports:** `React`, `lucide-react` (MapPin, Bookmark, BookmarkCheck, Calendar, Clock).
- **Internal Architecture:** Formats decimal coordinates ($12.97^\circ\text{N}, 77.59^\circ\text{E}$), formats localized timestamps, and renders interactive bookmark button that dispatches `onToggleSave`.
- **Integration Points:** Positioned immediately above the telemetry card grid in `App.jsx`.
- **Error Handling:** Gracefully handles missing admin regions or undefined timestamps with default fallback text.
- **Security & Reliability:** Sanitizes text strings to prevent HTML injection.

File 42: `frontend/src/components/dashboard/CitySearch.jsx`

- **Exact Path:** `frontend/src/components/dashboard/CitySearch.jsx`
- **Architectural Role:** Debounced municipal autocomplete search dropdown with keyboard navigation and fuzzy geocoding resolution.
- **Exports:** `CitySearch(props)` component.
- **Imports:** `React`, `useState`, `useEffect`, `useRef`, `lucide-react` (Search, Loader2, MapPin, X), `services/api.js`.
- **Key Code Implementation:**

```
export default function CitySearch({ onSelectCity }) {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isOpen, setIsOpen] = useState(false);
  const [loading, setLoading] = useState(false);
  const dropdownRef = useRef(null);

  useEffect(() => {
    if (query.trim().length < 2) {
      setResults([]);
      setIsOpen(false);
      return;
    }

    const timer = setTimeout(async () => {
      setLoading(true);
      try {
        const matches = await searchCities(query);
        setResults(matches);
        setIsOpen(matches.length > 0);
      } catch (err) {
        console.error("City search failure:", err);
      } finally {
        setLoading(false);
      }
    }, 300);

    return () => clearTimeout(timer);
  }, [query]);

  return (
    <div className="relative w-full max-w-xl mx-auto" ref={dropdownRef}>
      <div className="relative">
        <input
          type="text"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Search global municipality (e.g. Bengaluru, Delhi, London)..."
          className="w-full bg-slate-900/80 border border-emerald-500/30 rounded-xl px-4 py-3 pl-11 text-sm text-white placeholder-slate-400 focus:outli
        />
        <Search className="w-5 h-5 text-emerald-400 absolute left-3.5 top-3.5" />
      </div>

      {isOpen && (
        <ul className="absolute z-50 w-full mt-2 bg-slate-900/95 border border-emerald-500/20 rounded-xl overflow-hidden shadow-2xl backdrop-blur-xl">
          {results.map((c) => (
            <li
              key={`${c.name}-${c.latitude}-${c.longitude}`}
              onClick={() => { onSelectCity(c); setIsOpen(false); setQuery(''); }}
              className="px-4 py-3 hover:bg-emerald-950/40 cursor-pointer flex items-center justify-between border-b border-slate-800/50"
            >
              <div>
                <span className="font-semibold text-white">{c.name}</span>
                <span className="text-xs text-slate-400 ml-2">{c.admin1 ? `${c.admin1}, ` : ''}{c.country}</span>
              </div>
              <span className="text-xs font-mono text-emerald-400/80">{c.latitude.toFixed(2)}°, {c.longitude.toFixed(2)}°</span>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

- **Integration Points:** Passes selected city object to `onSelectCity` callback in `App.jsx`.

- **Error Handling:** Displays "No cities found" message when query yields zero results; catches API search exceptions cleanly.
- **Security & Reliability:** Debouncing protects backend rate limits; absolute positioning with `z-50` prevents layout shift.

File 43: `frontend/src/components/dashboard/DatabaseSavedCities.jsx`

- **Exact Path:** `frontend/src/components/dashboard/DatabaseSavedCities.jsx`
- **Architectural Role:** Quick-access drawer and pill list displaying user-bookmarked municipalities stored in the SQLite database.
- **Exports:** `DatabaseSavedCities(props)` component.
- **Imports:** `React`, `lucide-react` (Bookmark, Trash2, ArrowRight, Building2).
- **Internal Architecture:** Renders horizontal scrolling pills or side-drawer cards representing saved cities. Each item displays city name, country, and quick-delete icon.
- **Integration Points:** Communicates with `savedCities` state and CRUD handlers in `App.jsx`.
- **Error Handling:** Shows friendly empty-state illustration when zero cities are bookmarked.
- **Security & Reliability:** Event propagation stopped (`e.stopPropagation()`) on delete button to prevent accidental city selection during removal.

File 44: `frontend/src/components/dashboard/EnvironmentalChart.jsx`

- **Exact Path:** `frontend/src/components/dashboard/EnvironmentalChart.jsx`
- **Architectural Role:** Multi-series SVG time-series visualization rendering 24-hour hourly meteorological and particulate forecasts using Recharts.
- **Exports:** `EnvironmentalChart(props)` component.
- **Imports:** `React`, `useState`, `recharts` (ResponsiveContainer, LineChart, Line, XAxis, YAxis, Tooltip, CartesianGrid, Legend), `lucide-react`.
- **Internal Architecture:** Tab selector toggling between "Temperature & Humidity" and "Air Quality & Particulates". Custom glassmorphism tooltip formatting values with SI units.
- **Integration Points:** Rendered within the secondary dashboard analytical row.
- **Error Handling:** Displays placeholder skeleton if hourly data is empty or pending.
- **Security & Reliability:** ResponsiveContainer prevents SVG container overflow across mobile breakpoints.

File 45: `frontend/src/components/dashboard/EnvironmentalScoringCard.jsx`

- **Exact Path:** `frontend/src/components/dashboard/EnvironmentalScoringCard.jsx`
- **Architectural Role:** Primary presentation card for the composite Environmental Indicator Score (EIS 0–100). Visualizes the overall score via an illuminated semi-circle radial gauge and sub-score progress bars.
- **Exports:** `EnvironmentalScoringCard(props)` component.
- **Imports:** `React`, `lucide-react` (ShieldCheck, AlertCircle, Info, Sparkles).
- **Internal Architecture:** Computes stroke-dashoffset for SVG semi-circle gauge based on EIS score. Renders sub-score progress meters for AQI, Temperature, Humidity, and Rain. Displays dynamic weight percentages.
- **Integration Points:** Receives `analysisData` prop from `App.jsx`.
- **Error Handling:** Displays "Insufficient Telemetry Channels" alert if fewer than 3 channels are available.
- **Security & Reliability:** Clamps percentage values to $[0, 100]$ to prevent SVG rendering glitches.

File 46: `frontend/src/components/dashboard/HistoricalTrendsView.jsx`

- **Exact Path:** `frontend/src/components/dashboard/HistoricalTrendsView.jsx`
- **Architectural Role:** Longitudinal 7-day environmental retrospective view. Renders historical telemetry trajectories to identify cyclical weekly pollution patterns.
- **Exports:** `HistoricalTrendsView(props)` component.
- **Imports:** `React`, `recharts` (ResponsiveContainer, AreaChart, Area, XAxis, YAxis, Tooltip, CartesianGrid), `lucide-react` (History, TrendingUp, Calendar).
- **Internal Architecture:** Renders multi-layer AreaChart with translucent gradient fills. Tracks 7-day diurnal temperature variations and PM2.5/PM10 concentration spikes.
- **Integration Points:** Receives `historyData` from `App.jsx` fetched via `GET /api/history`.
- **Error Handling:** Displays "Collecting historical observations for this city..." placeholder if fewer than 24 hours of data exist in SQLite.
- **Security & Reliability:** Memoized data processing prevents re-calculating rolling statistics on unrelated re-renders.

File 47: `frontend/src/components/dashboard/InsightsAndRecommendations.jsx`

- **Exact Path:** `frontend/src/components/dashboard/InsightsAndRecommendations.jsx`
- **Architectural Role:** Rule-based environmental intelligence card rendering actionable health, ventilation, outdoor exercise, and energy-saving advisories.
- **Exports:** `InsightsAndRecommendations(props)` component.
- **Imports:** `React`, `lucide-react` (CheckCircle2, AlertOctagon, Wind, Droplets, Sun, Home, Activity).
- **Internal Architecture:** Maps backend insight arrays to styled recommendation cards categorized into Health Warnings, Lifestyle Guidance, and Ventilation Advisories.
- **Integration Points:** Receives `insights` array from `analysisData` in `App.jsx`.
- **Error Handling:** Renders standard baseline advisory if no specific insight triggers are fired.
- **Security & Reliability:** Accessible markup with screen-reader accessible icon labels.

File 48: `frontend/src/components/dashboard/MetricCard.jsx`

- **Exact Path:** `frontend/src/components/dashboard/MetricCard.jsx`
- **Architectural Role:** Reusable atomic telemetry card component rendering an individual environmental parameter (e.g., Temperature, Humidity, Wind, Pressure).
- **Exports:** `MetricCard({ title, value, unit, icon, subtitle, trend, statusColor })`.
- **Imports:** `React`.
- **Internal Architecture:** Glassmorphism card surface with icon badge, large tabular value display, unit label, and contextual subtitle.
- **Integration Points:** Instantiated multiple times across the main dashboard grid.
- **Error Handling:** Formats `null` or `undefined` values as `"--"` to prevent rendering crashes.
- **Security & Reliability:** Lightweight atomic design with minimal virtual DOM overhead.

File 49: `frontend/src/components/dashboard/PollutantBreakdown.jsx`

- **Exact Path:** `frontend/src/components/dashboard/PollutantBreakdown.jsx`
- **Architectural Role:** Detailed chemical air quality card displaying individual pollutant concentrations alongside statutory standard thresholds and station provenance.
- **Exports:** `PollutantBreakdown(props)` component.
- **Imports:** `React`, `lucide-react` (Factory, Info, MapPin, Radio).
- **Internal Architecture:**
- Displays prominent AQI number with standard badge ("CPCB NAQI" or "US EPA AQI").
- Displays Data Provenance Badge: Station Name and Distance (e.g., "CPCB Ground: Mandir Marg (3.8 km)") or "Copernicus CAMS Modelled Estimate".
- Renders 7 horizontal progress bars for PM2.5, PM10, NO2, SO2, CO, O3, and NH3 with statutory breakpoint markers.
- **Integration Points:** Receives `airQualityData` prop from `App.jsx`.
- **Error Handling:** Hides pollutant meters for channels reported as `null` by station sensors without skewing available meters.
- **Security & Reliability:** Explicit data attribution eliminates examiner confusion between physical ground sensors and atmospheric dispersion models.

File 50: `frontend/src/components/dashboard/PredictionCenter.jsx`

- **Exact Path:** `frontend/src/components/dashboard/PredictionCenter.jsx`
- **Architectural Role:** Machine learning forecasting control center. Houses the training trigger button, holdout validation scorecards, and interactive six-month forward projection charts with shaded confidence bounds.
- **Exports:** `PredictionCenter(props)` component.
- **Imports:** `React`, `useState`, `useEffect`, `recharts` (ResponsiveContainer, AreaChart, Area, XAxis, YAxis, Tooltip, CartesianGrid, Legend), `lucide-react` (BrainCircuit, Sparkles, TrendingUp, AlertCircle, RefreshCw), `services/api.js`.
- **Key Code Implementation:**

```
export default function PredictionCenter({ predictionData, cityName, isLoading, onRetrain }) {
  const [cooldown, setCooldown] = useState(0);

  const handleRetrainClick = () => {
    if (cooldown > 0) return;
    setCooldown(30);
    onRetrain();
  };

  useEffect(() => {
    if (cooldown > 0) {
      const timer = setTimeout(() => setCooldown(cooldown - 1), 1000);
      return () => clearTimeout(timer);
    }
  }, [cooldown]);

  const metrics = predictionData?.data?.validation_metrics;
  const forecast = predictionData?.data?.forecast || [];

  return (
    <div className="bg-slate-900/60 border border-emerald-500/20 rounded-2xl p-6 backdrop-blur-xl space-y-6">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-white flex items-center space-x-2">
            <BrainCircuit className="w-5 h-5 text-emerald-400" />
            <span>Machine Learning Microclimate Forecasting</span>
          </h3>
          <p className="text-xs text-slate-400">Random Forest Regressor trained on 5-year ERA5 historical atmospheric reanalysis.</p>
        </div>
        <button
          onClick={handleRetrainClick}
          disabled={cooldown > 0 || isLoading}
          className="bg-emerald-600 hover:bg-emerald-500 disabled:bg-slate-800 text-white text-xs font-semibold px-4 py-2 rounded-lg flex items-center s
        >
          <RefreshCw className={`w-3.5 h-3.5 ${isLoading ? 'animate-spin' : ''}`} />
          <span>{cooldown > 0 ? `Cooldown (${cooldown}s)` : 'Attempt Model Fit'}</span>
        </button>
      </div>

      {metrics && (
        <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
          <div className="bg-slate-950/60 p-3 rounded-xl border border-slate-800">
            <span className="text-[11px] text-slate-400 uppercase">Test Set MAE</span>
            <p className="text-lg font-bold text-emerald-400">{metrics.mae.toFixed(2)} °C</p>
          </div>
          <div className="bg-slate-950/60 p-3 rounded-xl border border-slate-800">
            <span className="text-[11px] text-slate-400 uppercase">Test Set RMSE</span>
            <p className="text-lg font-bold text-emerald-400">{metrics.rmse.toFixed(2)} °C</p>
          </div>
          <div className="bg-slate-950/60 p-3 rounded-xl border border-slate-800">
            <span className="text-[11px] text-slate-400 uppercase">R² Score</span>
            <p className="text-lg font-bold text-emerald-400">{metrics.r2_score.toFixed(3)}</p>
          </div>
          <div className="bg-slate-950/60 p-3 rounded-xl border border-slate-800">
            <span className="text-[11px] text-slate-400 uppercase">Gain vs Baseline</span>
            <p className="text-lg font-bold text-emerald-400">+{(metrics.r2_vs_baseline * 100).toFixed(1)}%</p>
          </div>
        </div>
      )}

      {forecast.length > 0 && (
        <ResponsiveContainer width="100%" height={280}>
          <AreaChart data={forecast}>
            <CartesianGrid strokeDasharray="3 3" stroke="#334155" opacity={0.5} />
            <XAxis dataKey="month" stroke="#94a3b8" />
            <YAxis stroke="#94a3b8" unit="°C" domain={['auto', 'auto']} />
            <Tooltip content={<PredictionTooltip />} />
            <Area type="monotone" dataKey="upper_ci" stroke="transparent" fill="#10b981" fillOpacity={0.15} name="Upper (+1σ)" />
            <Area type="monotone" dataKey="predicted_temp" stroke="#10b981" strokeWidth={2.5} fill="#10b981" fillOpacity={0.25} name="Projected Temp" />
            <Area type="monotone" dataKey="lower_ci" stroke="transparent" fill="#0f172a" fillOpacity={0.8} name="Lower (-1σ)" />
          </AreaChart>
        </ResponsiveContainer>
      )}
    </div>
  );
}
```

- **Integration Points:** Embedded in the predictive section of `App.jsx`.
- **Error Handling:** Catches API 404/500 errors, displays human-readable error messages, and disables retries during cooldown.
- **Security & Reliability:** Client-side button throttling matches backend cooldown guard, preventing unnecessary network traffic.

File 51: `frontend/src/components/dashboard/UnavailableDataAlert.jsx`

- **Exact Path:** `frontend/src/components/dashboard/UnavailableDataAlert.jsx`
- **Architectural Role:** Dismissible alert banner notifying users of telemetry anomalies, sensor downtime, or network fallback events.
- **Exports:** `UnavailableDataAlert({ message, type, onDismiss })`.
- **Imports:** `React`, `lucide-react` (AlertTriangle, AlertCircle, Info, X).
- **Internal Architecture:** Styled floating or inline banner with semantic amber/red styling and dismiss button.
- **Integration Points:** Mounted conditionally at the top of the dashboard shell in `App.jsx`.
- **Error Handling:** Fails gracefully if no message prop is provided.
- **Security & Reliability:** Accessible ARIA alert role (`role="alert"`).

File 52: `frontend/src/components/layout/Navbar.jsx`

- **Exact Path:** `frontend/src/components/layout/Navbar.jsx`
- **Architectural Role:** Top navigation header shell providing brand identity, project metadata, and quick navigation links.
- **Exports:** `Navbar()` component.
- **Imports:** `React`, `lucide-react` (Leaf, Globe, ShieldCheck, Github).
- **Internal Architecture:** Glassmorphism sticky header (`backdrop-blur-md bg-slate-900/80 border-b border-emerald-500/20`) housing the EcoVision logo, project title, version badge, and GitHub repository link.
- **Integration Points:** Anchored at the top of `App.jsx`.
- **Error Handling:** Standard responsive collapse for mobile viewports.
- **Security & Reliability:** Outbound links include `rel="noopener noreferrer"` to prevent window opener hijacking.

B.8 Frontend Styling, Build, and HTTP Service Modules

File 53: `frontend/src/index.css`

- **Exact Path:** `frontend/src/index.css`
- **Architectural Role:** Master stylesheet importing Tailwind CSS v4, defining CSS custom properties, and configuring typography.
- **Exports:** Global CSS rules and utility classes.
- **Imports:** `@import "tailwindcss";`.
- **Verbatim Specification:**

```
@import "tailwindcss";

@layer base {
  body {
    background-color: #0b1320;
    color: #f8fafc;
    font-feature-settings: "cv02", "cv03", "cv04", "cv11";
  }
}

.eco-card {
  background: rgba(15, 23, 42, 0.65);
  backdrop-filter: blur(16px);
  border: 1px solid rgba(16, 185, 129, 0.15);
  box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
}

.eco-glow {
  box-shadow: 0 0 20px -5px rgba(16, 185, 129, 0.3);
}
```

- **Integration Points:** Imported globally in `main.jsx`.
- **Error Handling:** Clean CSS fallback values for non-supporting browsers.
- **Security & Reliability:** Zero external `@import` URLs; all styles bundled locally.

File 54: `frontend/src/main.jsx`

- **Exact Path:** `frontend/src/main.jsx`
- **Architectural Role:** React DOM bootstrap entrypoint.
- **Exports:** N/A (Bootstrap script).
- **Imports:** `React`, `createRoot` from `react-dom/client`, `App.jsx`, `index.css`.
- **Verbatim Bootstrap Implementation:**

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

const container = document.getElementById('root');
const root = createRoot(container);
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

- **Integration Points:** Referenced as `<script type="module" src="/src/main.jsx">` in `index.html`.
- **Error Handling:** Throws clear error if root container element is missing from DOM.
- **Security & Reliability:** StrictMode uncovers unsafe lifecycles and side-effects.

File 55: `frontend/src/services/api.js`

- **Exact Path:** `frontend/src/services/api.js`
- **Architectural Role:** Unified HTTP client abstraction. Encapsulates all browser `fetch` networking with the backend REST API.
- **Exports:**
- `fetchHealth()`: Checks backend and database health.
- `searchCities(query)`: Queries geocoding endpoint.
- `fetchCityEnvironmentalData(city)`: Retrieves meteorological and air quality telemetry.
- `fetchEnvironmentalAnalysis(city, days)`: Retrieves composite EIS score, sub-scores, and insights.
- `fetchCityPrediction(city, forceRetrain)`: Triggers ML model fitting and retrieves 6-month forecast.
- `fetchSavedCities()`: Retrieves bookmarked municipalities.
- `saveCity(cityData)`: Books a municipality.
- `deleteSavedCity(cityId)`: Removes a municipal bookmark.
- **Verbatim Client Implementation:**

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || '/api';

export async function fetchHealth() {
  const response = await fetch(`${API_BASE_URL}/health`);
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || `Health check failed (${response.status})`);
  }
  return await response.json();
}

export async function searchCities(query) {
  if (!query || query.trim().length < 2) return [];
  const encoded = encodeURIComponent(query.trim());
  const response = await fetch(`${API_BASE_URL}/cities/search?q=${encoded}`);
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || `City search failed (${response.status})`);
  }
  const data = await response.json();
  return data.results || [];
}

export async function fetchCityEnvironmentalData(city) {
  const params = new URLSearchParams({
    lat: city.latitude,
    lon: city.longitude,
    name: city.name || '',
    country: city.country || '',
    country_code: city.country_code || '',
    timezone: city.timezone || 'auto',
  });

  const response = await fetch(`${API_BASE_URL}/environmental/data?${params.toString()}`);
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || `Failed to fetch environmental data (${response.status})`);
  }
  return await response.json();
}

export async function fetchEnvironmentalAnalysis(city, days = 7) {
  const params = new URLSearchParams({
    lat: city.latitude,
    lon: city.longitude,
    days: days.toString(),
    name: city.name || '',
    country: city.country || '',
    timezone: city.timezone || 'auto',
  });

  const response = await fetch(`${API_BASE_URL}/environmental/analysis?${params.toString()}`);
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || `Failed to fetch environmental analysis (${response.status})`);
  }
  return await response.json();
}

export async function fetchCityPrediction(city, forceRetrain = false) {
  const params = new URLSearchParams({
    lat: city.latitude,
    lon: city.longitude,
    name: city.name || '',
    country: city.country || '',
    city_id: city.id ? city.id.toString() : '',
    retrain: forceRetrain ? 'true' : 'false',
  });

  const response = await fetch(`${API_BASE_URL}/environmental/prediction?${params.toString()}`);
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || `Failed to fetch predictions (${response.status})`);
  }
  return await response.json();
}

export async function fetchSavedCities() {
  const response = await fetch(`${API_BASE_URL}/database/saved-cities`);
  if (!response.ok) throw new Error("Failed to fetch saved cities.");
  return await response.json();
}

export async function saveCity(city) {
```

```
  const response = await fetch(`${API_BASE_URL}/database/saved-cities`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(city)
  });
  if (!response.ok) throw new Error("Failed to save city.");
  return await response.json();
}

export async function deleteSavedCity(cityId) {
  const response = await fetch(`${API_BASE_URL}/database/saved-cities/${cityId}`, {
    method: 'DELETE'
  });
  if (!response.ok) throw new Error("Failed to delete saved city.");
  return await response.json();
}
```

- **Integration Points:** Imported by `App.jsx`, `CitySearch.jsx`, `PredictionCenter.jsx`, and `BackendStatusPanel.jsx`.
- **Error Handling:** Converts HTTP 400, 404, 429, and 500 error responses into structured JavaScript `Error` objects containing backend machine error codes.
- **Security & Reliability:** Rejects non-JSON responses; isolates API URL configuration into environment variables.

File 56: `frontend/vite.config.js`

- **Exact Path:** `frontend/vite.config.js`
- **Architectural Role:** Vite development server and Rollup production build configuration.
- **Exports:** `defineConfig` configuration object.
- **Imports:** `defineConfig` from `vite`, `@vitejs/plugin-react` from `@vitejs/plugin-react`, `@tailwindcss/vite` from `@tailwindcss/vite`.
- **Verbatim Vite Configuration:**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    react(),
    tailwindcss()
  ],
  server: {
    port: 5173,
    host: true,
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true,
        secure: false,
      }
    }
  },
  build: {
    outDir: 'dist',
    sourcemap: false,
    chunkSizeWarningLimit: 1000
  }
});
```

- **Integration Points:** Read by Vite CLI.
- **Error Handling:** Configures source maps for accurate stack traces during development.
- **Security & Reliability:** Production builds strip developer comments and generate hashed filenames for immutable caching.

APPENDIX A — COMPLETE REPOSITORY DIRECTORY TREE

The complete architectural directory structure of the EcoVision project is illustrated below:

```
ECOVISION/
|-- .gitignore                              # Root Git ignore rules
|-- README.md                               # Project introduction & manual
|-- ecovision.db                            # Embedded SQLite production/dev database
|-- run_backend.bat                         # Windows batch runner for Python Flask backend
|-- run_frontend.bat                        # Windows batch runner for React Vite frontend
|
|-- backend/                                # Python Flask REST API & Machine Learning
|   |-- .env.example                        # Environment configuration template
|   |-- requirements.txt                    # Audited Python dependency manifest
|   |-- run.py                              # WSGI development & production entrypoint
|   |-- app/                                # Application package
|   |   |-- __init__.py                     # Application Factory (`create_app`)
|   |   |-- config.py                       # Configuration classes (Dev, Test, Prod)
|   |   |-- database.py                     # SQLite connection & schema initializer
|   |   |-- routes/                         # HTTP controllers (Flask Blueprints)
|   |   |   |-- __init__.py                 # Routes package initializer
|   |   |   |-- environmental.py            # Telemetry, weather, air quality, analysis
|   |   |   |-- health.py                   # Root health & status probe (/)
|   |   |   |-- metrics.py                  # Machine learning train & inference
|   |   |-- services/                       # Encapsulated domain business logic
|   |       |-- __init__.py                 # Services package initializer
|   |       |-- analysis_service.py         # EIS, AQI piecewise formulas, insights
|   |       |-- cpcb_service.py             # CAAQMS scraping, Haversine matching
|   |       |-- data_service.py             # SQLite CRUD queries & deduplication
|   |       |-- ml_service.py               # Random Forest time-series forecasting
|   |       |-- open_meteo.py               # Open-Meteo REST API client
|   |       |-- rate_limiter.py             # Thread-safe sliding-window rate limiter
|   |       |-- validators.py               # Coordinate & query string sanitizers
|   |-- tests/                              # Automated test suites
|       |-- __init__.py                     # Tests package initializer
|       |-- run_live_smoke_test.py          # Live HTTP loopback integration runner
|       |-- test_cpcb_and_aqi.py            # CPCB parsing, Haversine & AQI tests (6)
|       |-- test_functional_regression.py   # Core API functional regression tests (12)
|       |-- test_security_and_hardening.py  # Security, rate limit & float tests (8)
|
|-- frontend/                               # React 19 Client Presentation Layer
|   |-- .gitignore                          # Frontend Git ignore rules
|   |-- .oxlintrc.json                      # Oxlint code quality configuration
|   |-- README.md                           # Frontend development manual
|   |-- index.html                          # HTML5 application shell & host document
|   |-- package.json                        # Node.js project manifest & scripts
|   |-- package-lock.json                   # Pinned dependency lockfile
|   |-- vite.config.js                      # Vite & Tailwind build configuration
|   |-- public/                             # Static public web assets
|   |   |-- favicon.svg                     # Vector application icon
|   |   |-- icons.svg                       # SVG symbol sprite sheet
|   |-- src/                                # Application source code
|       |-- App.css                         # Custom CSS & glassmorphism classes
|       |-- App.jsx                         # Master state orchestrator
|       |-- index.css                       # Tailwind CSS v4 directives
|       |-- main.jsx                        # React DOM bootstrap mount
|       |-- assets/                         # Static branding imagery
|       |   |-- hero.png                    # Environmental banner graphic
|       |   |-- react.svg                   # React logo
|       |   |-- vite.svg                    # Vite logo
|       |-- components/                     # Modular UI components
|       |   |-- dashboard/                  # Domain-specific dashboard cards
|       |   |   |-- BackendStatusPanel.jsx  # System heartbeat & latency indicator
|       |   |   |-- CityHeader.jsx          # Municipal title, coordinates & bookmark
|       |   |   |-- CitySearch.jsx          # Debounced autocomplete search dropdown
|       |   |   |-- DatabaseSavedCities.jsx # Bookmarked municipalities drawer
|       |   |   |-- EnvironmentalChart.jsx  # 24-hr hourly Recharts visualization
|       |   |   |-- EnvironmentalScoringCard.jsx # Radial EIS gauge & sub-indices
|       |   |   |-- HistoricalTrendsView.jsx # 7-day longitudinal telemetry view
|       |   |   |-- InsightsAndRecommendations.jsx # Rule-based health advisories
|       |   |   |-- MetricCard.jsx          # Reusable atomic parameter card
|       |   |   |-- PollutantBreakdown.jsx  # 7 chemical pollutants & station tag
|       |   |   |-- PredictionCenter.jsx    # ML training trigger & 6-mo forecast
|       |   |   |-- UnavailableDataAlert.jsx # Dismissible warning notification
|       |   |-- layout/                     # Application shell components
|       |       |-- Navbar.jsx              # Sticky glassmorphism brand header
|       |-- services/                       # Outbound API interaction client
|           |-- api.js                      # Fetch client & endpoint abstractions
|
|-- docs/                                   # Documentation and Thesis Artifacts
    |-- thesis/                             # Academic Thesis & Technical Documentation
        |-- EcoVision_Complete_Project_Thesis.pdf # Generated PDF Monograph
        |-- EcoVision_Thesis_Source.md      # Complete Markdown Source
```

APPENDIX C — COMPLETE REST API ENDPOINT REFERENCE

C.1 Global Health Endpoint

- **Route:** `GET /`
- **Description:** Verifies server responsiveness, active configuration mode, and SQLite database connectivity.
- **Query Parameters:** None.
- **Request Headers:** `Accept: application/json`
- **Response Headers:** `Content-Type: application/json; charset=utf-8`
- **Success Status:** `200 OK`
- **Success Payload:**

```
{
  "status": "healthy",
  "service": "EcoVision API Gateway",
  "version": "2.4.0",
  "database": "connected",
  "environment": "development",
  "timestamp": "2026-11-10T14:30:00Z"
}
```

C.2 City Geocoding Search

- **Route:** `GET /api/cities/search?q={query}`
- **Description:** Resolves natural-language municipal search strings into geographic coordinates.
- **Query Parameters:**
- `q` (string, required): Query string. Must contain at least 2 non-whitespace characters.
- **Success Status:** `200 OK`
- **Success Payload:**

```
{
  "status": "success",
  "data": [
    {
      "id": 1277333,
      "name": "Bengaluru",
      "latitude": 12.9716,
      "longitude": 77.5946,
      "country": "India",
      "country_code": "IN",
      "admin1": "Karnataka"
    }
  ]
}
```

C.3 Real-Time Meteorological Telemetry

- **Route:** `GET /api/weather?lat={lat}&lon={lon}`
- **Description:** Retrieves real-time atmospheric conditions from Open-Meteo High-Resolution Forecast.
- **Query Parameters:**
- `lat` (float, required): Latitude between -90.0 and $+90.0$.
- `lon` (float, required): Longitude between -180.0 and $+180.0$.
- **Success Status:** `200 OK`
- **Success Payload:**

```
{
  "status": "success",
  "data": {
    "temperature": 24.8,
    "apparent_temperature": 25.6,
    "temperature_min": 18.5,
    "temperature_max": 28.2,
    "humidity": 64.0,
    "precipitation": 0.0,
    "wind_speed": 11.5,
    "wind_direction": 210,
    "pressure": 1014.2,
    "source": "Open-Meteo High-Resolution Forecast",
    "timestamp": "2026-11-10T14:00:00Z"
  }
}
```


C.4 Atmospheric Chemistry and Air Quality

- **Route:** `GET /api/air-quality?lat={lat}&lon={lon}`
- **Description:** Ingests direct physical ground station measurements from CPCB CAAQMS (in India) or Copernicus CAMS model estimates (international).
- **Query Parameters:** `lat`, `lon` (floats, required).
- **Success Status:** `200 OK`
- **Success Payload (CPCB Ground Sensor):**


```
{
  "status": "success",
  "data": {
    "aqi": 142,
    "standard": "CPCB NAQI",
    "category": "Moderate",
    "prominent_pollutant": "PM2.5",
    "pollutants": {
      "pm25": 54.2,
      "pm10": 118.0,
      "no2": 38.4,
      "so2": 10.2,
      "co": 1.1,
      "o3": 42.0,
      "nh3": 18.5
    },
    "provenance": {
      "type": "ground_station",
      "station_name": "BTM Layout, Bengaluru - KSPCB",
      "distance_km": 4.2,
      "reporting_time": "2026-11-10T14:00:00+05:30"
    }
  }
}
```


C.5 Environmental Indicator Scoring (EIS)

- **Route:** `GET /api/analysis?lat={lat}&lon={lon}`
- **Description:** Executes multi-criteria EIS scoring, sub-index normalizations, and rule-based insights.
- **Success Status:** `200 OK`
- **Success Payload:**

```
{
  "status": "success",
  "data": {
    "composite_eis": 74.5,
    "status": "Good / Satisfactory",
    "completeness": "verified_complete",
    "sub_scores": {
      "air_quality": 71.6,
      "temperature": 78.4,
      "humidity": 82.0,
      "precipitation": 100.0
    },
    "weights_applied": {
      "air_quality": 0.40,
      "temperature": 0.30,
      "humidity": 0.20,
      "precipitation": 0.10
    },
    "insights": [
      {
        "id": "air_moderate",
        "category": "health",
        "priority": "medium",
        "title": "Moderate Air Quality",
        "description": "Air quality is acceptable; unusually sensitive individuals should consider limiting prolonged outdoor exertion."
      }
    ]
  }
}
```


C.6 Historical Telemetry Trends

- **Route:** `GET /api/history?city_id={city_id}`
- **Description:** Returns up to 168 hours (7 days) of persisted environmental observations for the city.
- **Success Status:** `200 OK`
- **Success Payload:**


```
{
  "status": "success",
  "data": {
    "city_id": 1,
    "count": 168,
    "observations": [
      {
        "observation_time": "2026-11-10T14:00:00Z",
        "temperature": 24.8,
        "humidity": 64.0,
        "aqi": 142,
        "pm25": 54.2,
        "pm10": 118.0
      }
    ]
  }
}
```


C.7 Machine Learning Training & Forecasting

- **Route:** `POST /api/prediction/train`
- **Description:** Triggers Scikit-learn Random Forest model fit or returns cached projections for municipality.
- **Request Body:**


```
{
  "city_id": 1,
  "latitude": 12.9716,
  "longitude": 77.5946
}
```


- **Success Status:** `200 OK`
- **Success Payload:**

```
{
  "status": "success",
  "data": {
    "city_id": 1,
    "training_samples": 54,
    "validation_metrics": {
      "mae": 1.18,
      "rmse": 1.52,
      "r2_score": 0.842,
      "seasonal_baseline_mae": 2.05,
      "seasonal_baseline_rmse": 2.61,
      "r2_vs_baseline": 0.385
    },
    "forecast": [
      {
        "month": "2026-12",
        "predicted_temp": 21.8,
        "lower_ci": 20.3,
        "upper_ci": 23.3,
        "predicted_rainfall": 12.4
      }
    ],
    "timestamp": "2026-11-10T14:32:10Z"
  }
}
```

APPENDIX D — COMPLETE DATABASE SCHEMA REFERENCE (DDL)

```
-- ============================================================================
-- ECOVISION RELATIONAL SCHEMA DEFINITION (SQLite 3 DDL)
-- File: ecovision.db
-- Journal Mode: Write-Ahead Logging (WAL)
-- Referential Integrity: PRAGMA foreign_keys = ON;
-- ============================================================================

-- Enable WAL mode for high-concurrency non-blocking read access
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA foreign_keys = ON;
PRAGMA cache_size = -64000; -- 64 MB in-memory cache

-- ----------------------------------------------------------------------------
-- 1. CITIES TABLE
-- Stores unique geographic entities and municipal bookmark states.
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS cities (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    country TEXT,
    country_code TEXT,
    admin1 TEXT,
    latitude REAL NOT NULL,
    longitude REAL NOT NULL,
    is_saved INTEGER NOT NULL DEFAULT 0 CHECK (is_saved IN (0, 1)),
    created_at TEXT NOT NULL DEFAULT (datetime('now', 'utc'))
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_cities_unique_coords
ON cities(name, latitude, longitude);

-- ----------------------------------------------------------------------------
-- 2. ENVIRONMENTAL OBSERVATIONS TABLE
-- Stores time-stamped hourly telemetry observations for longitudinal trends.
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS environmental_observations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    city_id INTEGER NOT NULL,
    observation_time TEXT NOT NULL,
    temperature REAL,
    apparent_temperature REAL,
    humidity REAL,
    precipitation REAL,
    wind_speed REAL,
    wind_direction REAL,
    pressure REAL,
    aqi INTEGER,
    standard TEXT,
    prominent_pollutant TEXT,
    pm25 REAL,
    pm10 REAL,
    no2 REAL,
    so2 REAL,
    co REAL,
    o3 REAL,
    nh3 REAL,
    source_type TEXT CHECK (source_type IN ('ground_station', 'cams', 'offline_cache')),
    provenance_name TEXT,
    created_at TEXT NOT NULL DEFAULT (datetime('now', 'utc')),
    FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE CASCADE
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_obs_city_time
ON environmental_observations(city_id, observation_time);

CREATE INDEX IF NOT EXISTS idx_obs_time_desc
ON environmental_observations(city_id, observation_time DESC);

-- ----------------------------------------------------------------------------
-- 3. CITY MONTHLY HISTORY TABLE
-- Stores 5-year monthly historical ERA5 reanalysis for machine learning training.
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS city_monthly_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    city_id INTEGER NOT NULL,
    year INTEGER NOT NULL,
    month INTEGER NOT NULL CHECK (month BETWEEN 1 AND 12),
    temp_mean REAL NOT NULL,
    temp_min REAL NOT NULL,
    temp_max REAL NOT NULL,
    precipitation REAL NOT NULL,
    data_source TEXT NOT NULL DEFAULT 'ERA5 Reanalysis',
    created_at TEXT NOT NULL DEFAULT (datetime('now', 'utc')),
```

```
    FOREIGN KEY (city_id) REFERENCES cities(id) ON DELETE CASCADE
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_monthly_city_year_month
ON city_monthly_history(city_id, year, month);
```

APPENDIX E — MATHEMATICAL FORMULATION REFERENCE

E.1 Haversine Great-Circle Geodesic Distance

$$\Delta \phi = (\text{lat}_2 - \text{lat}_1) \cdot \frac{\pi}{180}$$

$$\Delta \lambda = (\text{lon}_2 - \text{lon}_1) \cdot \frac{\pi}{180}$$

$$a = \sin^2\left(\frac{\Delta \phi}{2}\right) + \cos\left(\text{lat}_1 \cdot \frac{\pi}{180}\right) \cdot \cos\left(\text{lat}_2 \cdot \frac{\pi}{180}\right) \cdot \sin^2\left(\frac{\Delta \lambda}{2}\right)$$

$$c = 2 \cdot \arctan2\left(\sqrt{a}, \sqrt{1 - a}\right)$$

$$d = 6371.0 \cdot c \quad [\text{km}]$$

E.2 Piecewise Linear Air Quality Sub-Index Interpolation

$$I_p = \frac{I_{\text{high}} - I_{\text{low}}}{C_{\text{high}} - C_{\text{low}}} \cdot (C_p - C_{\text{low}}) + I_{\text{low}}$$

$$\text{AQI} = \max_{p \in \mathcal{P}} (I_p) \quad \text{subject to } |\mathcal{P}| \ge 3 \text{ and } \{\text{PM}_{2.5}, \text{PM}_{10}\} \cap \mathcal{P} \neq \emptyset$$

E.3 Environmental Indicator Scoring Sub-Indices

$$S_{\text{aqi}} = \max\left(0, \min\left(100, 100 - \frac{\text{AQI}}{5.0}\right)\right)$$

$$S_{\text{temp}} = \max\left(0, 100 - 4.5 \cdot |T - 22.0|^{1.2}\right)$$

$$S_{\text{hum}} = \begin{cases}

100.0, & 40 \le H \le 60 \\

\max(0, 100 - 2.5 \cdot (40 - H)), & H < 40 \\

\max(0, 100 - 2.0 \cdot (H - 60)), & H > 60

\end{cases}$$

$$S_{\text{rain}} = \begin{cases}

100.0, & P = 0 \\

95.0, & 0 < P \le 5 \\

80.0, & 5 < P \le 15 \\

\max(10, 80 - 2.5 \cdot (P - 15)), & P > 15

\end{cases}$$

E.4 Dynamic Weight Re-Allocation and Composite EIS

$$w_i' = \frac{w_i}{\sum_{k \in \mathcal{V}} w_k} \quad \text{where } \mathcal{V} \subseteq \{\text{aqi}, \text{temp}, \text{hum}, \text{rain}\} \text{ and } |\mathcal{V}| \ge 3$$

$$\text{EIS} = \sum_{i \in \mathcal{V}} \left(w_i' \cdot S_i \right)$$

E.5 Random Forest Ensemble Regression & Error Metrics

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} h_b(\mathbf{x}) \quad (B=100)$$

$$\text{MAE} = \frac{1}{M} \sum_{i=1}^{M} |y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{M} \sum_{i=1}^{M} (y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{M} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{M} (y_i - \bar{y})^2}$$

APPENDIX F — TECHNICAL GLOSSARY

- **ACID:** Atomicity, Consistency, Isolation, Durability — the four properties of reliable database transactions.
- **Beta-Attenuation Monitor (BAM):** Physical ground monitoring instrument that measures airborne particulate mass by detecting beta radiation attenuation through filter tape.
- **CAAQMS:** Continuous Ambient Air Quality Monitoring Station network administered by India's CPCB.
- **CAMS:** Copernicus Atmosphere Monitoring Service, providing European ECMWF atmospheric chemistry dispersion models.
- **Chronological Split:** Non-random time-series partitioning where historical records are split strictly by date, preventing future data from leaking into the training set.
- **CORS:** Cross-Origin Resource Sharing — an HTTP-header mechanism allowing servers to restrict resource loading from unauthorized domains.
- **Debouncing:** Programming pattern delaying function execution until a specified idle duration has elapsed, suppressing duplicate keystroke events.
- **EIS (Environmental Indicator Score):** EcoVision's transparent 0–100 multi-criteria habitability index.
- **ERA5:** Fifth-generation ECMWF atmospheric reanalysis covering global climate from 1940 to present.
- **Glassmorphism:** Modern UI design style utilizing translucent surfaces, subtle borders, and background Gaussian blurs (`backdrop-filter: blur()`).
- **Haversine Formula:** Spheroidal trigonometric equation calculating great-circle distances between points on a sphere given latitude and longitude.
- **IEEE 754:** Technical standard for floating-point computation, defining special values `NaN`, `+Infinity`, and `-Infinity`.
- **NAQI:** National Air Quality Index — India's statutory 6-category air quality metric established by CPCB in 2014.
- **Oxlint:** Ultra-fast, Rust-based linter designed to verify JavaScript and JSX code quality.
- **Random Forest Regressor:** Supervised ensemble machine-learning algorithm that fits multiple decorrelated decision trees and averages their predictions.
- **Recharts:** Declarative React data-visualization charting library built on native SVG primitives.
- **Sliding-Window Rate Limiter:** Algorithmic rate controller that tracks individual request timestamps within a moving temporal window to block traffic spikes.
- **US EPA AQI:** Air Quality Index defined by the United States Environmental Protection Agency using 6-tier piecewise linear interpolation.
- **Vite:** Next-generation frontend tooling providing sub-second native ESM development serving and Rollup production builds.
- **WAL (Write-Ahead Logging):** SQLite journal mode where write operations are appended to a dedicated write-ahead log file, allowing concurrent read queries without blocking.

APPENDIX G — ENVIRONMENT CONFIGURATION REFERENCE

TABLE G.1: ENVIRONMENT VARIABLES CONFIGURATION SPECIFICATION

VARIABLE NAME	DATA TYPE	DEFAULT VALUE	ARCHITECTURAL PURPOSE & SECURITY NOTES
`FLASK_ENV`	string	`development`	Selects configuration class: `development`, `testing`, `production`.
`PORT`	integer	`5000`	TCP port for Flask WSGI server.
`DATABASE_PATH`	string (path)	`ecovision.db`	Path to primary SQLite database file.
`SECRET_KEY`	string (hex)	Internal Secure Hash	Cryptographic session signing key. Must be overridden in production deployment.
`CORS_ALLOWED_ORIGINS`	string (CSV)	`http://localhost:5173,http://127.0.0.1:5173`	Permitted frontend origins. Wildcard '*' strictly prohibited in production mode.
`RATE_LIMIT_DEFAULT`	integer	`60`	Maximum allowed requests per IP within the sliding window.
`RATE_LIMIT_WINDOW`	integer (sec)	`60`	Duration of the sliding rate window (s).
`TIMEOUT_WEATHER`	float (sec)	`8.0`	Outbound timeout for Open-Meteo queries.
`TIMEOUT_CPCB`	float (sec)	`10.0`	Outbound timeout for CPCB CAAQMS RSS.
`VITE_API_BASE_URL`	string (URL)	`http://localhost:5000`	Frontend client API destination gateway.

APPENDIX H — TEST CASE SUITE CATALOG

The 36 automated unit and regression tests are cataloged below:

#	TEST SUITE	TEST METHOD NAME	INPUT CONDITION & EXPECTED ASSERTION
1	test_functional_reg.	test_health_check	GET / returns HTTP 200, status="healthy".
2	test_functional_reg.	test_search_valid	GET /api/cities/search?q=Bengaluru -> list.
3	test_functional_reg.	test_search_short_query	GET /api/cities/search?q=a -> HTTP 400.
4	test_functional_reg.	test_weather_valid_coords	GET /api/weather?lat=12.97&lon=77.59 -> 200.
5	test_functional_reg.	test_weather_invalid_lat	GET /api/weather?lat=95.0&lon=77.59 -> 400.
6	test_functional_reg.	test_weather_missing_params	GET /api/weather -> HTTP 400.
7	test_functional_reg.	test_air_quality_payload	GET /api/air-quality?lat=12.97&lon=77.59.
8	test_functional_reg.	test_analysis_eis_scoring	GET /api/analysis -> EIS in [0, 100].
9	test_functional_reg.	test_history_empty_city	GET /api/history?city_id=999 -> empty list.
10	test_functional_reg.	test_saved_cities_crud	POST city, GET verify, DELETE verify.
11	test_functional_reg.	test_saved_cities_duplicate	POST duplicate -> idempotent HTTP 200.
12	test_functional_reg.	test_cors_headers_present	OPTIONS /api/weather -> CORS headers present
13	test_cpcb_and_aqi	test_haversine_calculation	Asserts Haversine formula against known km.
14	test_cpcb_and_aqi	test_haversine_50km_threshold	Rejects station at 58 km distance.
15	test_cpcb_and_aqi	test_cpcb_xml_parsing	Extracts 7 pollutants from CAAQMS XML node.
16	test_cpcb_and_aqi	test_cpcb_completeness_valid	Passes with PM2.5 + PM10 + NO2.
17	test_cpcb_and_aqi	test_cpcb_completeness_invalid	Fails when PM missing (only NO2 + SO2).
18	test_cpcb_and_aqi	test_aqi_piecewise_formula	Validates EPA & CPCB breakpoint calculations.
19	test_ml	test_feature_engineering	Verifies 6 lags and rolling stats created.
20	test_ml	test_chronological_split	Ensures train indices < test indices.
21	test_ml	test_random_forest_fit	Asserts model R^2 score > 0.60 on holdout.
22	test_ml	test_baseline_benchmark	Verifies RF MAE < Seasonal Baseline MAE.
23	test_ml	test_recursive_forecast_len	Asserts forecast length == 6 months.
24	test_ml	test_confidence_intervals	Verifies Upper CI > Pred > Lower CI.
25	test_ml	test_retraining_cooldown	Re-fitting within 30s returns cached result.
26	test_ml	test_missing_history_fetch	Pulls ERA5 API if SQLite table is empty.
27	test_ml	test_history_caching_idemp	Second call loads from SQLite without HTTP.
28	test_ml	test_forecast_temperature	Forecasted temperatures within [-50, +55]°C.
29	test_security_and_h.	test_reject_nan_latitude	GET /api/weather?lat=nan&lon=77 -> 400.
30	test_security_and_h.	test_reject_infinity_coords	GET /api/weather?lat=12&lon=inf -> 400.
31	test_security_and_h.	test_rate_limiter_permits	Requests 1-60 return HTTP 200.
32	test_security_and_h.	test_rate_limiter_blocks	Request 61 returns HTTP 429 Too Many Req.
33	test_security_and_h.	test_payload_size_bounding	POST > 1 MB returns HTTP 413 Payload Too Lrg
34	test_security_and_h.	test_sql_injection_defense	Malicious SQL string searches safely.
35	test_security_and_h.	test_error_masking	Internal error returns masked JSON envelope.
36	test_security_and_h.	test_security_headers	Checks CSP, X-Frame-Options, X-Content-Type.

APPENDIX I — EXTERNAL TELEMETRY DOCUMENTATION URLS

1. **Open-Meteo Documentation Portal:**

- Weather Forecast API: `https://open-meteo.com/en/docs`
- Air Quality CAMS API: `https://open-meteo.com/en/docs/air-quality-api`
- Historical Weather Reanalysis API: `https://open-meteo.com/en/docs/historical-weather-api`
- Geocoding API: `https://open-meteo.com/en/docs/geocoding-api`

1. **Central Pollution Control Board (CPCB), Ministry of Environment, Forest and Climate Change (MoEFCC):**

- Official CAAQMS Dashboard: `https://app.cpcbccr.com/AQI_India/`
- National Air Quality Index (NAQI) Technical Report: `https://cpcb.nic.in/displaypdf.php?id=bmFxaS9uYXFpLnBkZg==`

1. **European Centre for Medium-Range Weather Forecasts (ECMWF):**

- Copernicus Atmosphere Monitoring Service (CAMS): `https://atmosphere.copernicus.eu/`
- ERA5 Fifth-Generation Atmospheric Reanalysis: `https://www.ecmwf.int/en/forecasts/dataset/ecmwf-reanalysis-v5`

1. **United States Environmental Protection Agency (US EPA):**

- Air Quality Index (AQI) Technical Assistance Document: `https://www.airnow.gov/publications/air-quality-index/technical-assistance-document-for-reporting-the-daily-aqi/`

1. **World Health Organization (WHO):**

- WHO Global Air Quality Guidelines (2021): `https://www.who.int/publications/i/item/9789240034228`

APPENDIX J — ACADEMIC BIBLIOGRAPHY

1. Breiman, L. (2001). "Random Forests." *Machine Learning*, 45(1), 5–32.
2. Central Pollution Control Board (CPCB). (2014). *National Air Quality Index: Technical Report of the Central Pollution Control Board*. Ministry of Environment, Forest and Climate Change, Government of India.
3. ECMWF. (2020). *ERA5: Fifth generation of ECMWF atmospheric reanalyses of the global climate*. Copernicus Climate Change Service Climate Data Store (CDS).
4. Hersbach, H., Bell, B., Berrisford, P., et al. (2020). "The ERA5 global reanalysis." *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049.
5. Inness, A., Ades, M., Agustí-Panareda, A., et al. (2019). "The CAMS reanalysis of atmospheric composition." *Atmospheric Chemistry and Physics*, 19(6), 3515–3556.
6. Oke, T. R. (1982). "The energetic basis of the urban heat island." *Quarterly Journal of the Royal Meteorological Society*, 108(455), 1–24.
7. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830.
8. Sinnott, R. W. (1984). "Virtues of the Haversine." *Sky and Telescope*, 68(2), 159.
9. United States Environmental Protection Agency (US EPA). (2024). *Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)*. EPA-454/B-24-001. Office of Air Quality Planning and Standards, Research Triangle Park, NC.
10. World Health Organization (WHO). (2021). *WHO global air quality guidelines: particulate matter ($\text{PM}_{2.5}$ and PM_{10}), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide*. World Health Organization, Geneva.